# 03 Matplotlib

## Plan for this lecture

1. Matplotlib Introduction
2. Functional approach
3. OOP approach
4. Exercises

## Introduction to Matplotlib

- Matplotlib is a a package that plots visualisations in Python.
- It can generate Line plots, Histogram, Scatter Plots, 3D plots and plot images by mapping pixels to coordinates on x and y axes.
- It provides an Object-Oriented API – applied by Tkinter– and has also kept its legacy procedural “pylab” interface, which resembles MATLAB from the 1980s.
- More documentation available at: https://matplotlib.org/stable/plot_types/index.html

## Import

In [1…
```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

In [1…
```
plt
```

Out[1…
```
<module 'matplotlib.pyplot' from '/Users/nick/Library/Python/3.9/lib/python/site-packages/matplotlib/pyplot.py'>
```

## Using plt inline

In [1…
```
# display matplotlib plots inside the notebook
%matplotlib inline
```

### Check we have access to `np` and `pd` as well...

In [1…
```
import numpy as np
import pandas as pd

#pd.set_option('precision', 6)
#pd.options.display.float_format = '{:.2f}'.format
```

In [1…
```
np
```

Out[1…
```
<module 'numpy' from '/Users/nick/Library/Python/3.9/lib/python/site-packages/numpy/__init__.py'>
```

In [1…
```
pd
```

Out[1…
```
<module 'pandas' from '/Users/nick/Library/Python/3.9/lib/python/site-packages/pandas/__init__.py'>
```

## Functional Plot

In [7…
```
x = np.arange(1, 10, 1)
x
```

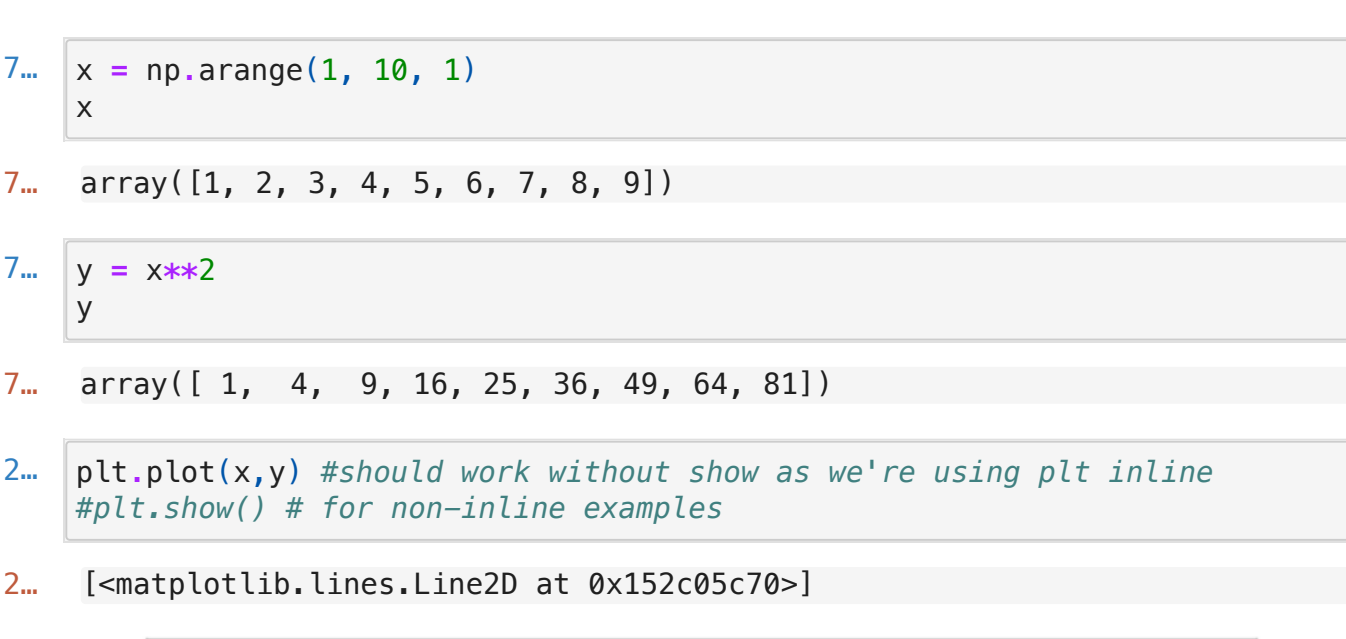
Out[7…
```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [7…
```
y = x**2
y
```

Out[7…
```
array([ 1,  4,  9, 16, 25, 36, 49, 64, 81])
```

In [2…
```
plt.plot(x,y) #should work without show as we're using plt inline
#plt.show() # for non-inline examples
```

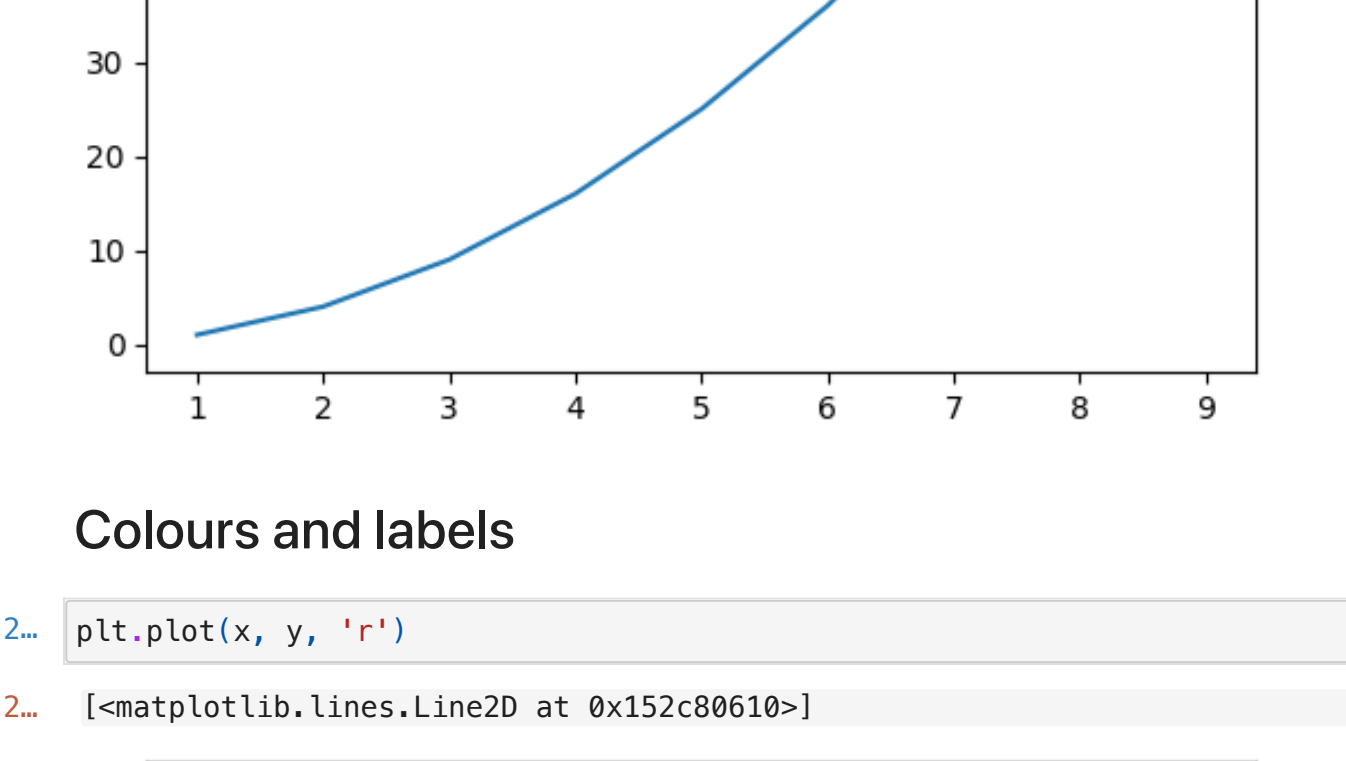
Out[2…
```
[<matplotlib.lines.Line2D at 0x152c05c70>]
```



## Colours and labels

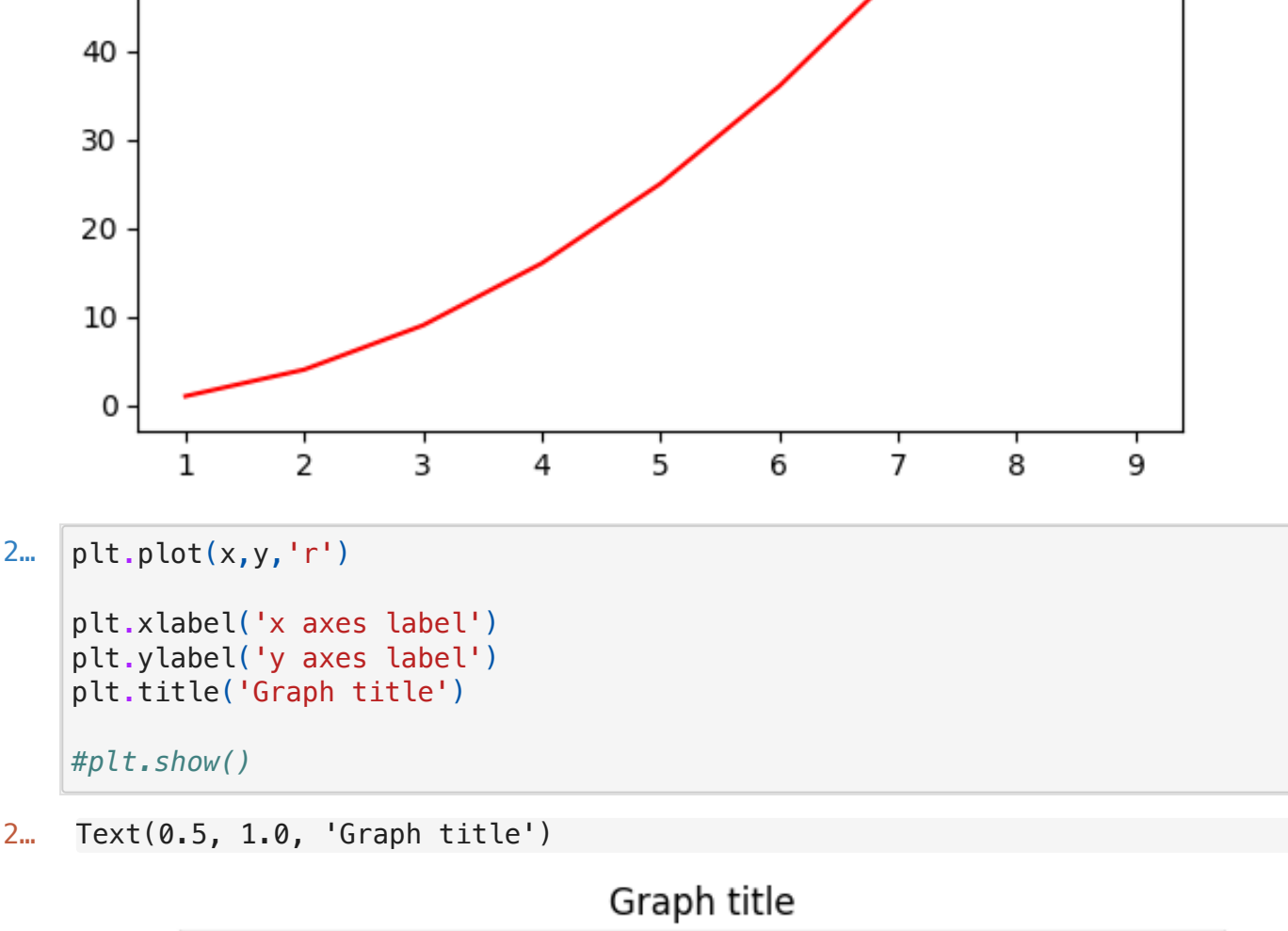
In [2…
```
plt.plot(x, y, 'r')
```

Out[2…
```
[<matplotlib.lines.Line2D at 0x152c80610>]
```



In [2…
```
plt.plot(x,y,'r')

plt.xlabel('x axes label')
plt.ylabel('y axes label')
plt.title('Graph title')

#plt.show()
```

Out[2…
```
Text(0.5, 1.0, 'Graph title')
```



In [6…
```
# numpy.linspace(start, stop, num)
# Returns num evenly spaced samples, calculated over the interval [start, stop].
x_1 = np.linspace(0, 5, 11)
y_1 = x_1**3
```

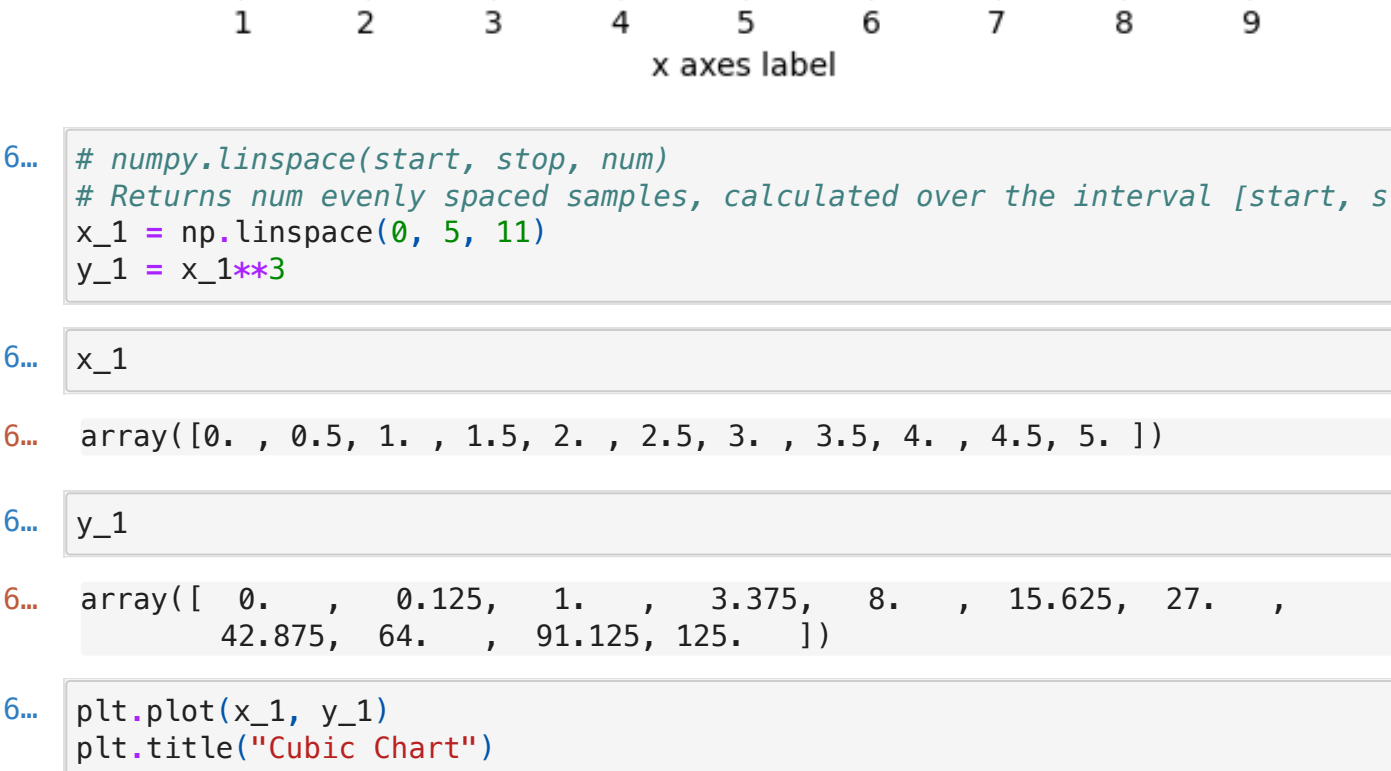
In [6…
```
x_1
```

Out[6…
```
array([0. , 0.5, 1. , 1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5, 5. ])
```

In [6…
```
y_1
```

Out[6…
```
array([  0.   ,   0.125,   1.   ,   3.375,   8.   ,  15.625,  27.   ,
        42.875,  64.   ,  91.125, 125.   ])
```

In [6…
```
plt.plot(x_1, y_1)
plt.title("Cubic Chart")
plt.xlabel("Values")
_ = plt.ylabel("Cubic Values")
```

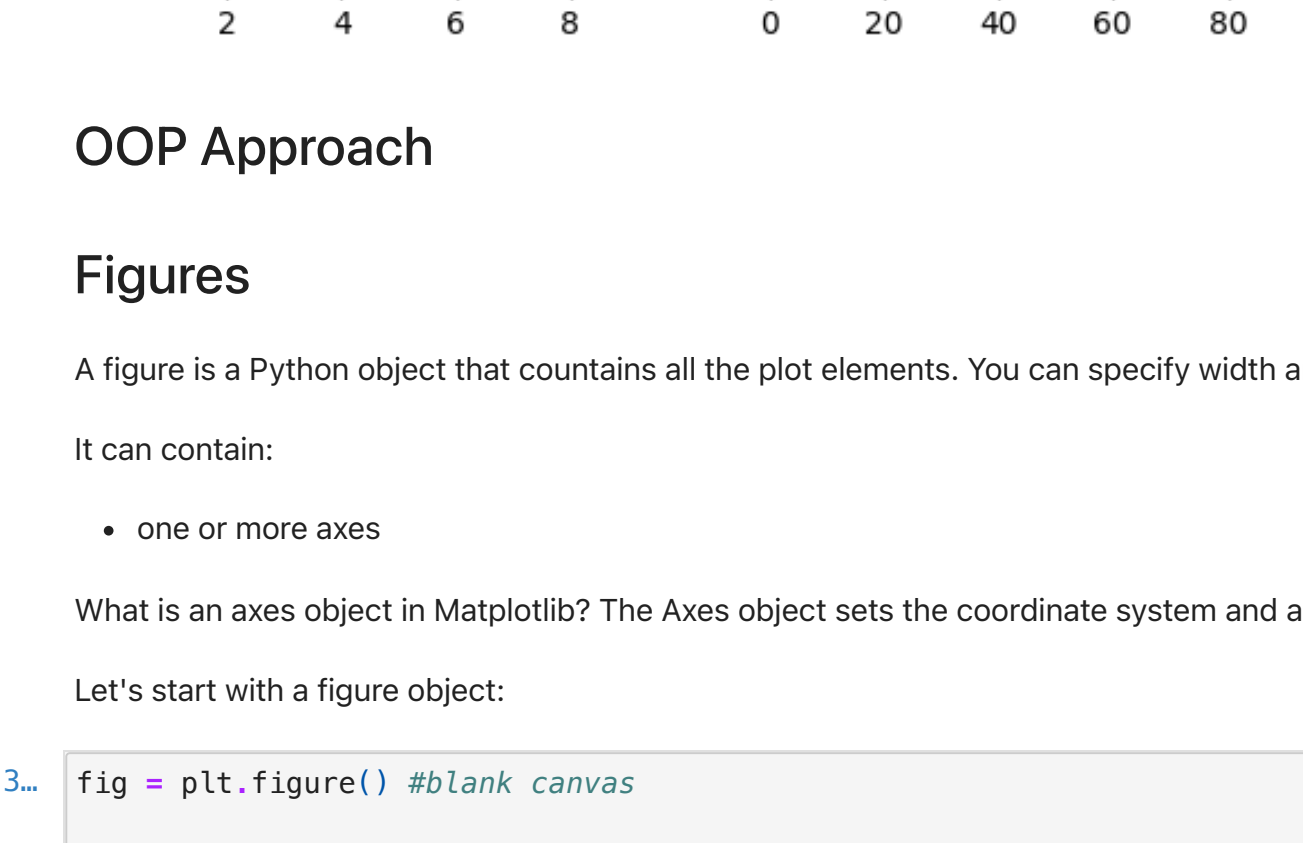


## Subplots

- The first parameter, 1, indicates the number of rows in the subplot grid.
- The second parameter, 2, indicates the number of columns in the subplot grid.
- The third parameter, 1, indicates the index of the specific subplot being created.

In [2…
```
plt.subplot(1, 2, 1) # row, col, index
plt.plot(x, y, 'r') # r for red– matlab

plt.subplot(1, 2, 2) # row, col, index
plt.plot(y, x, 'b') # b for blue–matlab

#plt.show()
```

Out[2…
```
[<matplotlib.lines.Line2D at 0x165f72df0>]
```



## OOP Approach

## Figures

A figure is a Python object that countains all the plot elements. You can specify width and height in inches

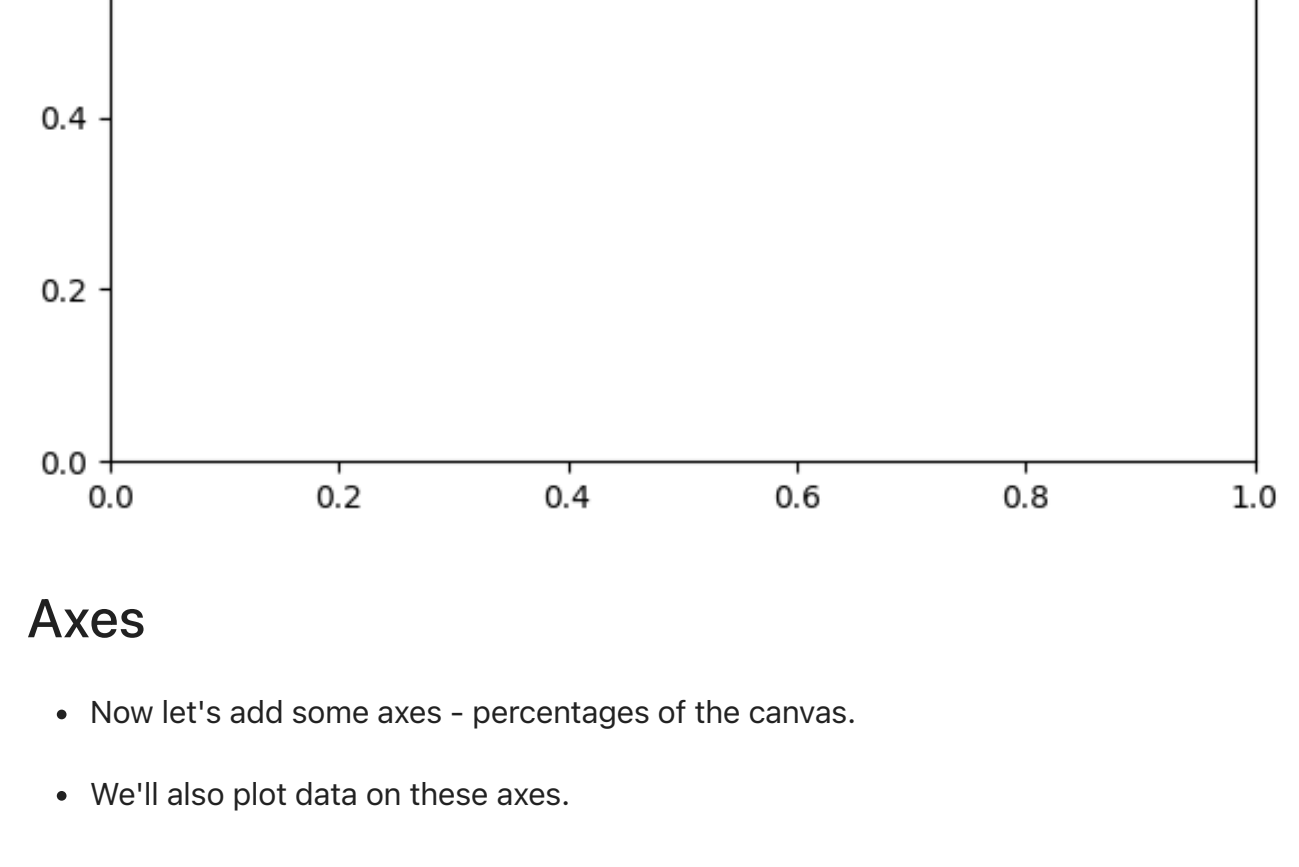
It can contain:

- one or more axes

What is an axes object in Matplotlib? The Axes object sets the coordinate system and acts as a container for most of the figure elements.

Let's start with a figure object:

In [3…
```
fig = plt.figure() #blank canvas

axes = fig.add_axes([0.1, 0.1, 0.8, 0.8])
                    #left, bottom, width, height
```

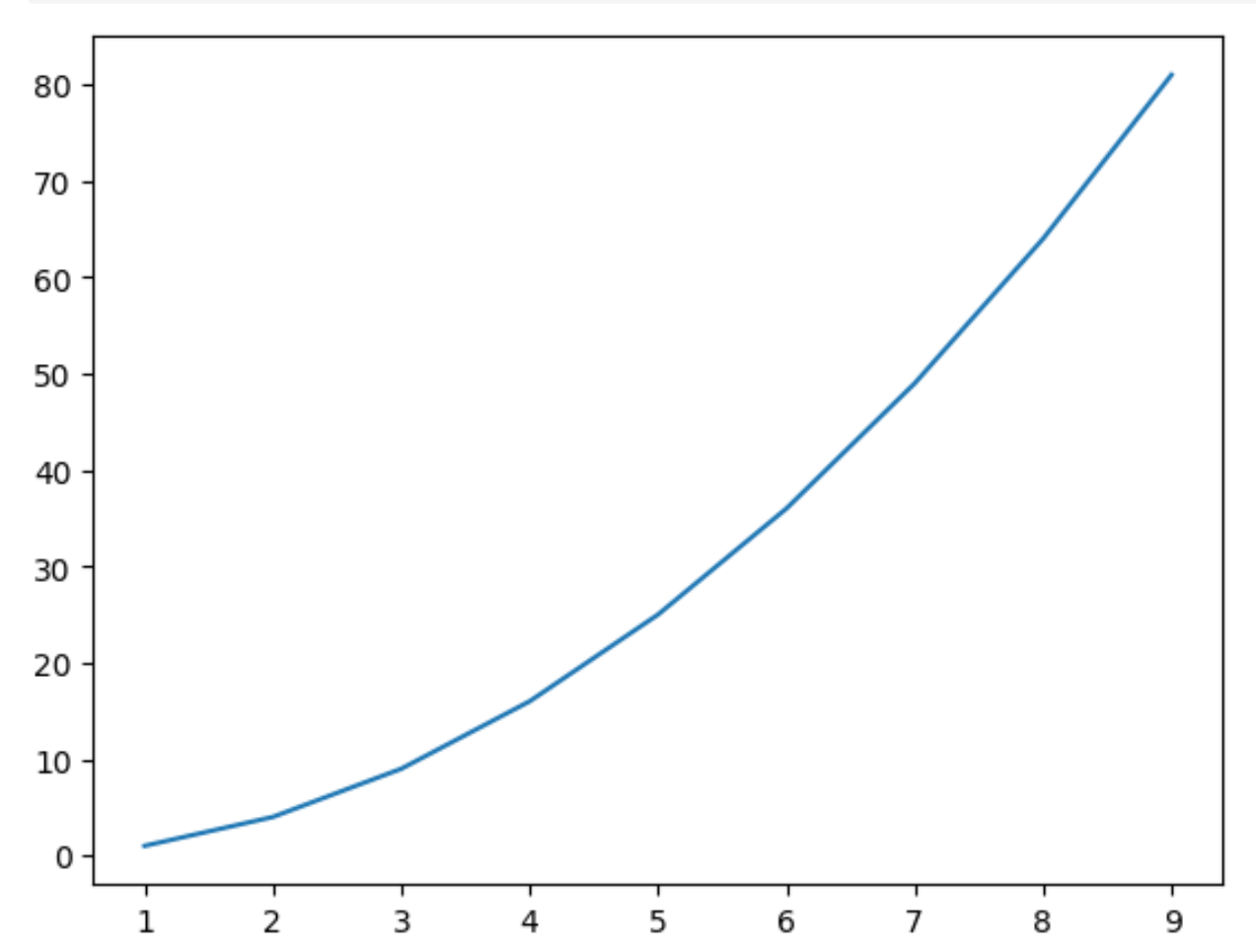


## Axes

- Now let's add some axes - percentages of the canvas.
- We'll also plot data on these axes.

In [3…
```
fig = plt.figure() #blank canvas

axes = fig.add_axes([0.1, 0.1, 0.8, 0.8])
                    #left, bottom, width, height
axes.plot(x, y)
```

Out[3…
```
[<matplotlib.lines.Line2D at 0x152ed3910>]
```



## Labels on an axes

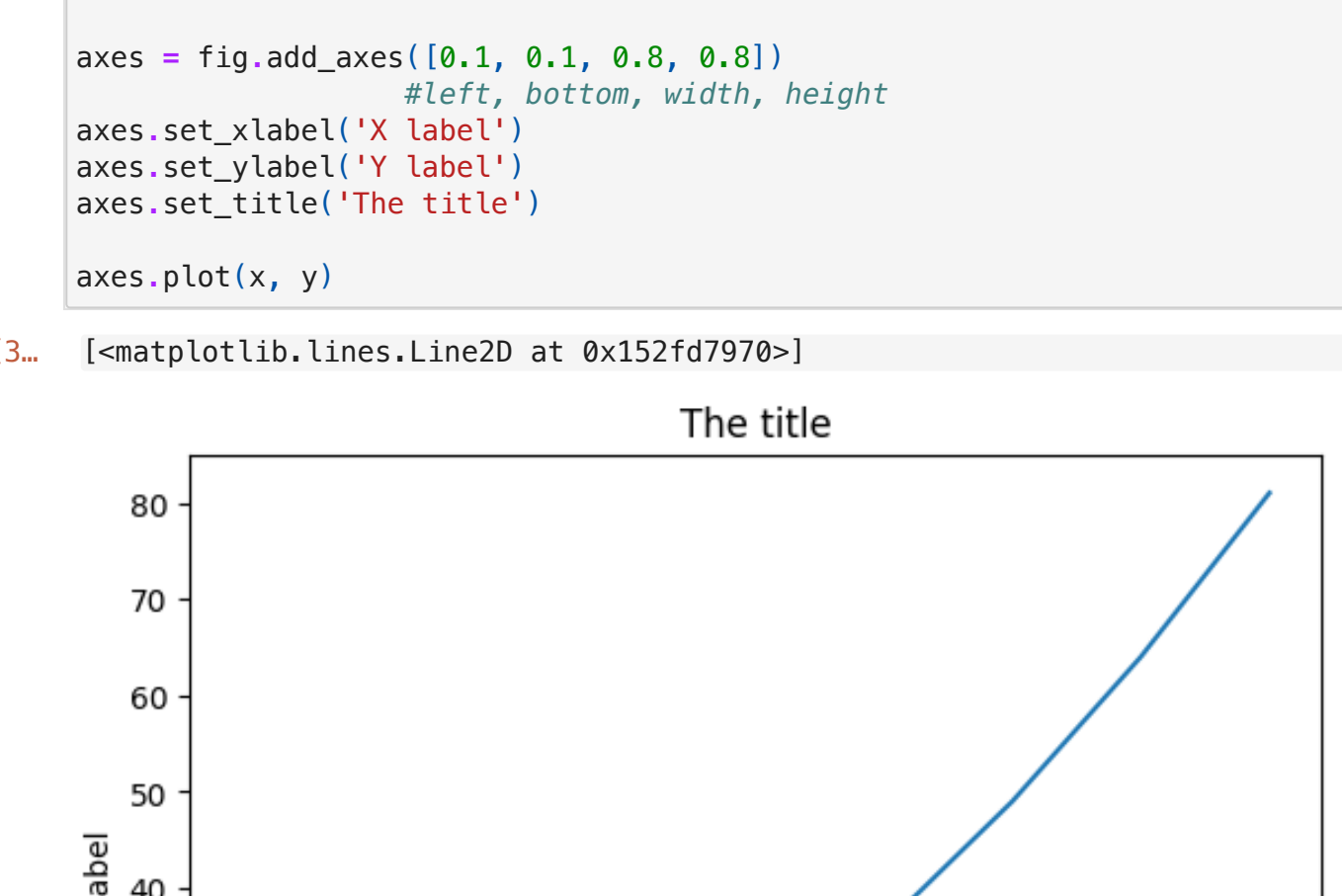
- Functional approach: `plt.xlabel("...")`
- OOP uses the 'set' / 'get' terminology: `set_xlabel("...")`

In [ …
```
axes.set_xlabel('X label')
axes.set_ylabel('Y label')
axes.set_title('The title')
```

Out[ …
```
Text(0.5, 1.0, 'The title')
```

In [3…
```
fig = plt.figure() #blank canvas

axes = fig.add_axes([0.1, 0.1, 0.8, 0.8])
                    #left, bottom, width, height
axes.set_xlabel('X label')
axes.set_ylabel('Y label')
axes.set_title('The title')

axes.plot(x, y)
```

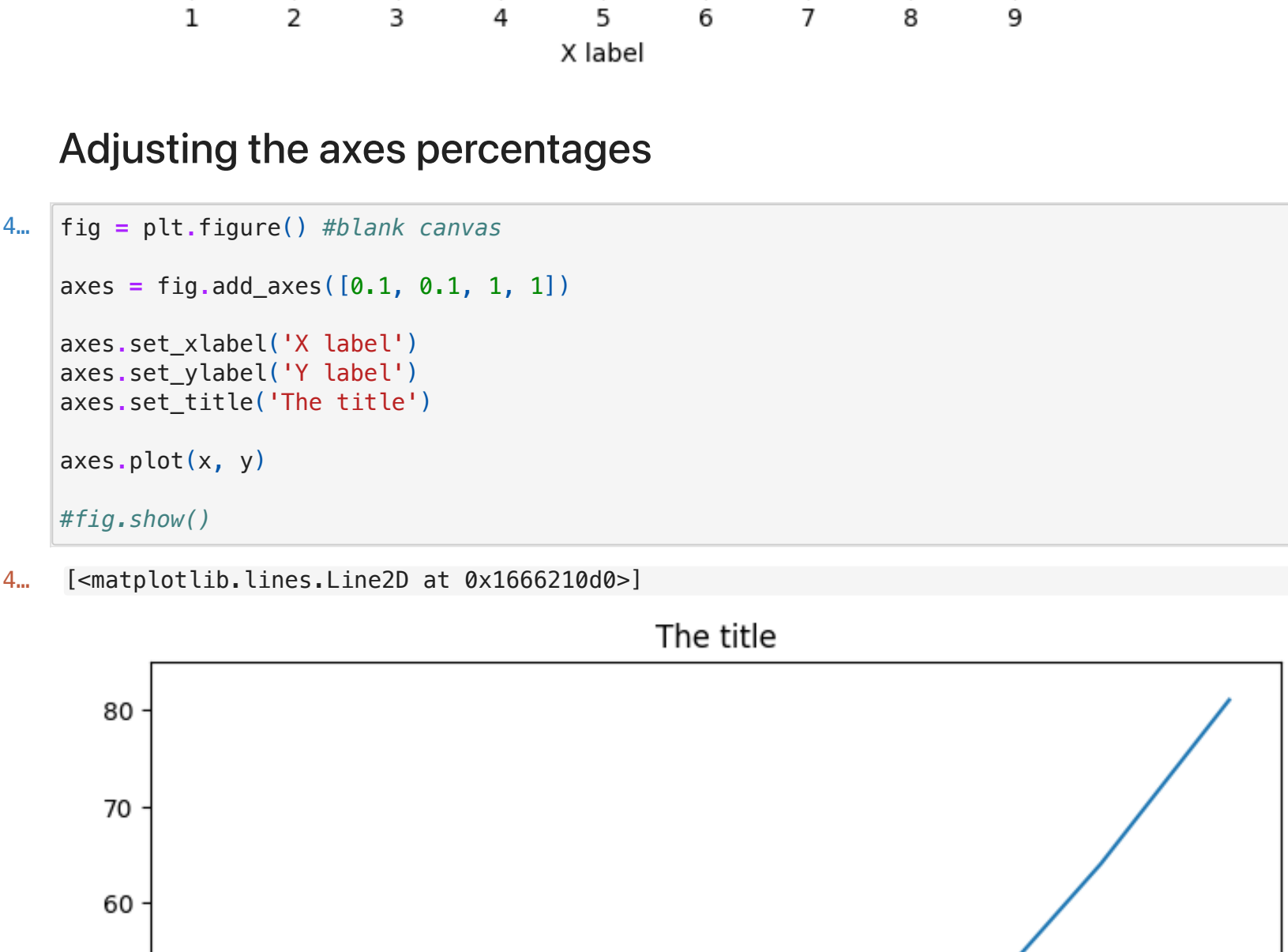
Out[3…
```
[<matplotlib.lines.Line2D at 0x152fd7970>]
```



## Adjusting the axes percentages

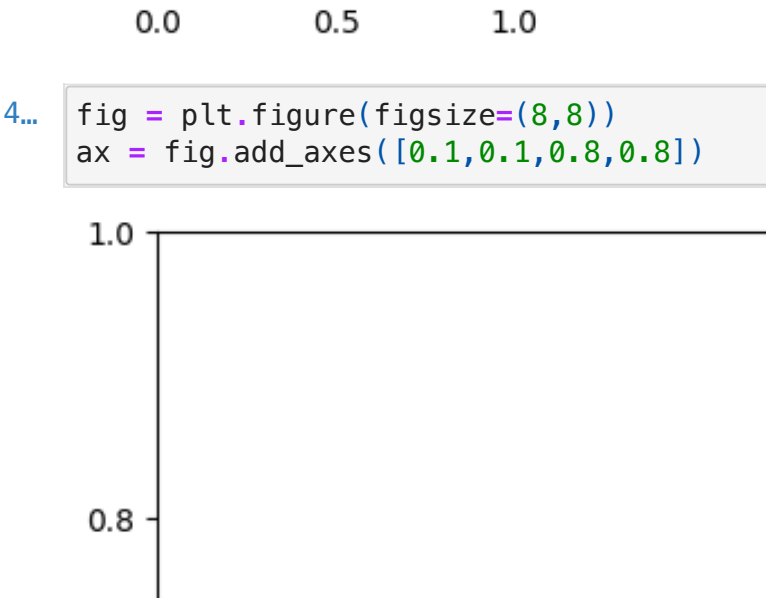
In [4…
```
fig = plt.figure() #blank canvas

axes = fig.add_axes([0.1, 0.1, 1, 1])

axes.set_xlabel('X label')
axes.set_ylabel('Y label')
axes.set_title('The title')

axes.plot(x, y)

#fig.show()
```

Out[4…
```
[<matplotlib.lines.Line2D at 0x1666210d0>]
```

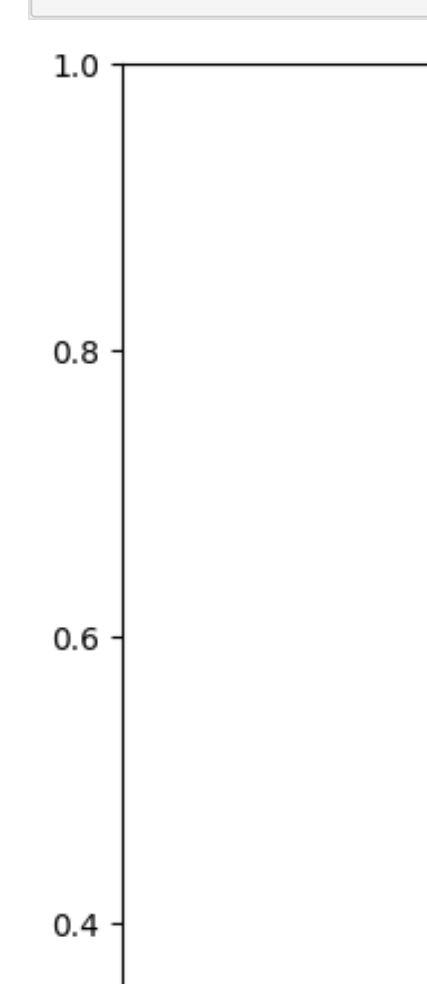


## `figsize()`

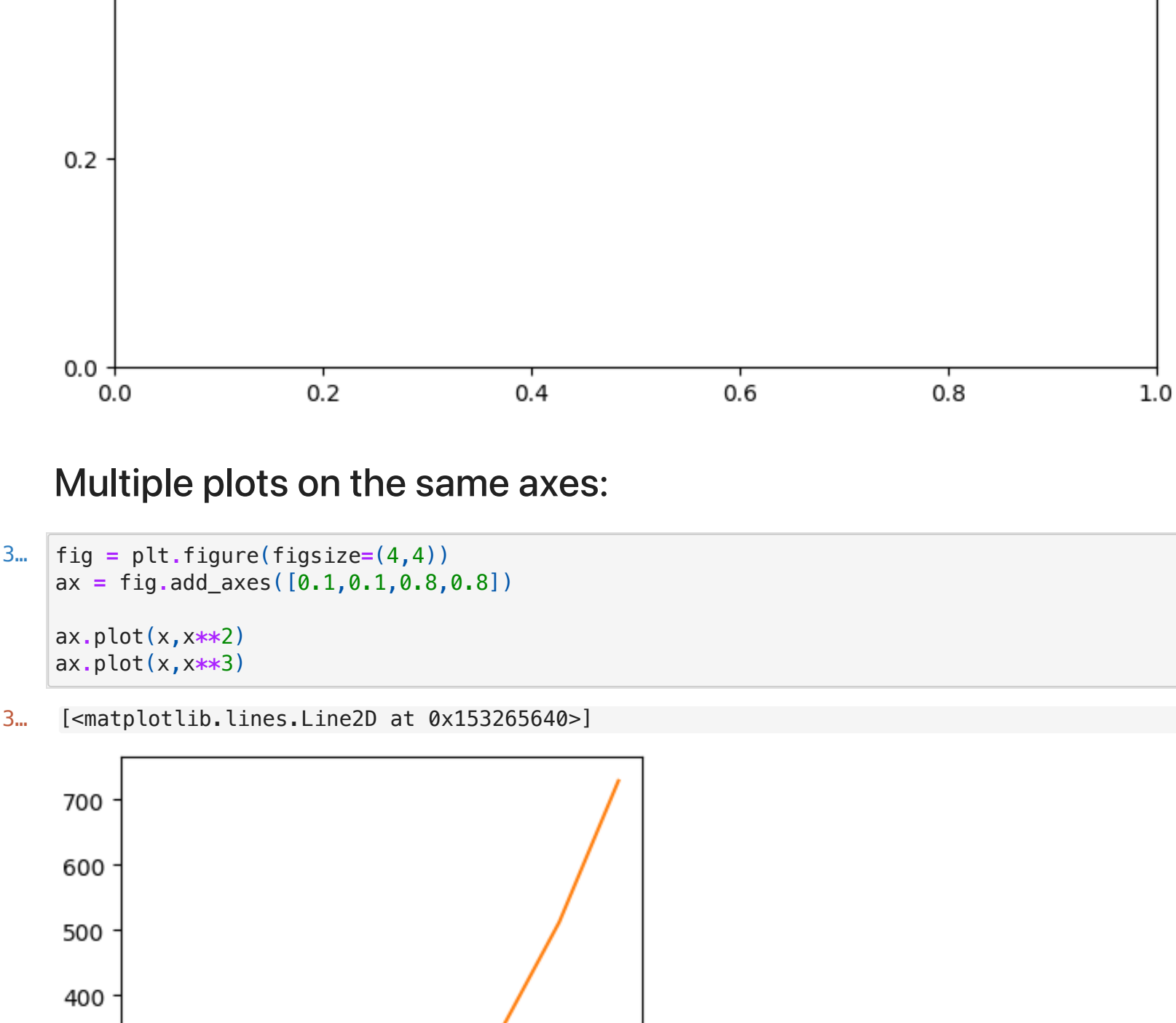
In [4…
```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])
```



In [4…
```
fig = plt.figure(figsize=(2,2))
ax = fig.add_axes([0.1,0.1,0.8,0.8])
```



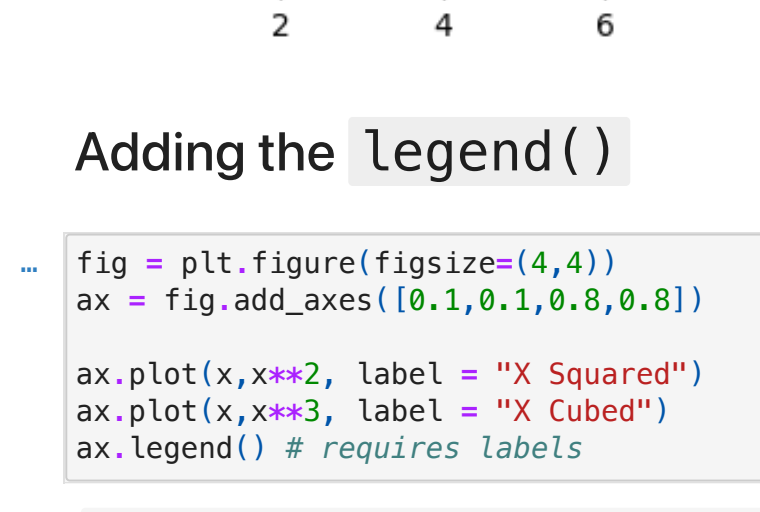
In [4…
```
fig = plt.figure(figsize=(8,8))
ax = fig.add_axes([0.1,0.1,0.8,0.8])
```



## Multiple plots on the same axes:

In [3…
```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x,x**2)
ax.plot(x,x**3)
```

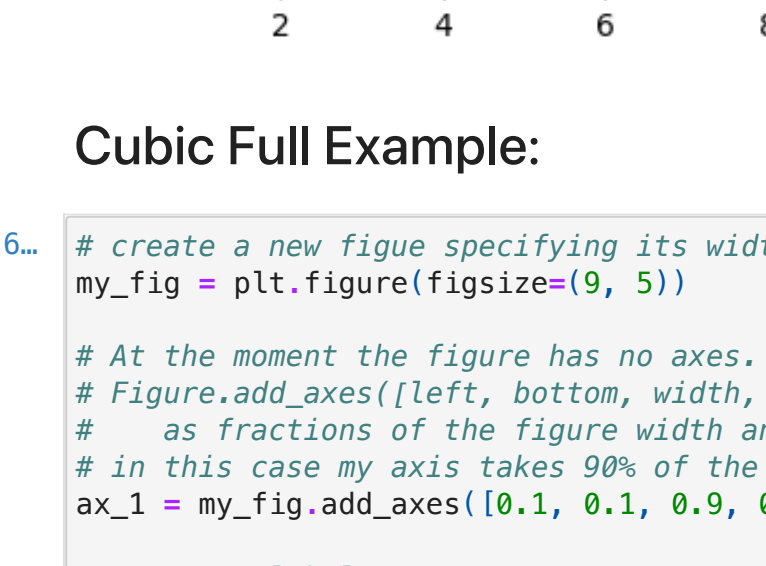
Out[3…
```
[<matplotlib.lines.Line2D at 0x153265640>]
```



## Adding the `legend()`

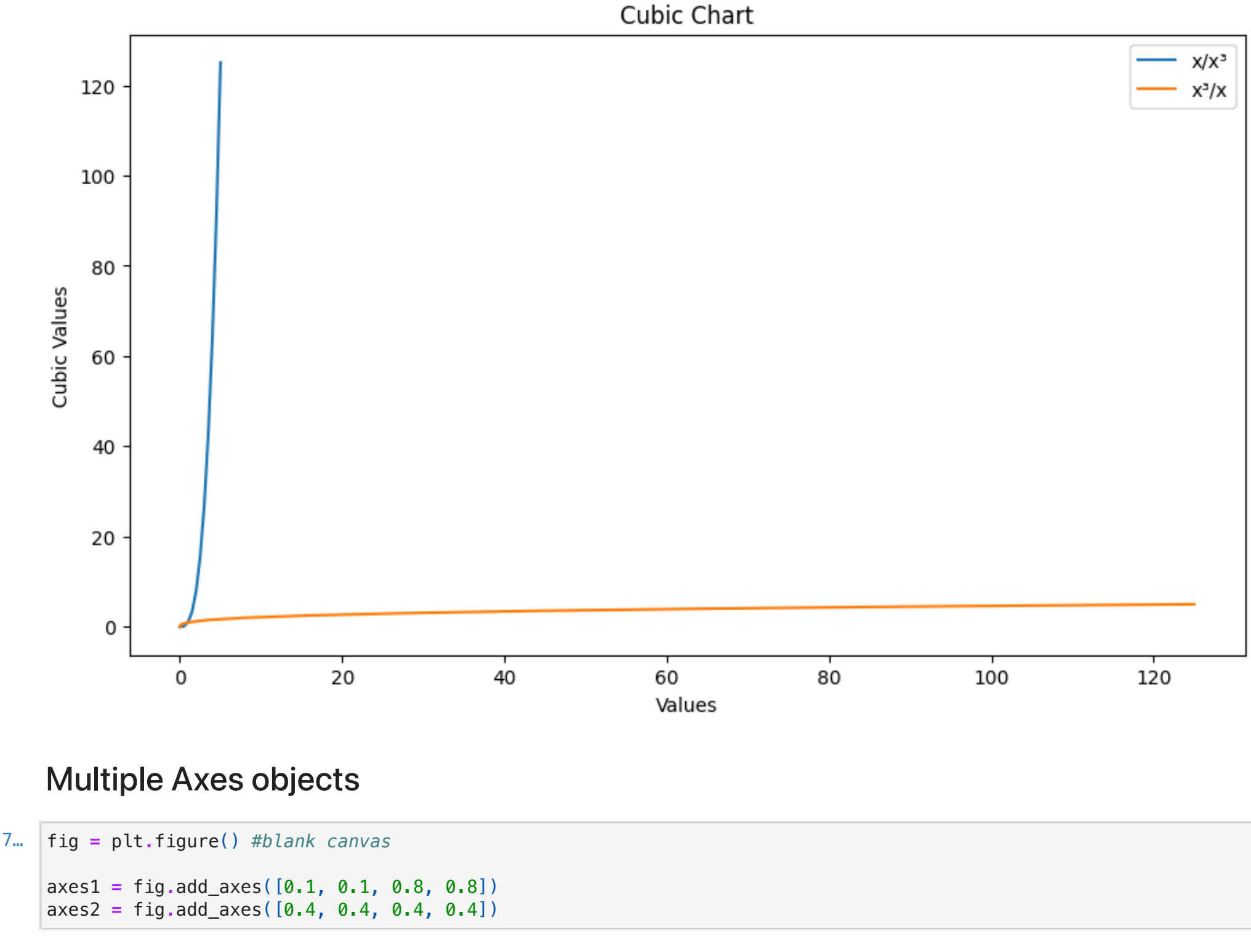
In [ …
```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x,x**2, label = "X Squared")
ax.plot(x,x**3, label = "X Cubed")
ax.legend() # requires labels
```

Out[ …
```
<matplotlib.legend.Legend at 0x16a6b5310>
```



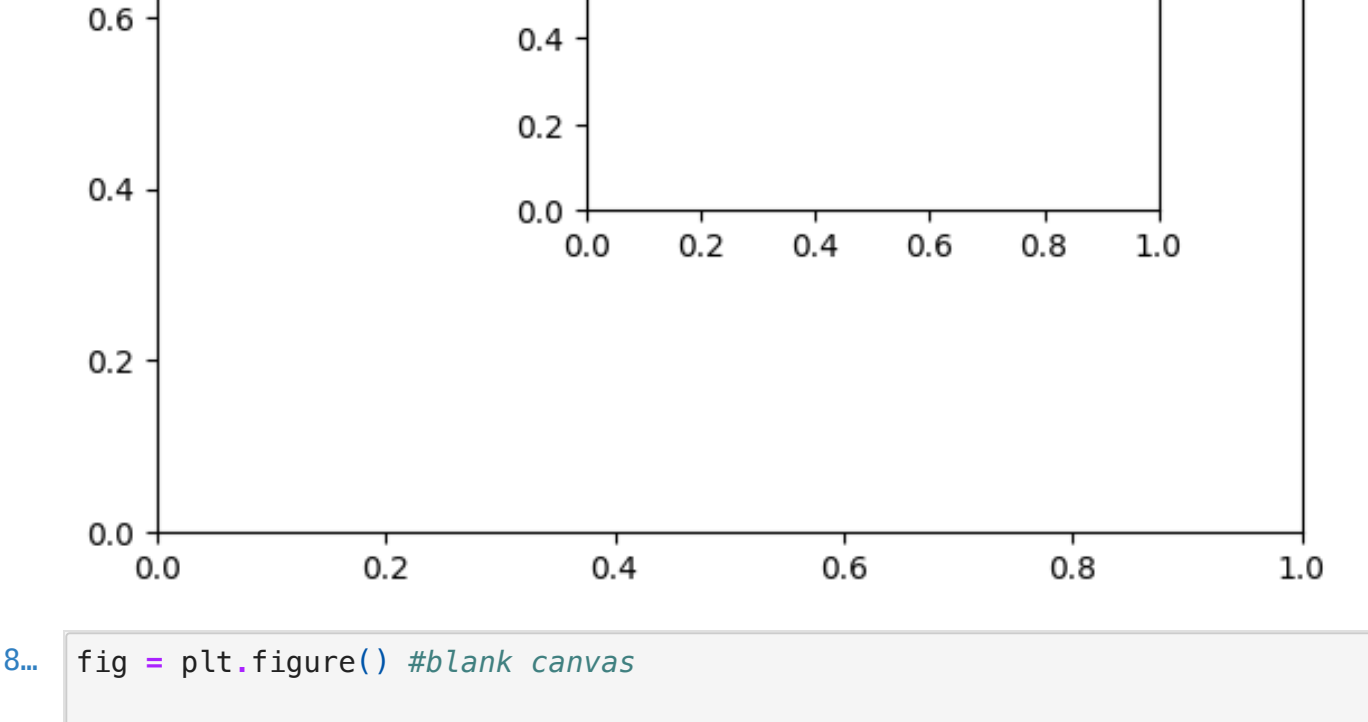
## Cubic Full Example:

In [6…
```
# create a new figue specifying its width and height in inches
my_fig = plt.figure(figsize=(9, 5))

# At the moment the figure has no axes. Let's add a new axis
# Figure.add_axes([left, bottom, width, height]) The dimensions of the new axes are expressed
#    as fractions of the figure width and height
# in this case my axis takes 90% of the width and 90% of the height.
ax_1 = my_fig.add_axes([0.1, 0.1, 0.9, 0.9])

# set axes labels
ax_1.set_xlabel("Values")
ax_1.set_ylabel("Cubic Values")
ax_1.set_title("Cubic Chart")

# plot some stuff
ax_1.plot(x_1, y_1, label="x/x³")
ax_1.plot(y_1, x_1, label="x³/x")
# loc = 0 => tries to find out the best location to place the legend
# legend position: 1 => upper right, 2 => upper left, 3 => lower right, 2 => lower left,
_ = ax_1.legend(loc=0)
```



## Multiple Axes objects

In [7…
```
fig = plt.figure() #blank canvas

axes1 = fig.add_axes([0.1, 0.1, 0.8, 0.8])
axes2 = fig.add_axes([0.4, 0.4, 0.4, 0.4])
```

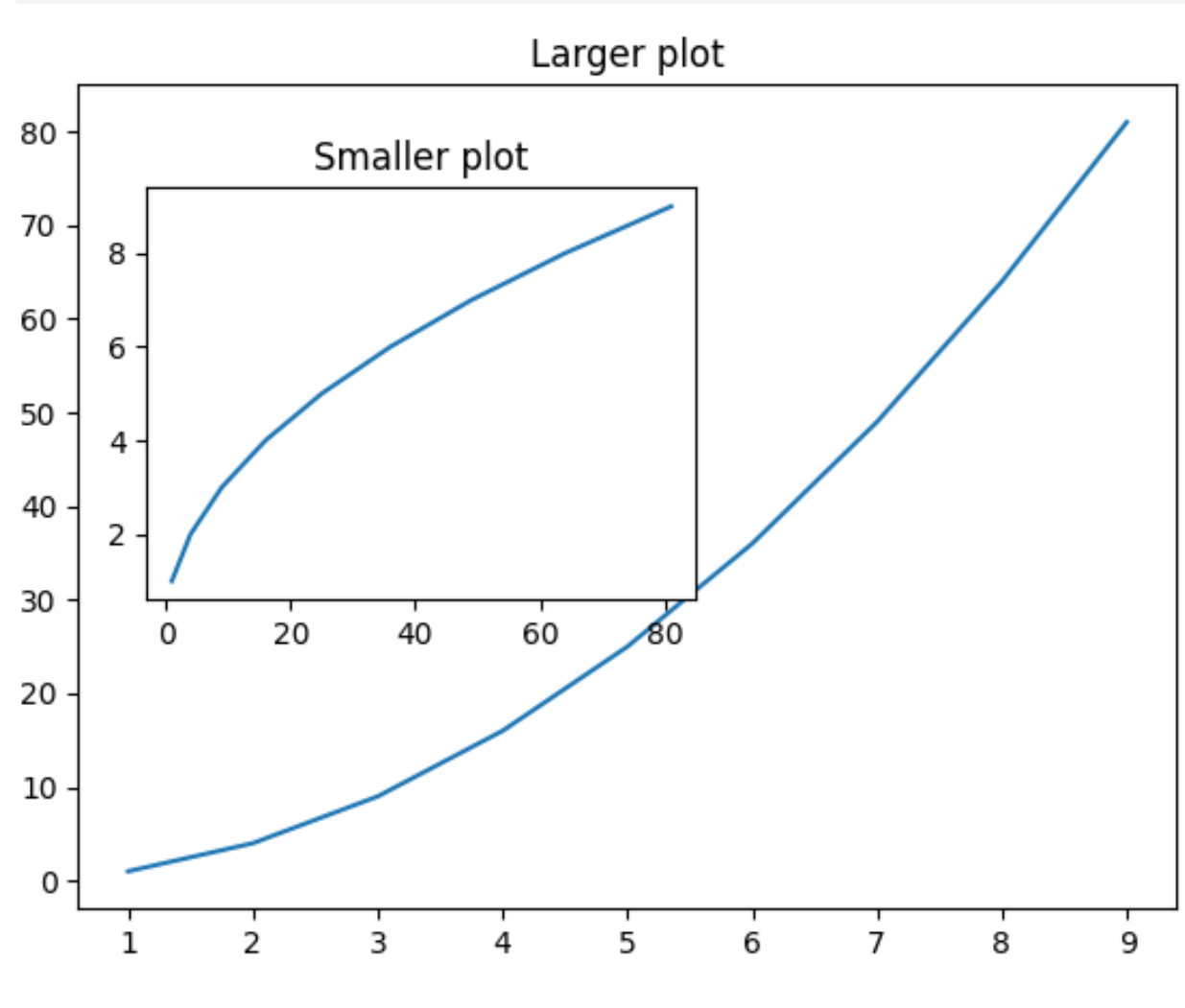


In [8…
```
fig = plt.figure() #blank canvas

axes1 = fig.add_axes([0.1, 0.1, 0.8, 0.8])
axes2 = fig.add_axes([0.15, 0.4, 0.4, 0.4])
```

```
axes1.set_title('Larger plot')
axes2.set_title('Smaller plot')

axes1.plot(x,y)
axes2.plot(y,x)

#fig
```

Out[8…

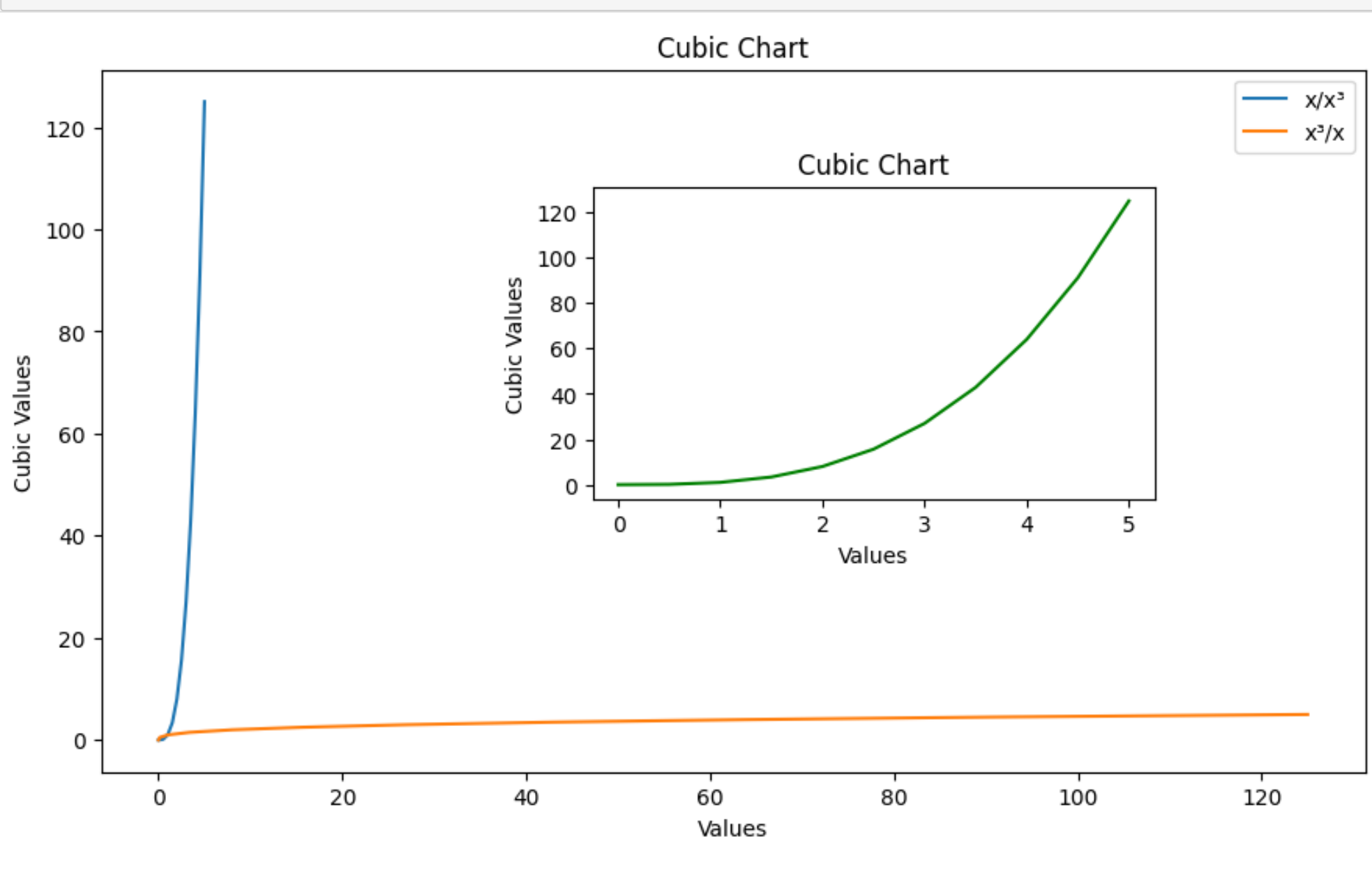
```
[<matplotlib.lines.Line2D at 0x169b3a880>]
```



More developed example:

In [8…

```
# same as before
new_fig = plt.figure(figsize=(9, 5))
ax_1 = new_fig.add_axes([0.1, 0.1, 0.9, 0.9])
ax_1.set_xlabel("Values")
ax_1.set_ylabel("Cubic Values")
ax_1.set_title("Cubic Chart")
ax_1.plot(x_1, y_1, label="x/x³")
ax_1.plot(y_1, x_1, label="x³/x")
ax_1.legend(loc=0)

# I am now adding a new axis system to my existing figure
ax_2 = new_fig.add_axes([0.45, 0.45, 0.4, 0.4])
ax_2.set_xlabel("Values")
ax_2.set_ylabel("Cubic Values")
ax_2.set_title("Cubic Chart")
_ = ax_2.plot(x_1, y_1, 'g')
```



I have effectively created a chart inside another chart!

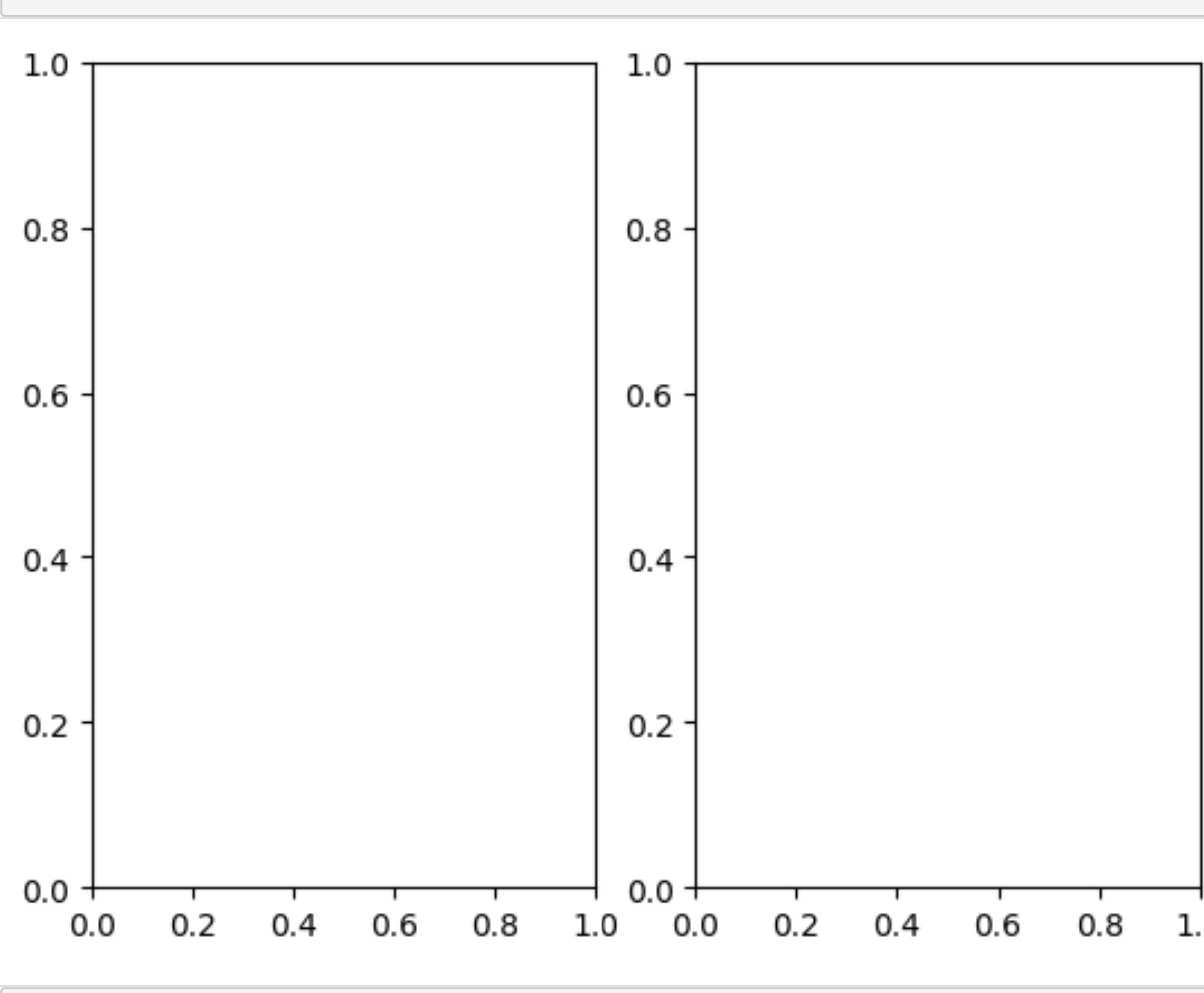
So far we haven't plotted anything yet. Let's do it now.

## Subplots

You can define more plots with subplots

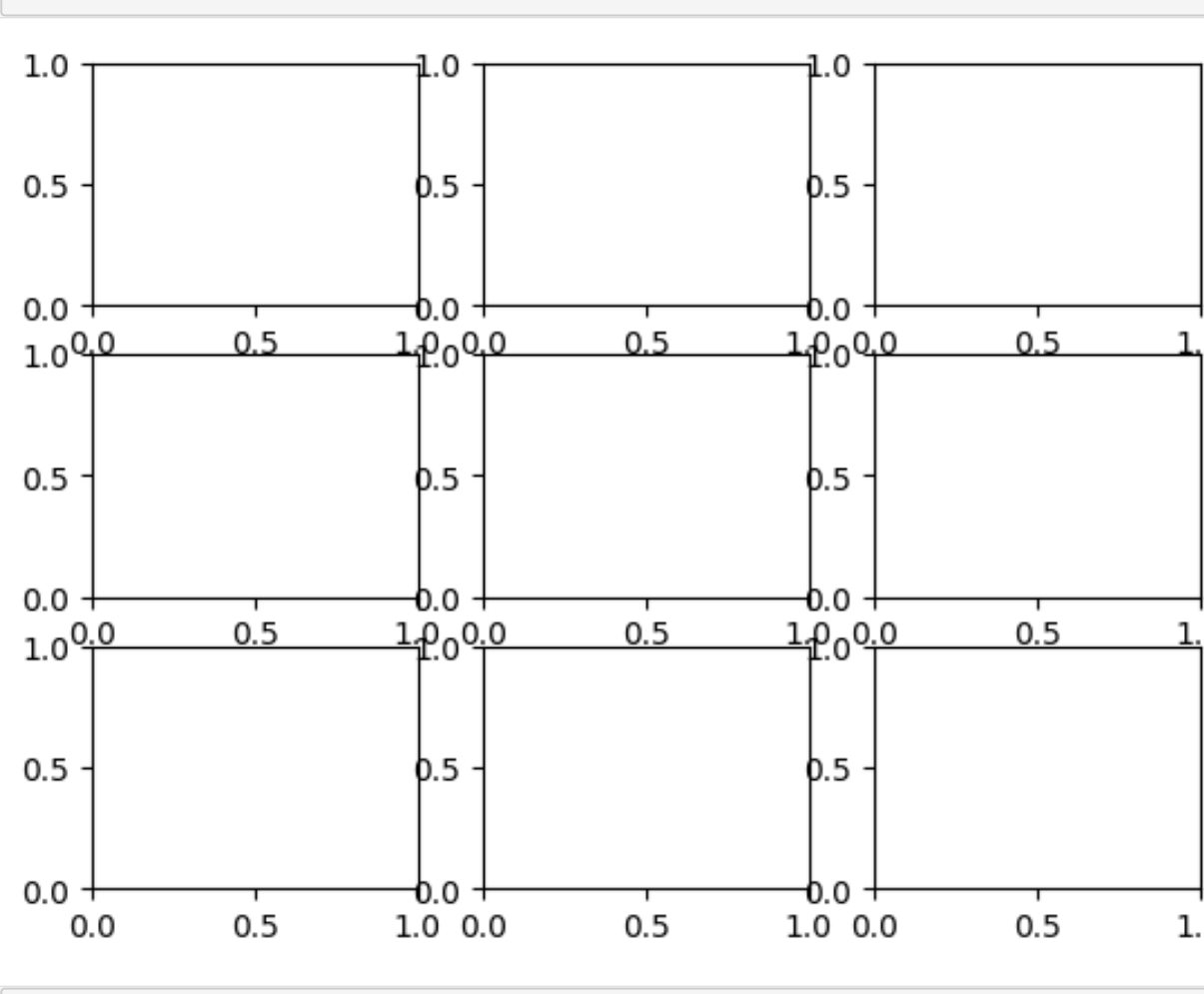
In [5…

```
fig, axes = plt.subplots(nrows=1, ncols=2)
```



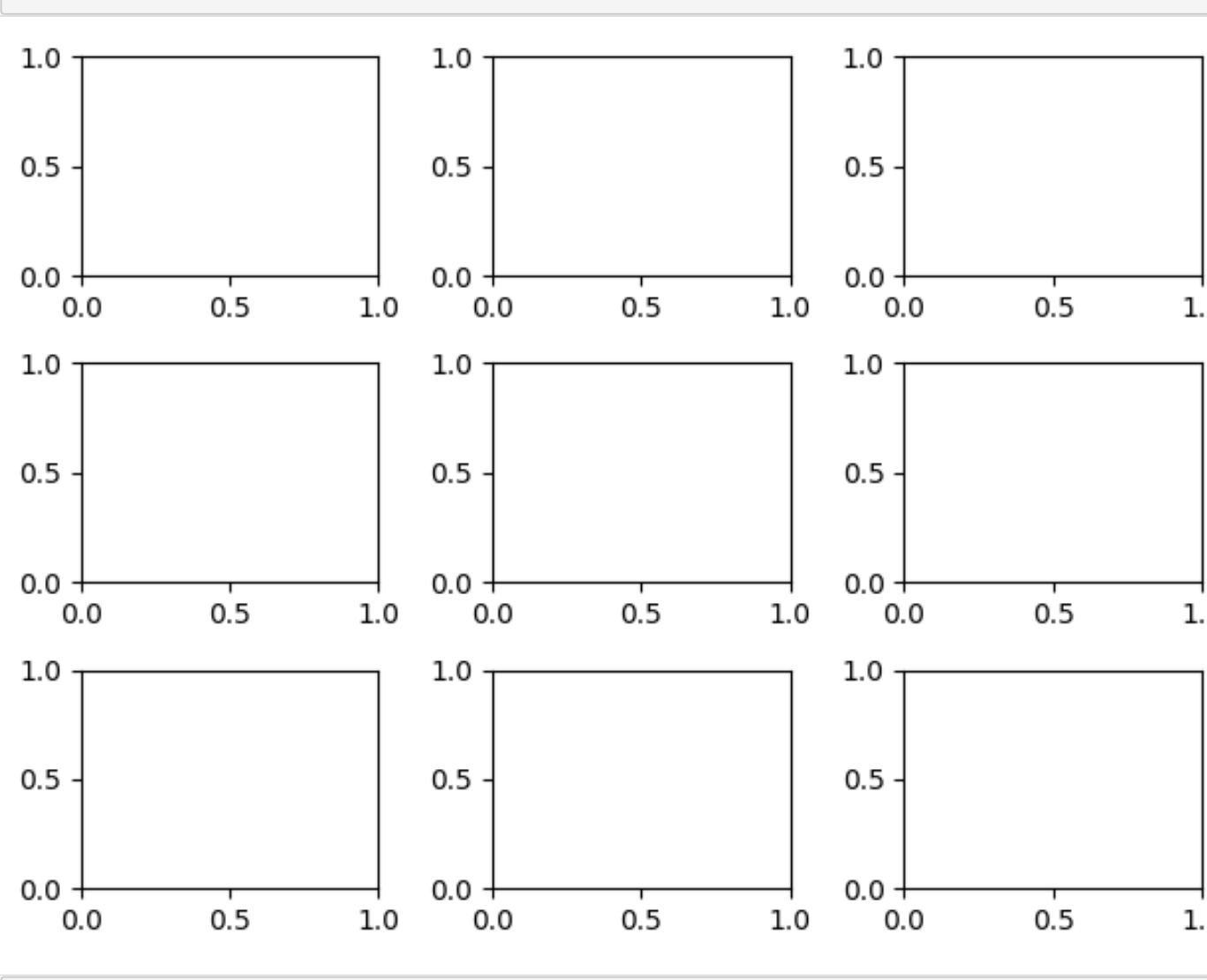
In [5…

```
fig, axes = plt.subplots(nrows=3, ncols=3)
```



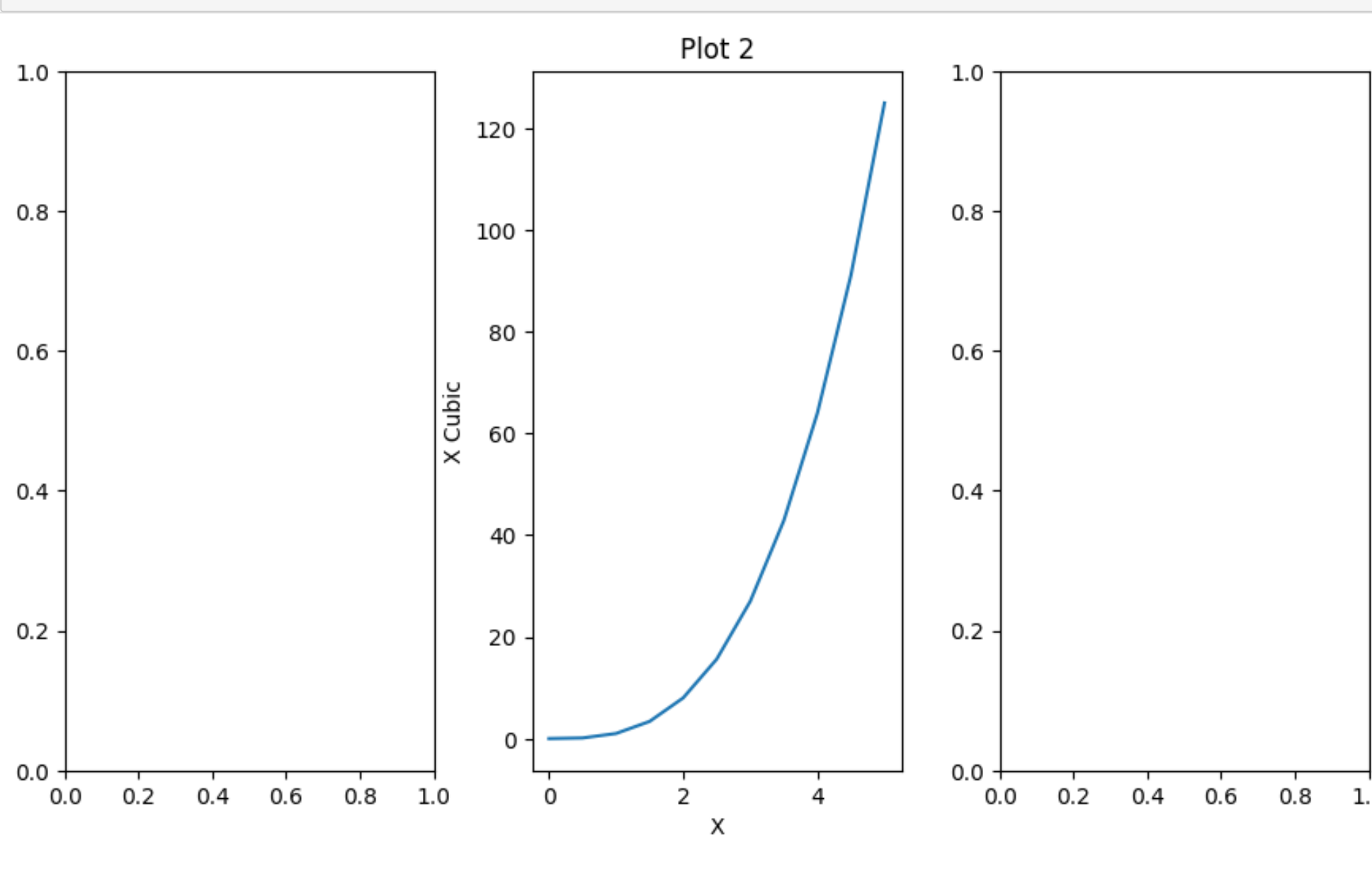
In [5…

```
fig, axes = plt.subplots(nrows=3, ncols=3)
plt.tight_layout()
```



In [6…

```
# create a figure with one axis having 3 subplots
# our figure/axis has a grid of subplots with 1 row and 3 columns
fig, ax = plt.subplots(figsize=(9, 5), nrows=1, ncols=3)

# Adjust the padding between and around subplots.
plt.tight_layout()

ax[1].set_title("Plot 2")
ax[1].set_xlabel("X")
ax[1].set_ylabel("X Cubic")
_ = ax[1].plot(x_1, y_1)
```



As you can see - the 'axes' object is subscriptable - acts like a list of axes that we can iterate through:

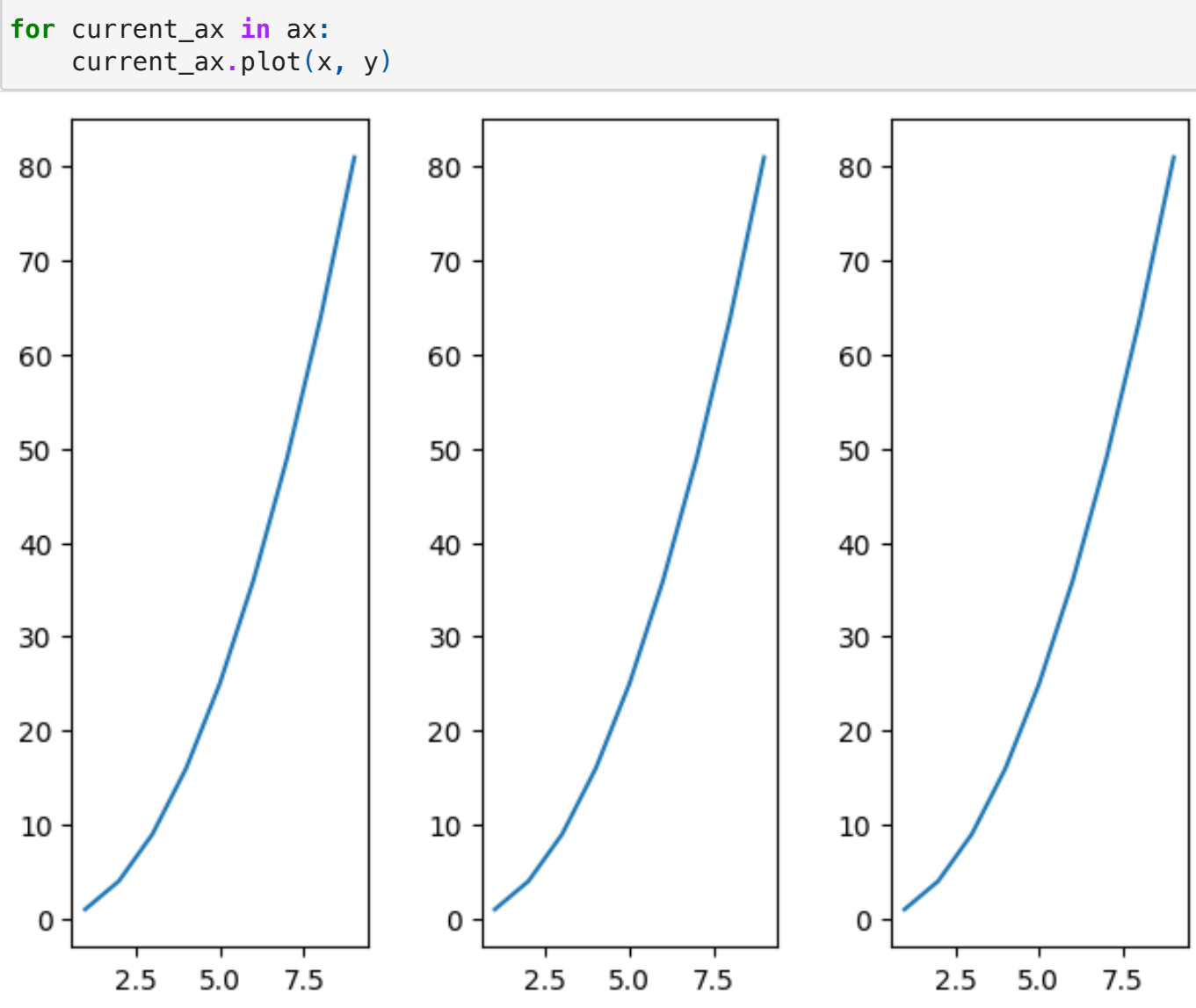
In [8…

```
l = [0,1,2]

for item in l:
    print(item)
```

```
0
1
2
```

In [8…

```
fig, ax = plt.subplots(nrows=1, ncols=3)
plt.tight_layout()

for current_ax in ax:
    current_ax.plot(x, y)
```

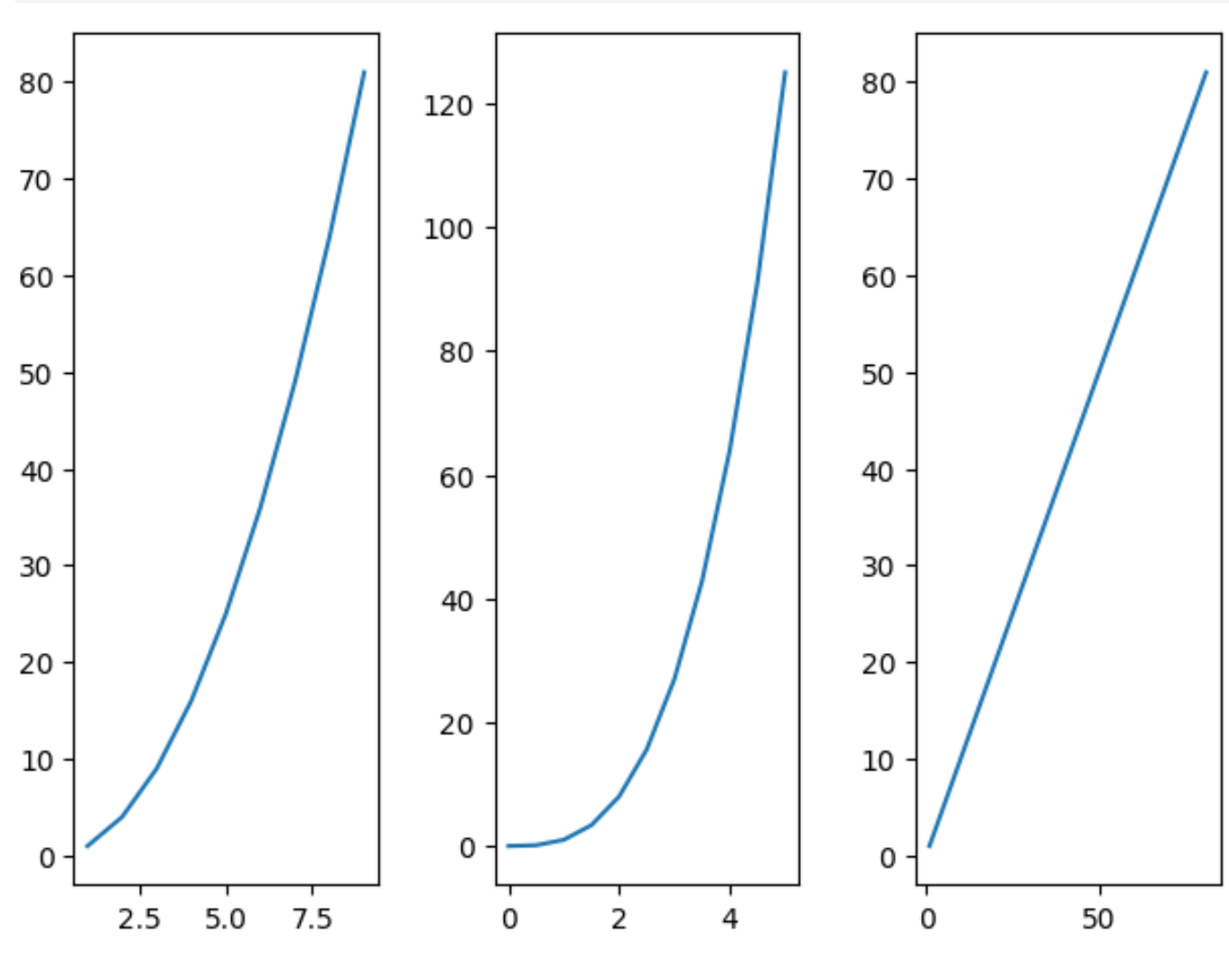


In [8…

```
fig, ax = plt.subplots(nrows=1, ncols=3)
plt.tight_layout()

ax[0].plot(x,y)
ax[1].plot(x_1, y_1)
ax[2].plot(x**2,y)
```

Out[8…

```
[<matplotlib.lines.Line2D at 0x15463ee80>]
```



## Formatting: Appearance Options

Default colours (by character):

- b: blue
- g: green
- r: red
- c: cyan
- m: magenta
- y: yellow
- k: black
- w: white

You can also define colours using hexadecimal codes: https://html-color.codes/

You can use colour names as standardised: https://www.w3schools.com/colors/colors_names.asp

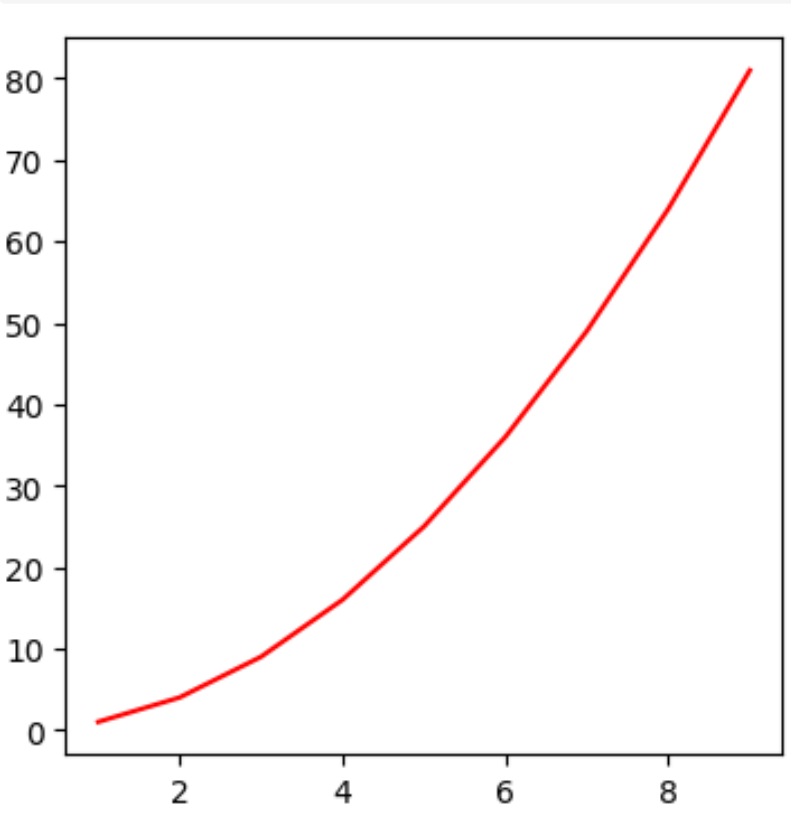
### `color`

In [8…

```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x, y, color = 'red')
```

Out[8…

```
[<matplotlib.lines.Line2D at 0x157167370>]
```



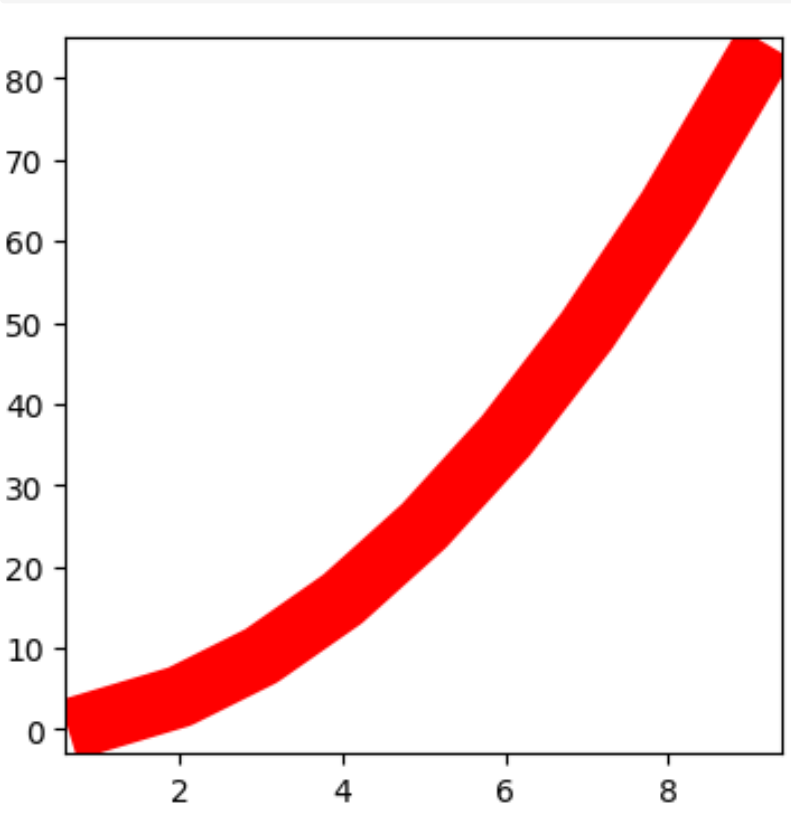
### `linewidth`

In [8…

```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x, y, color='red', linewidth=20)
```

Out[8…

```
[<matplotlib.lines.Line2D at 0x1546ad220>]
```



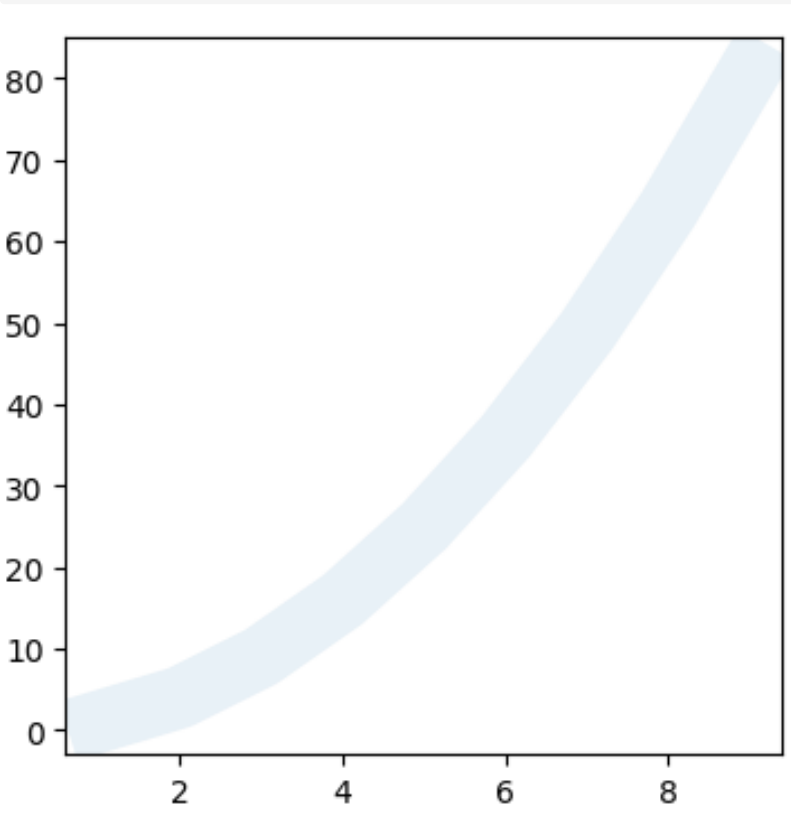
### `alpha` (transparency)

In [9…

```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x, y, linewidth=20, alpha = 0.1)
```

Out[9…

```
[<matplotlib.lines.Line2D at 0x156f52700>]
```



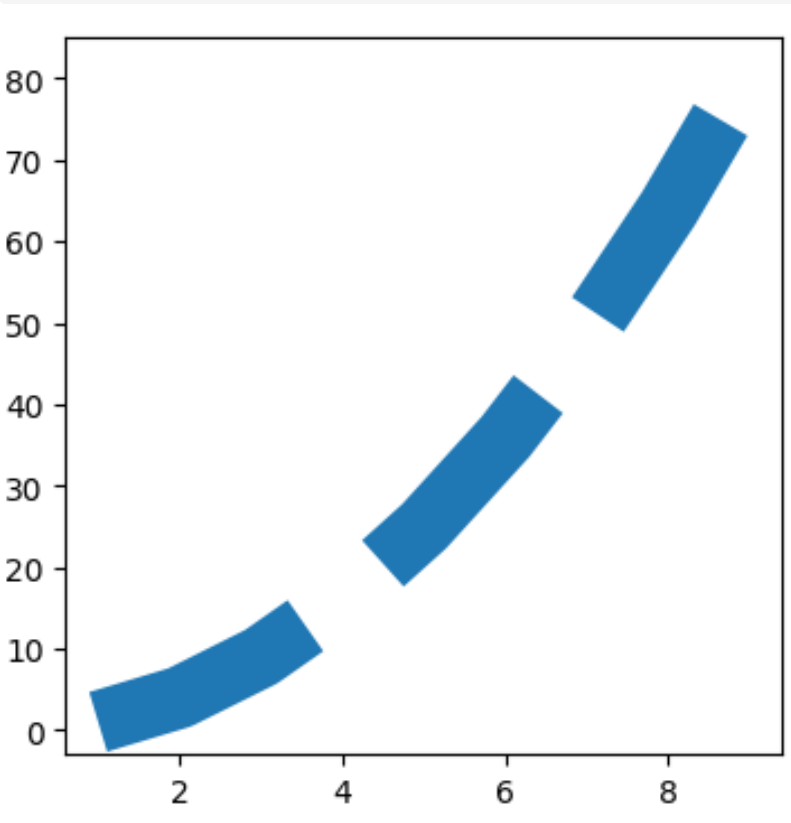
### `linestyle`

In [9…

```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x, y, linewidth=20, linestyle="--")
```

Out[9…

```
[<matplotlib.lines.Line2D at 0x1571e4e50>]
```



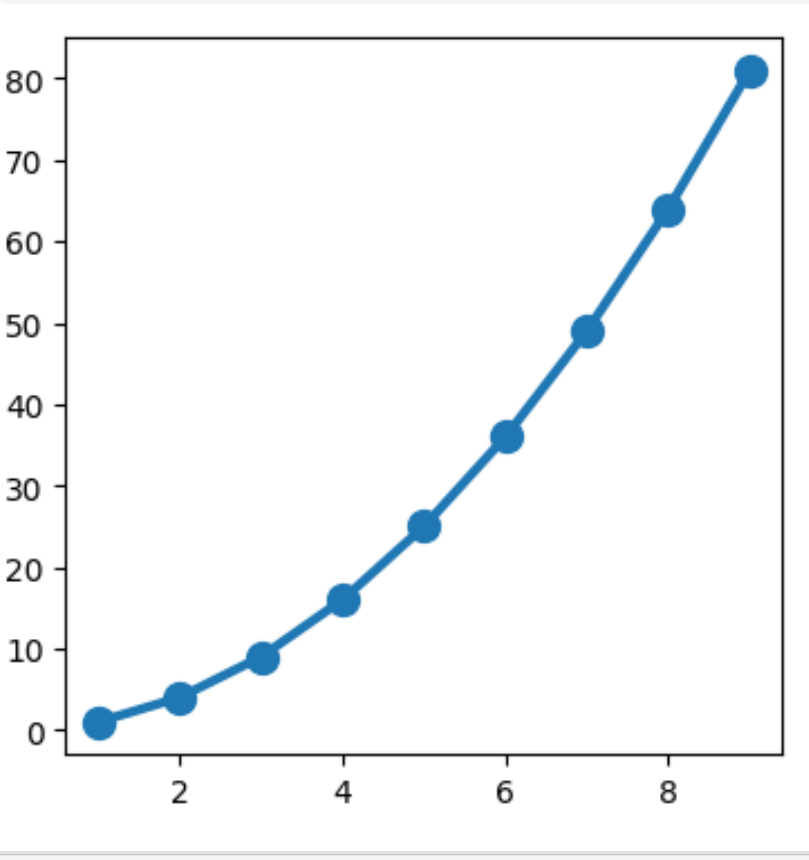
### `marker`

In [9…

```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x, y, linewidth=3, marker = "o", markersize = "10")
```

Out[9…

```
[<matplotlib.lines.Line2D at 0x1570d0580>]
```

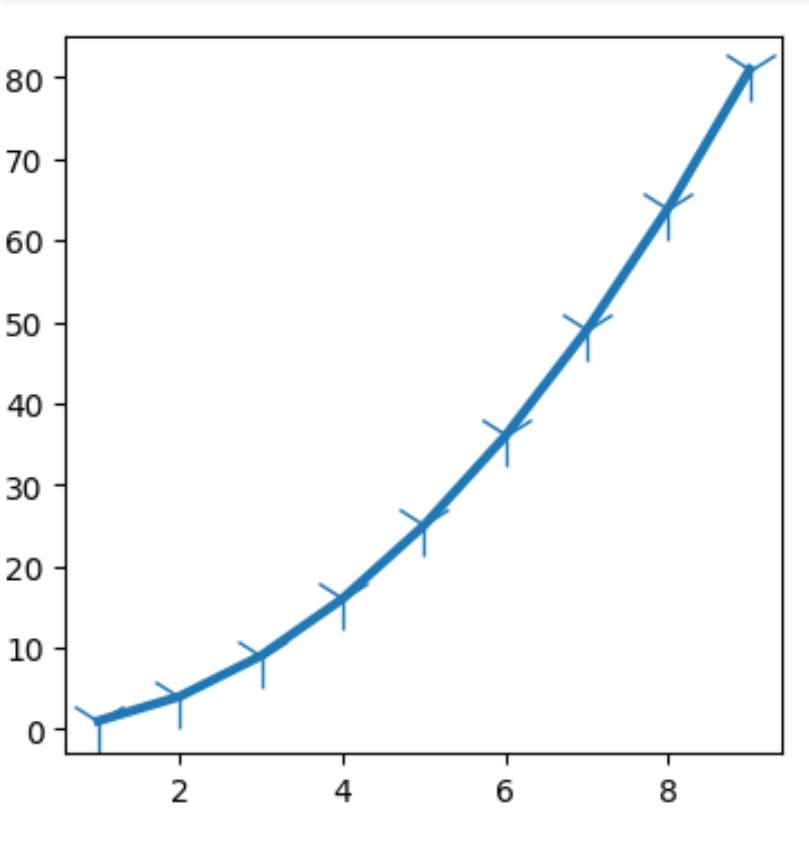


In [9…

```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x, y, linewidth=3, marker = "1", markersize = "20")
```

Out[9…

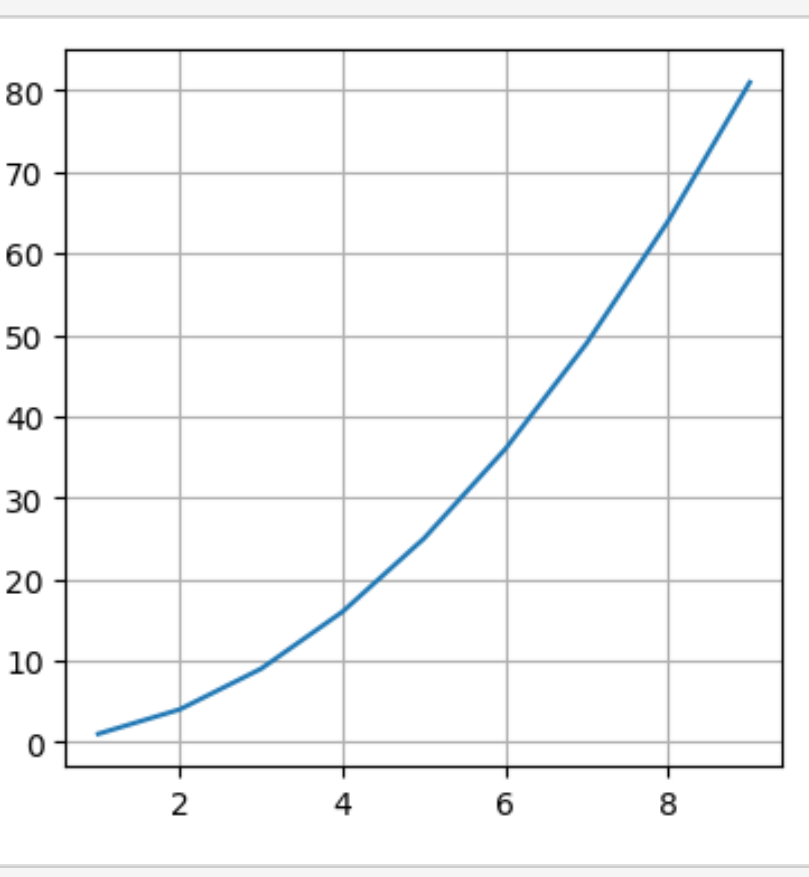
```
[<matplotlib.lines.Line2D at 0x169fbaac0>]
```



### `grid`

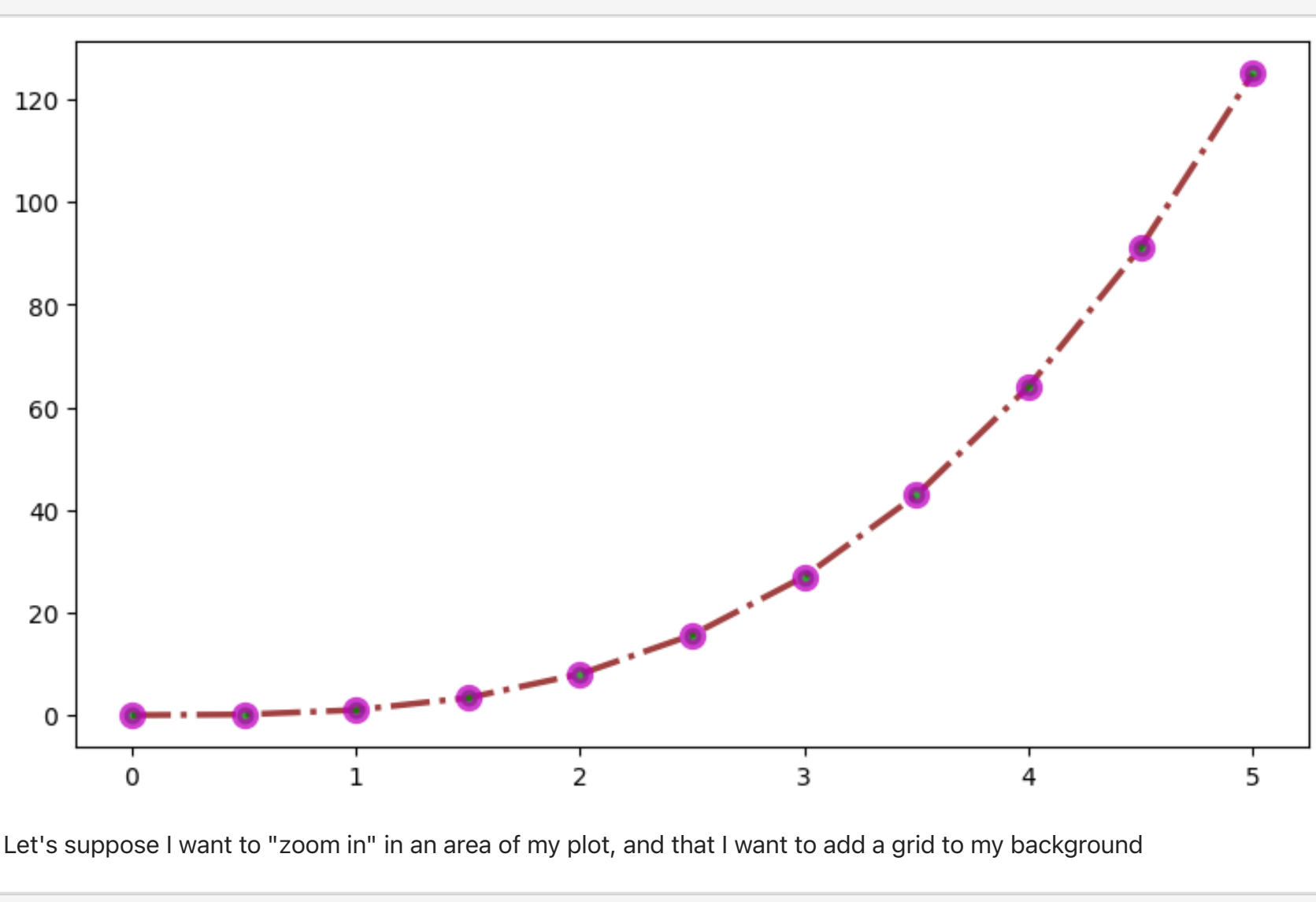
In [9…

```
fig = plt.figure(figsize=(4,4))
ax = fig.add_axes([0.1,0.1,0.8,0.8])

ax.plot(x, y)
plt.grid()
```



In [9…

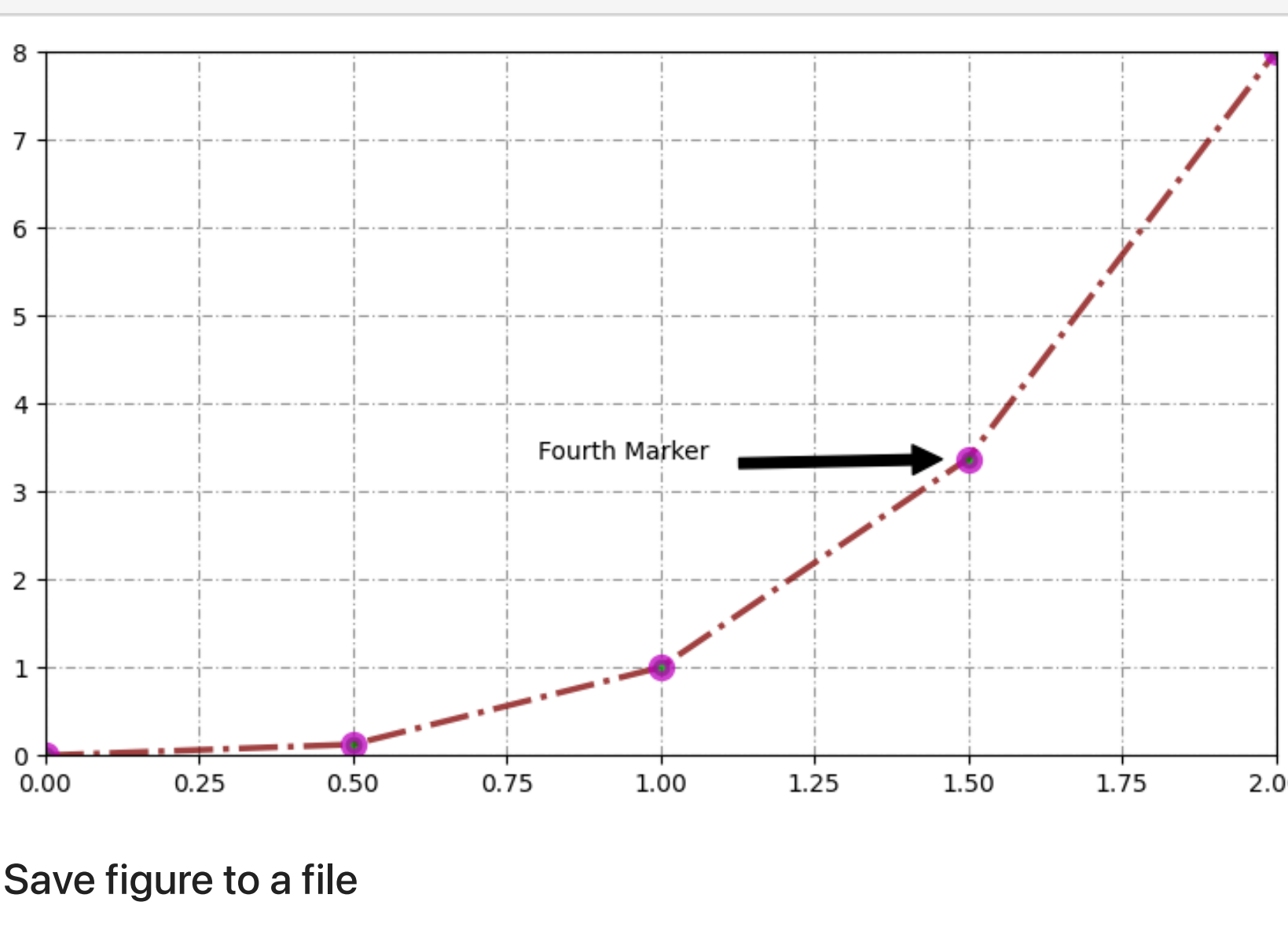
```
app_fig = plt.figure(figsize=(7, 4))
# I am taking all the space inside the figure
ax = app_fig.add_axes([0, 0, 1, 1])

_ = ax.plot(
    x_1,
    y_1,
    color="maroon",
    alpha=.75, # alpha channel (opacity, default 1)
    lw= 2.5, # line width (default 1)
    ls='-.', # line styles => https://matplotlib.org/stable/gallery/lines_bars_and_markers/linestyles.html
    marker='o', # markers, for sampling points =>https://matplotlib.org/stable/api/markers_api.html
    markersize=7, # marker size
    markerfacecolor='g',
    markeredgecolor='m',
    markeredgewidth=4
)
```



Let's suppose I want to "zoom in" in an area of my plot, and that I want to add a grid to my background

In [9…

```
app_fig = plt.figure(figsize=(7, 4))
# I am taking all the space inside the figure
ax = app_fig.add_axes([0, 0, 1, 1])

ax.plot(
    x_1,
    y_1,
    color="maroon",
    alpha=.75, # alpha channel (opacity, default 1)
    lw= 2.5, # line width (default 1)
    ls='-.', # line styles => https://matplotlib.org/stable/gallery/lines_bars_and_markers/linestyles.html
    marker='o', # markers, for sampling points =>https://matplotlib.org/stable/api/markers_api.html
    markersize=7, # marker size
    markerfacecolor='g',
    markeredgecolor='m',
    markeredgewidth=4
)

ax.set_xlim([0, 2])
ax.set_ylim([0, 8])

# color grey 60%, dashes of 5 points alternated to 1 point dashes separated by two spaces
ax.grid(True, color='0.6', dashes=(5, 2, 1, 2))

# annotate a point on the chart
_ = ax.annotate(
    "Fourth Marker",
    xy=(x_1[3], y_1[3]), # coordinates of the point to annotate
    xytext=(x_1[3] - 0.7, y_1[3]),  # coordinates of where to place the text
    arrowprops={   # arrow metadata
        'facecolor': 'black',
        'shrink': 0.1
    }
)
```



## Save figure to a file

You can save figures to a variety of formats: png, jpeg, eps, pdf, svg...

In [9…

```
app_fig.savefig('my-plot.png', dpi = 200)
```

In [9…

```
app_fig.savefig?
```

```
Signature: app_fig.savefig(fname, *, transparent=None, **kwargs)
Docstring:
Save the current figure.

Call signature::

  savefig(fname, *, transparent=None, dpi='figure', format=None,
          metadata=None, bbox_inches=None, pad_inches=0.1,
          facecolor='auto', edgecolor='auto', backend=None,
          **kwargs
         )

The available output formats depend on the backend being used.

Parameters
----------
fname : str or path-like or binary file-like
    A path, or a Python file-like object, or
    possibly some backend-dependent object such as
    `matplotlib.backends.backend_pdf.PdfPages`.

    If *format* is set, it determines the output format, and the file
    is saved as *fname*.  Note that *fname* is used verbatim, and there
    is no attempt to make the extension, if any, of *fname* match
    *format*, and no extension is appended.

    If *format* is not set, then the format is inferred from the
    extension of *fname*, if there is one.  If *format* is not
    set and *fname* has no extension, then the file is saved with
    :rc:`savefig.format` and the appropriate extension is appended to
    *fname*.

Other Parameters
----------------
transparent : bool, default: :rc:`savefig.transparent`
    If *True*, the Axes patches will all be transparent; the
    Figure patch will also be transparent unless *facecolor*
    and/or *edgecolor* are specified via kwargs.

    If *False* has no effect and the color of the Axes and
    Figure patches are unchanged (unless the Figure patch
    is specified via the *facecolor* and/or *edgecolor* keyword
    arguments in which case those colors are used).

    The transparency of these patches will be restored to their
    original values upon exit of this function.

    This is useful, for example, for displaying
    a plot on top of a colored background on a web page.

dpi : float or 'figure', default: :rc:`savefig.dpi`
    The resolution in dots per inch.  If 'figure', use the figure's
    dpi value.

format : str
    The file format, e.g. 'png', 'pdf', 'svg', ... The behavior when
    this is unset is documented under *fname*.

metadata : dict, optional
    Key/value pairs to store in the image metadata. The supported keys
    and defaults depend on the image format and backend:

    - 'png' with Agg backend: See the parameter ``metadata`` of
      `~.FigureCanvasAgg.print_png`.
    - 'pdf' with pdf backend: See the parameter ``metadata`` of
      `~.backend_pdf.PdfPages`.
    - 'svg' with svg backend: See the parameter ``metadata`` of
      `~.FigureCanvasSVG.print_svg`.
    - 'eps' and 'ps' with PS backend: Only 'Creator' is supported.
    Not supported for 'pgf', 'raw', and 'rgba' as those formats do not support
    embedding metadata.
    Does not currently support 'jpg', 'tiff', or 'webp', but may include
    embedding EXIF metadata in the future.

bbox_inches : str or `.Bbox`, default: :rc:`savefig.bbox`
    Bounding box in inches: only the given portion of the figure is
    saved.  If 'tight', try to figure out the tight bbox of the figure.

pad_inches : float or 'layout', default: :rc:`savefig.pad_inches`
    Amount of padding in inches around the figure when bbox_inches is
    'tight'. If 'layout' use the padding from the constrained or
    compressed layout engine; ignored if one of those engines is not in
    use.

facecolor : color or 'auto', default: :rc:`savefig.facecolor`
    The facecolor of the figure.  If 'auto', use the current figure
    facecolor.

edgecolor : color or 'auto', default: :rc:`savefig.edgecolor`
    The edgecolor of the figure.  If 'auto', use the current figure
    edgecolor.

backend : str, optional
    Use a non-default backend to render the file, e.g. to render a
    png file with the "cairo" backend rather than the default "agg",
    or a pdf file with the "pgf" backend rather than the default
    "pdf".  Note that the default backend is normally sufficient.  See
    :ref:`the-builtin-backends` for a list of valid backends for each
    file format.  Custom backends can be referenced as "module://...".

orientation : {'landscape', 'portrait'}
    Currently only supported by the postscript backend.

papertype : str
    One of 'letter', 'legal', 'executive', 'ledger', 'a0' through
    'a10', 'b0' through 'b10'. Only supported for postscript
    output.

bbox_extra_artists : list of `~matplotlib.artist.Artist`, optional
    A list of extra artists that will be considered when the
    tight bbox is calculated.

pil_kwargs : dict, optional
    Additional keyword arguments that are passed to
    `PIL.Image.Image.save` when saving the figure.
File:      ~/Library/Python/3.9/lib/python/site-packages/matplotlib/figure.py
Type:      method
```

## Integration with Pandas

If I want to pass columns from a pandas dataframe to a matplotlib plot I need to extract the numpy array out of the pandas dataframe

Another option is to use the `pd.DataFrame.plot()` method that leverages matplotlib under the hood

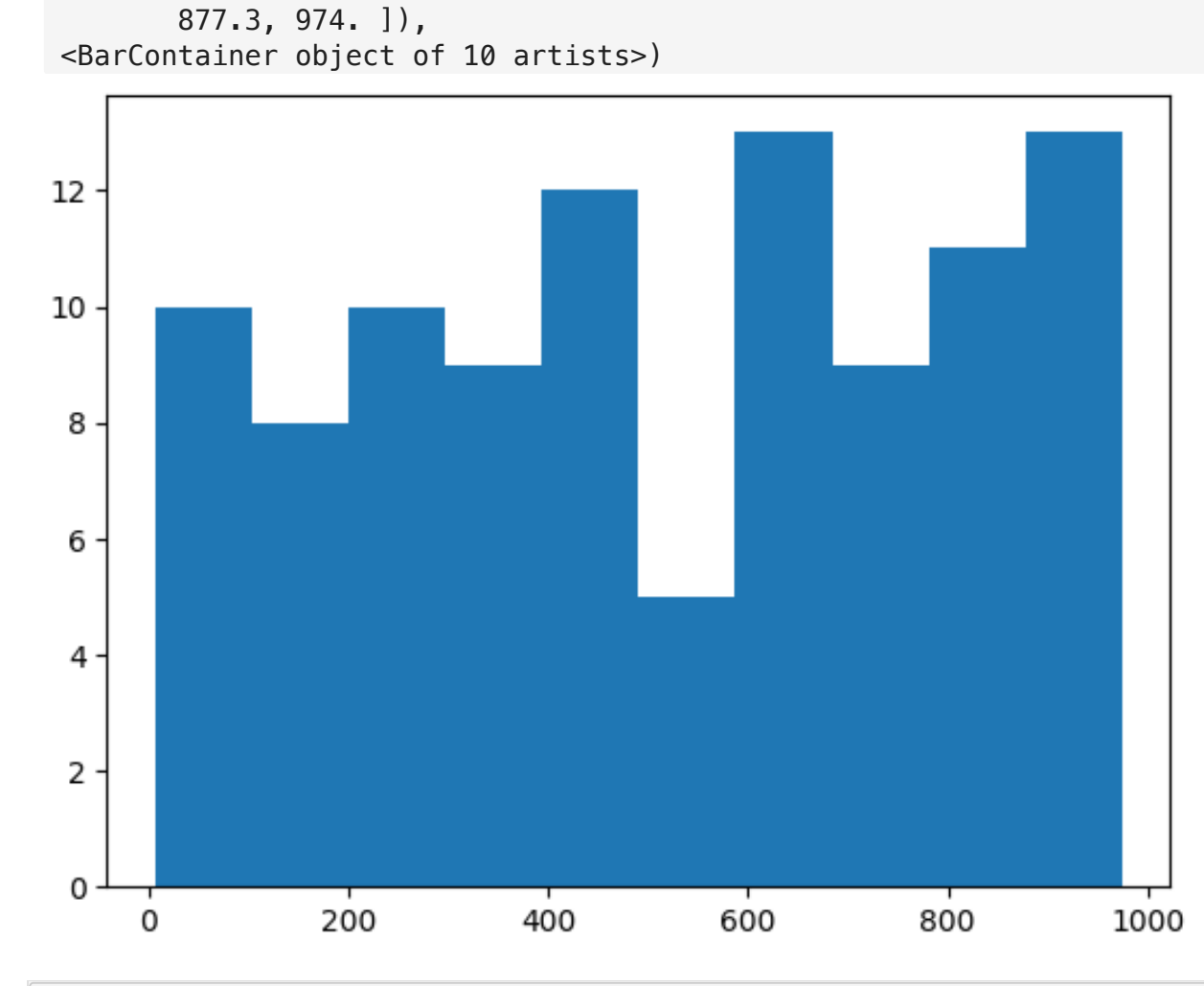
## Histograms

In [8…

```
from random import sample
data = sample(range(1, 1000), 100)
plt.hist(data)
```

Out[8…

```
(array([10.,  8., 10.,  9., 12.,  5., 13.,  9., 11., 13.]),
 array([  7. , 103.7, 200.4, 297.1, 393.8, 490.5, 587.2, 683.9, 780.6,
        877.3, 974. ]),
 <BarContainer object of 10 artists>)
```

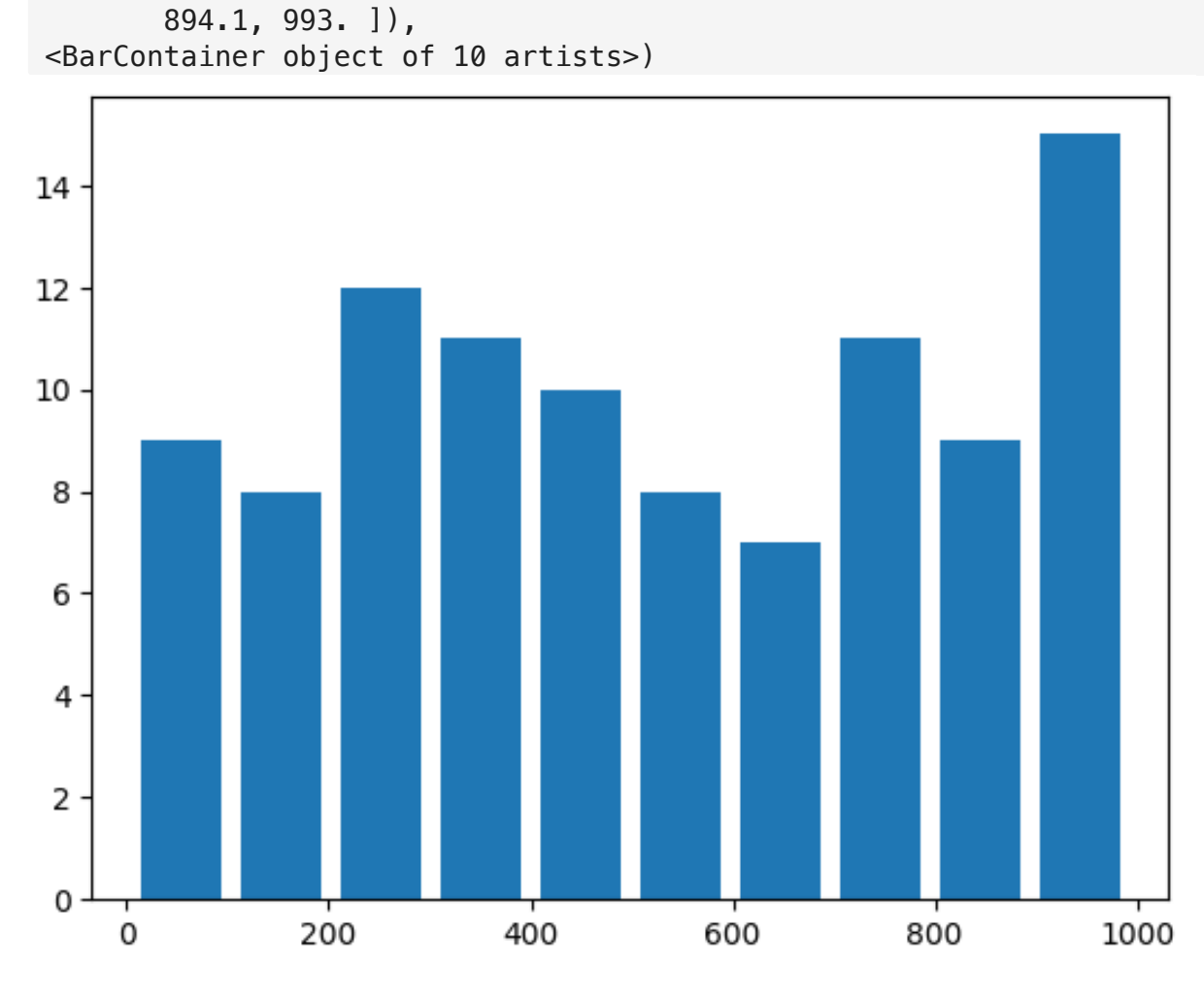


In [1…

```
from random import sample
data = sample(range(1, 1000), 100)
plt.hist(data, rwidth = 0.8)
```

Out[1…

```
(array([ 9.,  8., 12., 11., 10.,  8.,  7., 11.,  9., 15.]),
 array([  4. , 102.9, 201.8, 300.7, 399.6, 498.5, 597.4, 696.3, 795.2,
        894.1, 993. ]),
 <BarContainer object of 10 artists>)
```



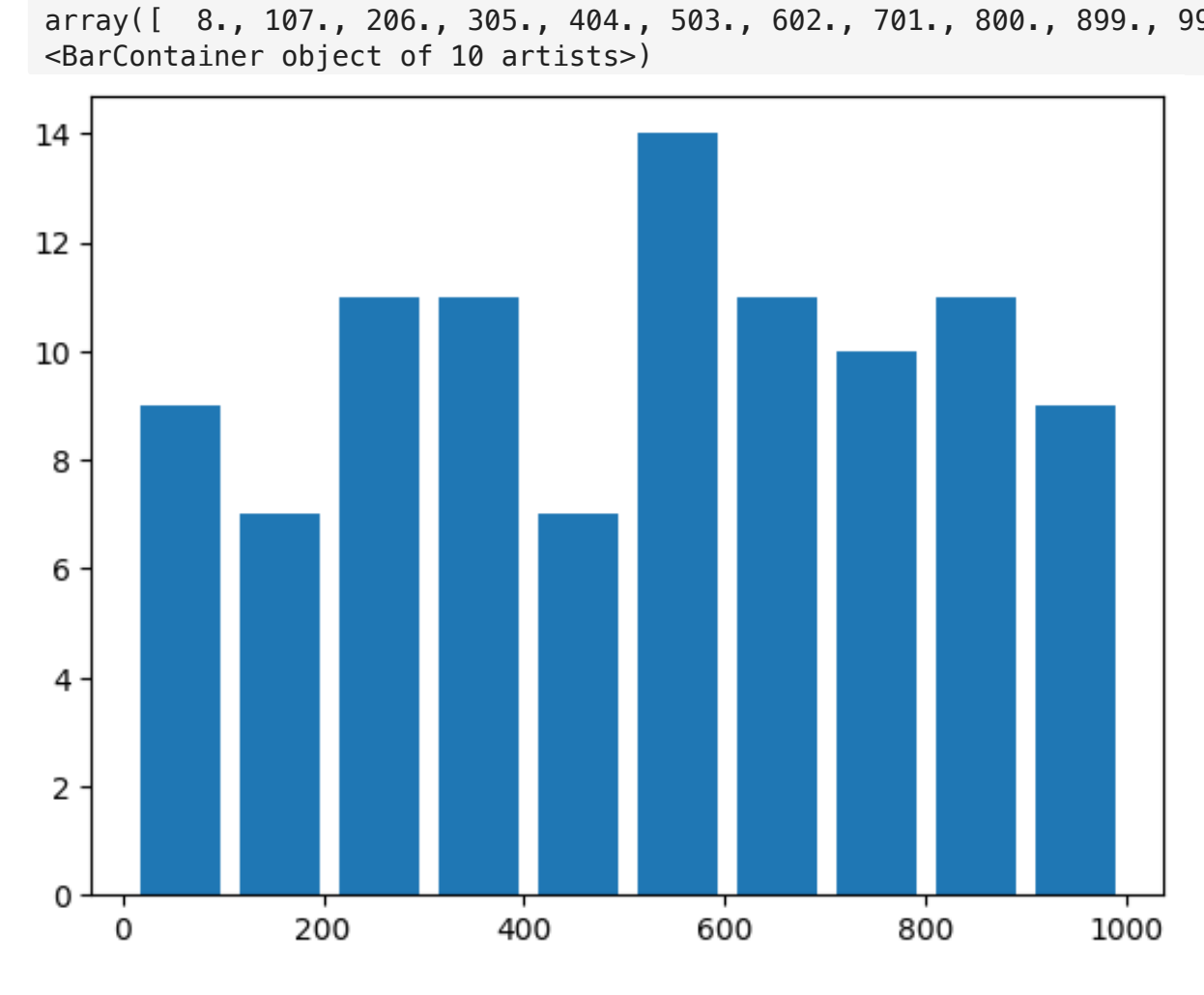
## `bins = 10` (intervals)

In [1…

```
from random import sample
data = sample(range(1, 1000), 100)
plt.hist(data, bins = 10, rwidth = 0.8)
```

Out[1…

```
(array([ 9.,  7., 11., 11.,  7., 14., 11., 10., 11.,  9.]),
 array([  8., 107., 206., 305., 404., 503., 602., 701., 800., 899., 998.]),
 <BarContainer object of 10 artists>)
```



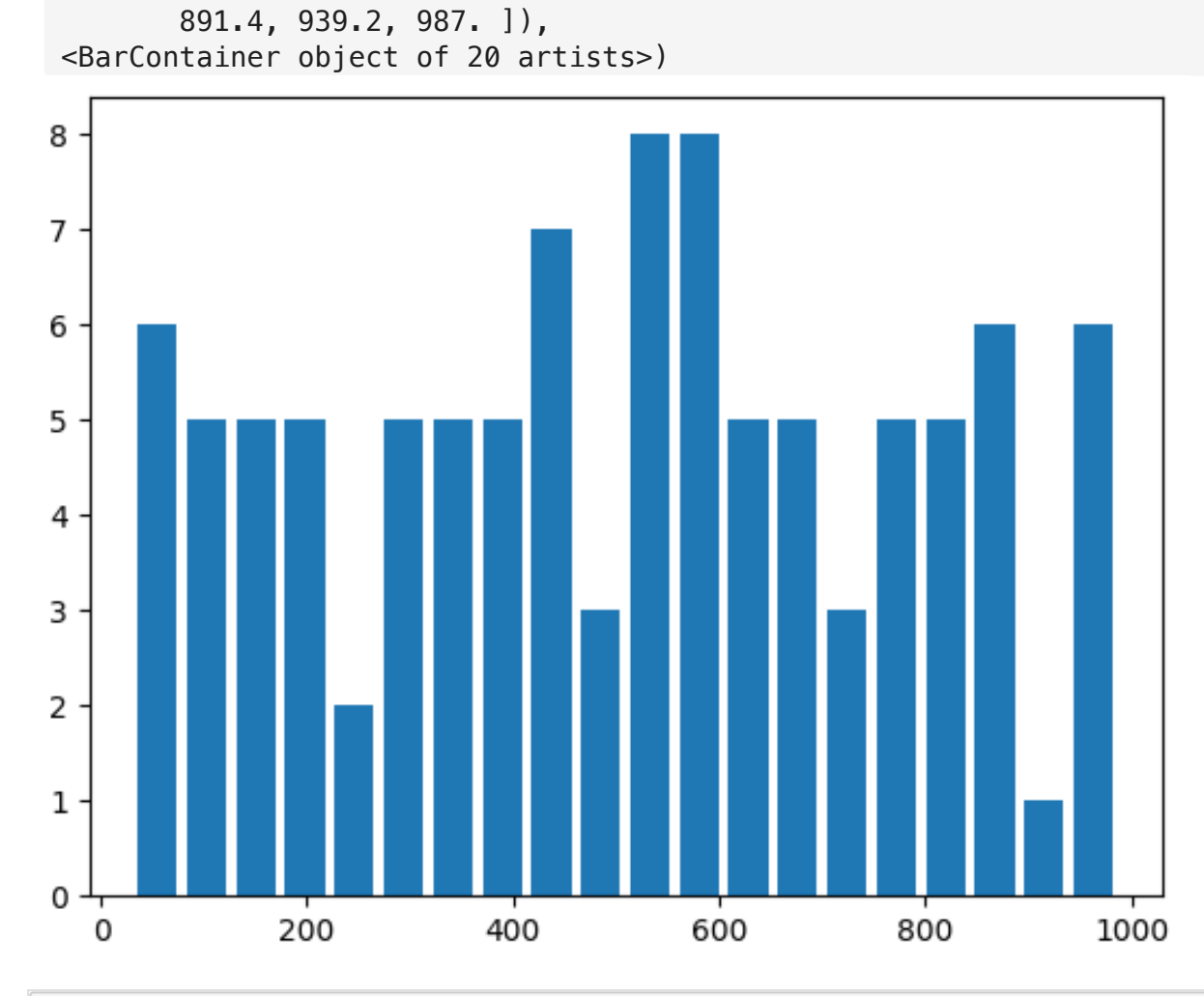
## `bins = 20` (intervals)

In [1…

```
from random import sample
data = sample(range(1, 1000), 100)
plt.hist(data, bins = 20, rwidth = 0.8)
```

Out[1…

```
(array([6., 5., 5., 5., 2., 5., 5., 5., 7., 3., 8., 8., 5., 5., 3., 5., 5.,
        6., 1., 6.]),
 array([ 31. ,  78.8, 126.6, 174.4, 222.2, 270. , 317.8, 365.6, 413.4,
        461.2, 509. , 556.8, 604.6, 652.4, 700.2, 748. , 795.8, 843.6,
        891.4, 939.2, 987. ]),
 <BarContainer object of 20 artists>)
```

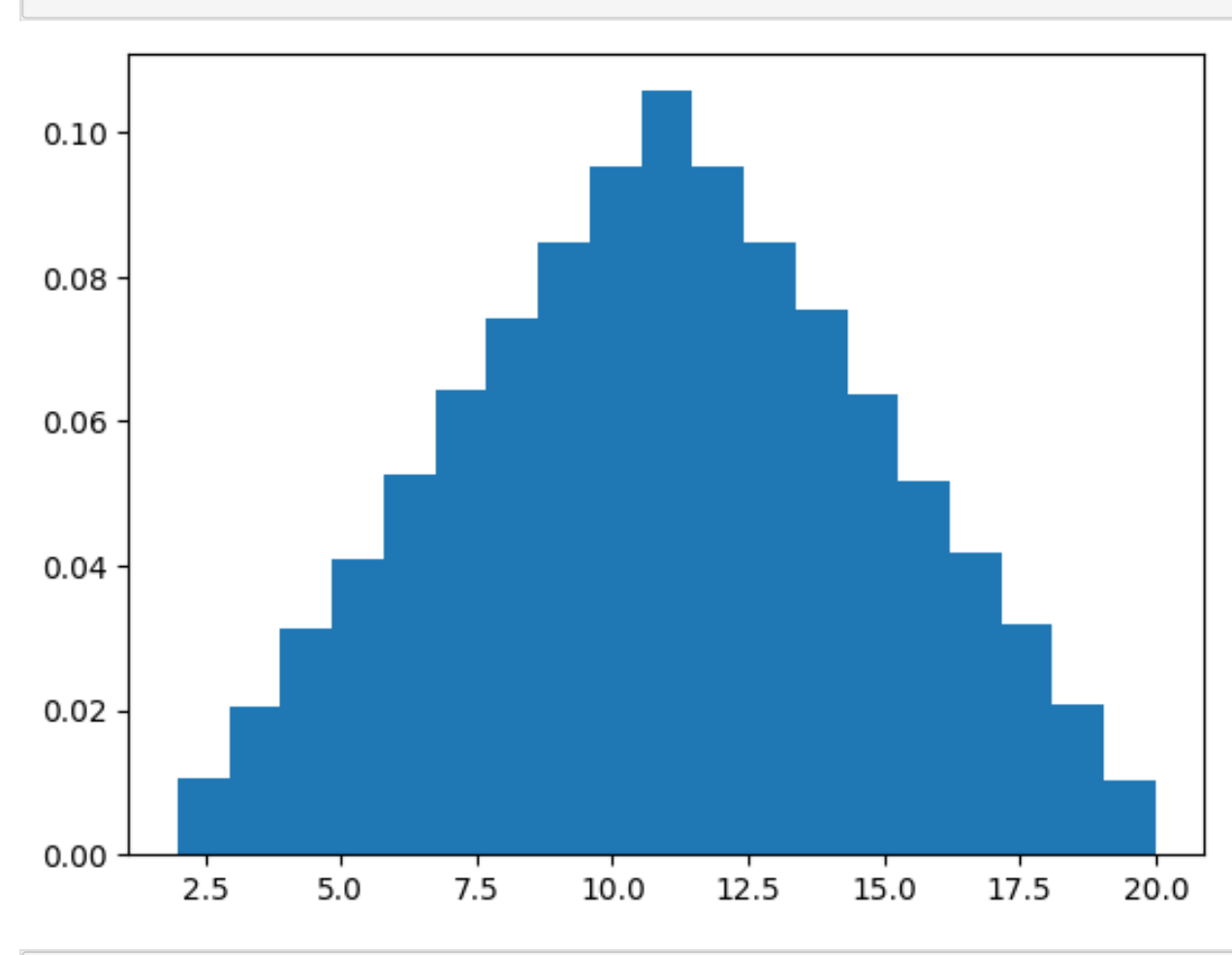


In [1…

```
np.random.seed(32)
# random arrays with values 1 to 10
randarr_1 = np.random.randint(1, 11, 100000)
randarr_2 = np.random.randint(1, 11, 100000)
randarr_3 = randarr_1 + randarr_2

# 45 different combinations (10 choose 2 = 10!/(2!(10-2)!))
```

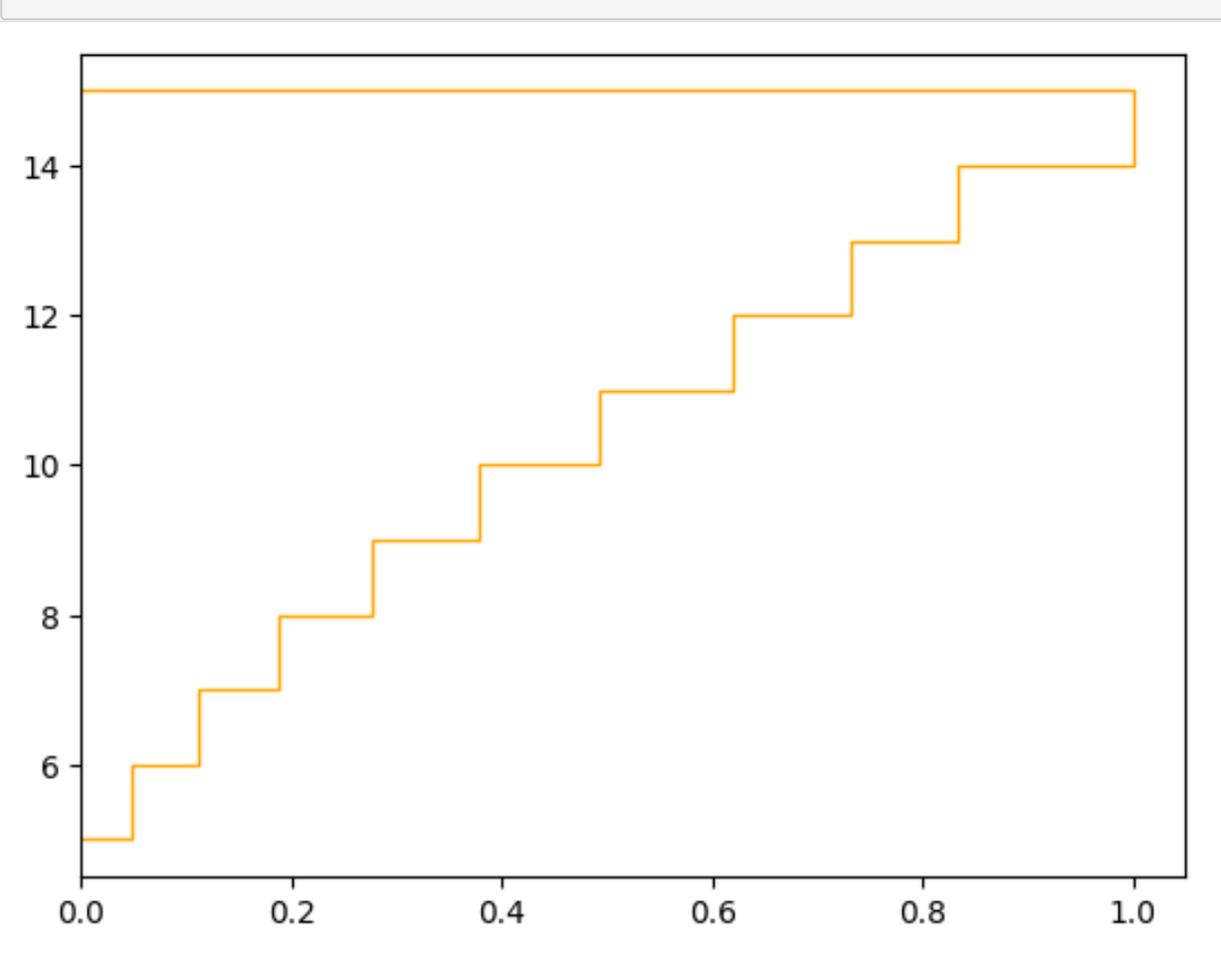
In [1…

```
_ = plt.hist(
    randarr_3,
    bins=19,
    density=True, # display the density value
    stacked=True
)
```



In [1…

```
# Some dditional options
# range: pass a tuple with interval to be binned and showed
# cumulative=True to get a cumulative distribution
# histtype="step" => you'll get it as a line plot
# orientation => you can set it to "horizontal" if you like
# color...
_ = plt.hist(
    randarr_3,
    bins=10,
    density=True, # display the density value
    stacked=True,
    range=(5, 15),
    color="orange",
    histtype="step",
    cumulative=True,
    orientation="horizontal"
)
```



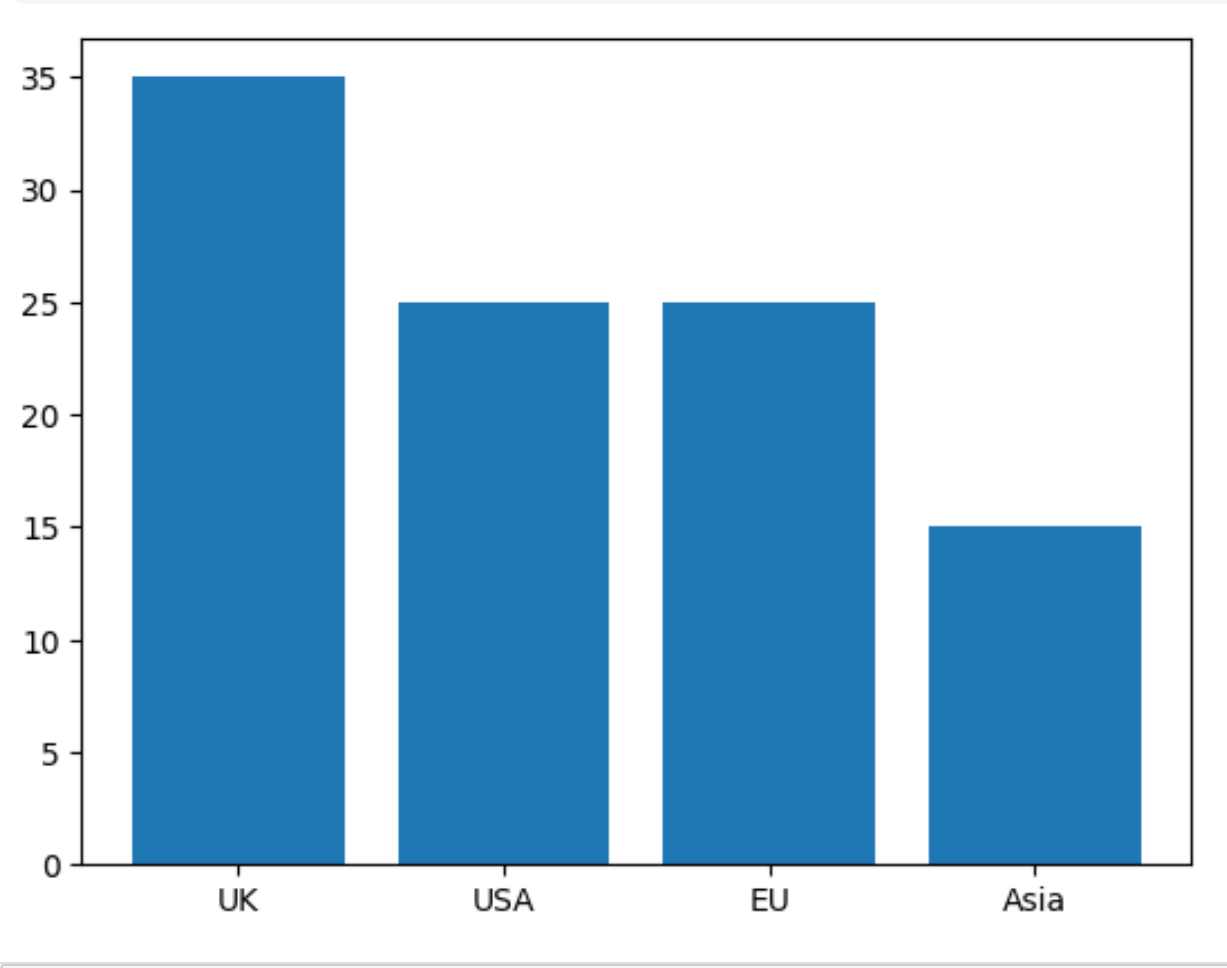
## Bar Charts

In [1…

```
data = [35, 25, 25, 15]
labels = ["UK", "USA", "EU", "Asia"]

plt.bar(labels, data)
```

Out[1…

```
<BarContainer object of 4 artists>
```

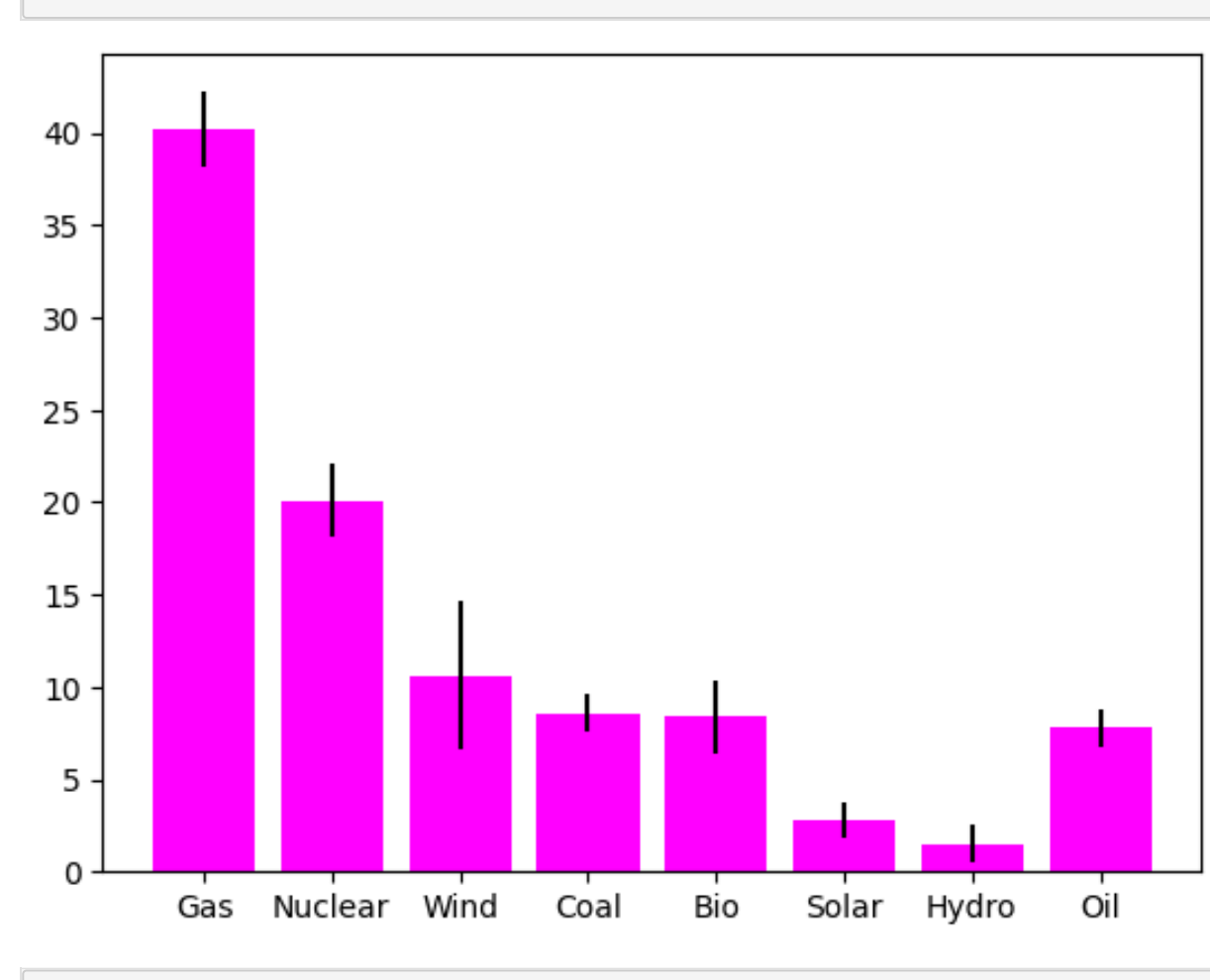


In [1…

```
# Energy statistics UK 2016
total_electricity = 357 # TWh
en_types = [
    "Gas", "Nuclear", "Wind", "Coal",
    "Bio", "Solar", "Hydro", "Oil"
]
en_values = [40.2, 20.1, 10.6, 8.6, 8.4, 2.8, 1.5, 7.8]
en_variances = [2, 2, 4, 1, 2, 1, 1, 1]
```

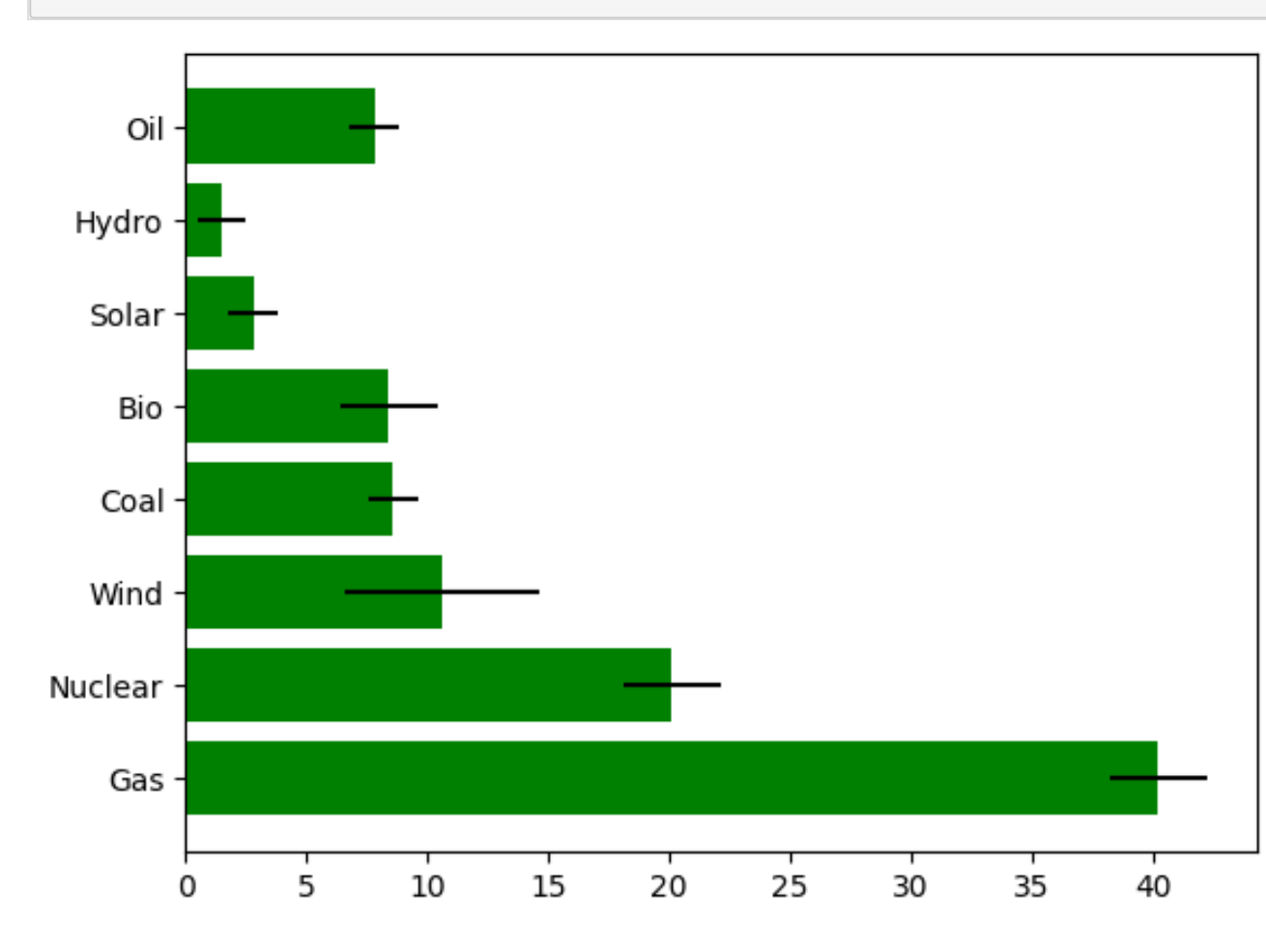
In [1…

```
_ = plt.bar(
    en_types,
    en_values,
    color="magenta",
    yerr=en_variances
)
```



In [1…

```
_ = plt.barh(
    en_types,
    en_values,
    color="green",
    xerr=en_variances
)
```



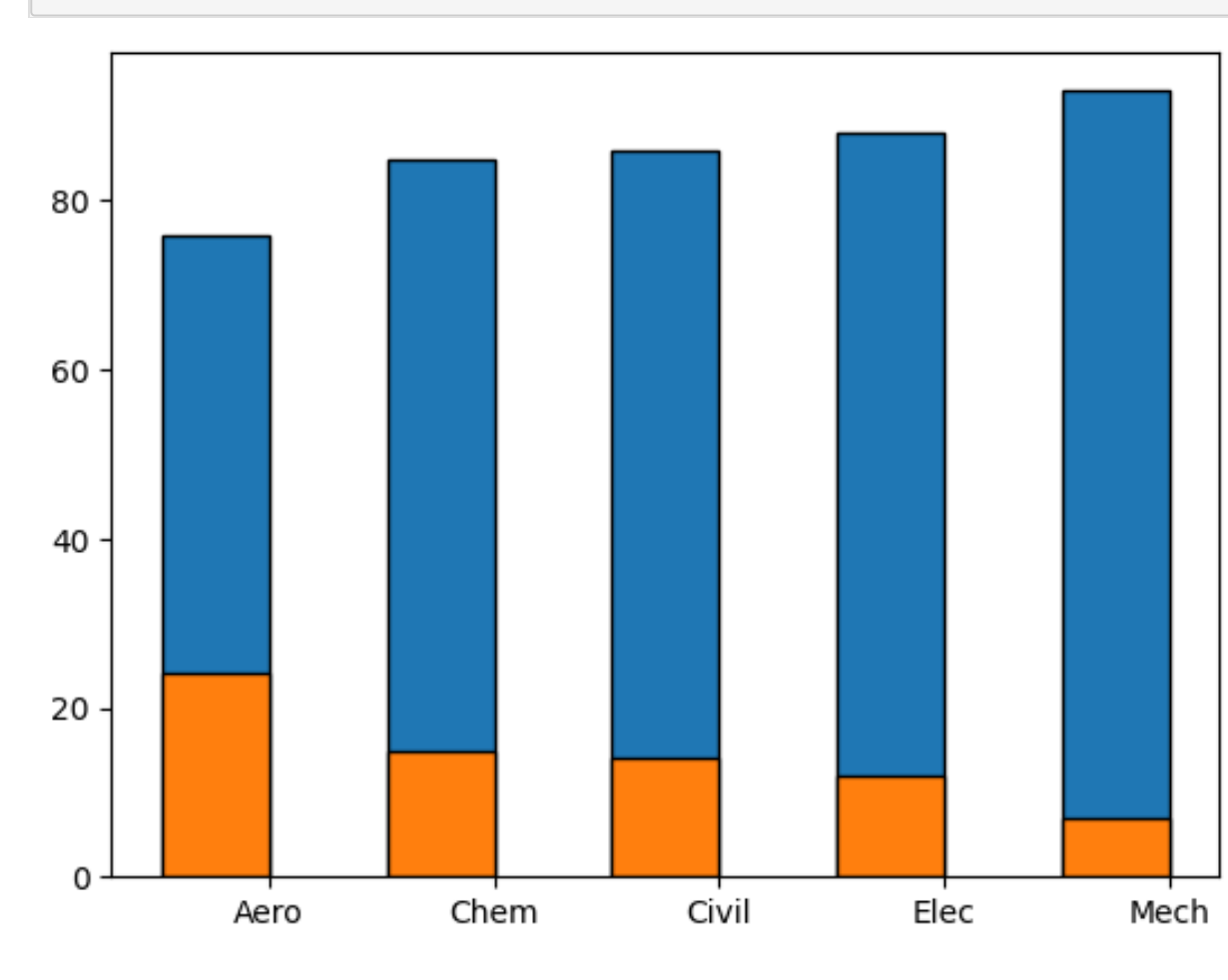
### Engineer Graduates by gender

In [1…

```
# percentage of males and females in engineers
eng_males = (76, 85, 86, 88, 93)
eng_females = (24, 15, 14, 12, 7)

spc = np.arange(5)
```

In [1…

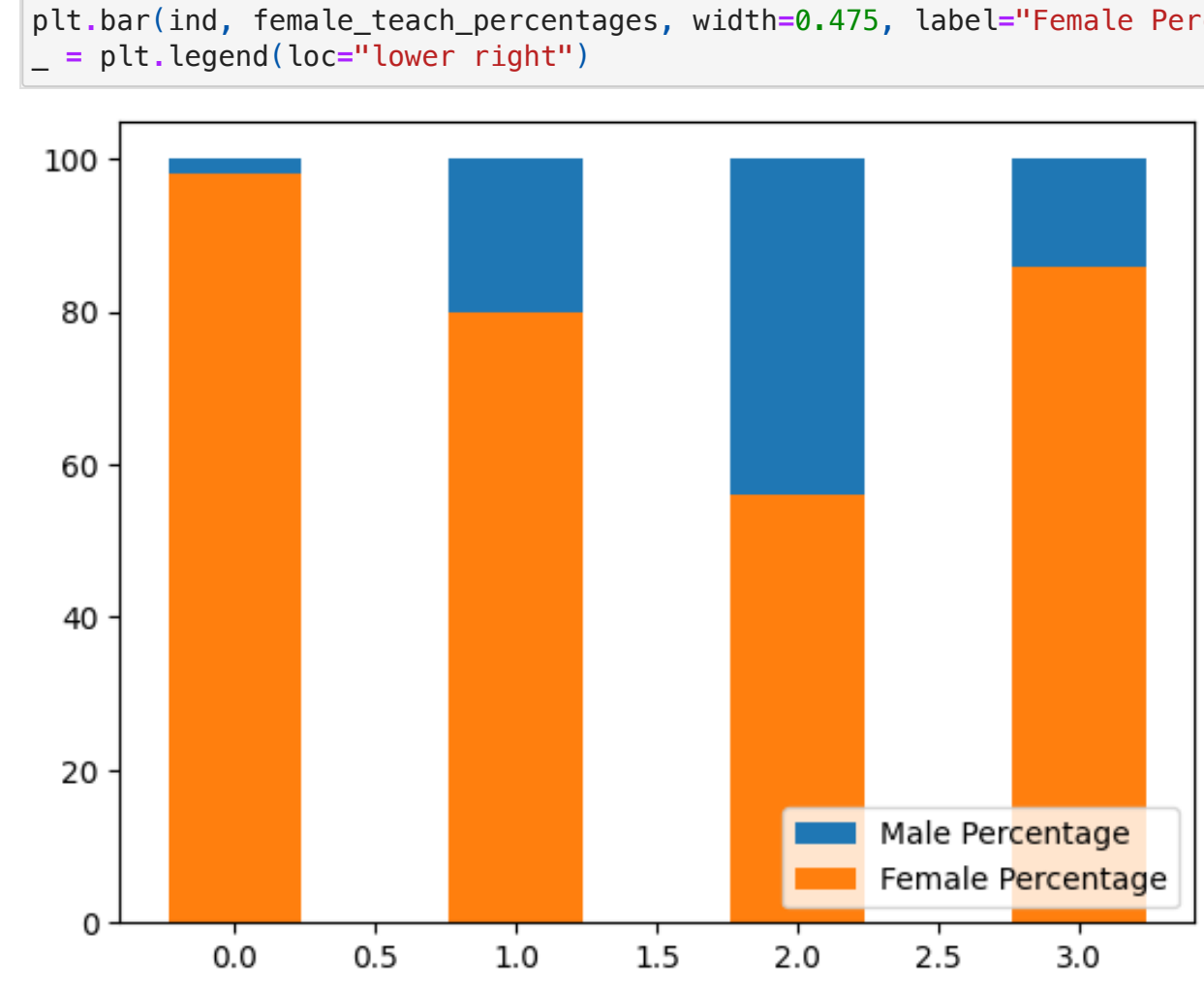
```
plt.bar(spc, eng_males, width=.475, label="% male", edgecolor="k")
plt.bar(spc, eng_females, width=.475, label="% female", edgecolor="k")
_ = plt.xticks(spc + 0.475 / 2,("Aero", "Chem", "Civil", "Elec", "Mech"))
```



### Stacked bar plot examples: teachers by gender

In [1…

```
teaching_type = ["Kindergarden", "Elementary", "Secondary", "Special Education"]
male_teach_percentages = np.array([2, 20, 44, 14])
female_teach_percentages = np.array([98, 80, 56, 86])

ind = [x for x, _ in enumerate(teaching_type)]
plt.bar(ind, male_teach_percentages, width=0.475, label="Male Percentage", bottom=female_teach_percentages)
plt.bar(ind, female_teach_percentages, width=0.475, label="Female Percentage")
_ = plt.legend(loc="lower right")
```



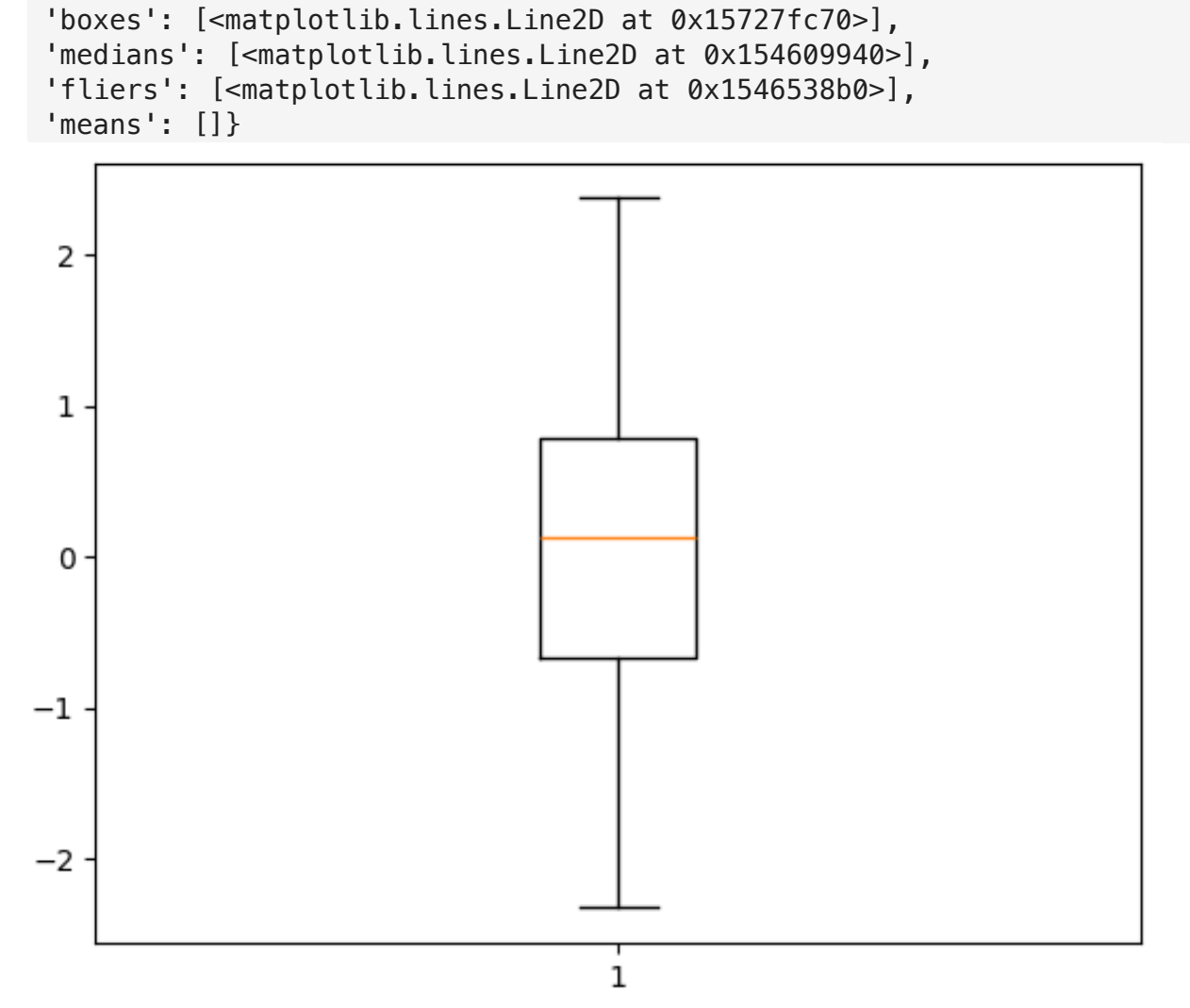
## Boxplots

In [1…

```
std = 1
data = [np.random.normal(0, std, 100)]
plt.boxplot(data, vert = True)
```

Out[1…

```
{'whiskers': [<matplotlib.lines.Line2D at 0x15450a8e0>,
  <matplotlib.lines.Line2D at 0x1547015b0>],
 'caps': [<matplotlib.lines.Line2D at 0x1546ea2e0>,
  <matplotlib.lines.Line2D at 0x1546eab80>],
 'boxes': [<matplotlib.lines.Line2D at 0x15727fc70>],
 'medians': [<matplotlib.lines.Line2D at 0x154609940>],
 'fliers': [<matplotlib.lines.Line2D at 0x1546538b0>],
 'means': []}
```

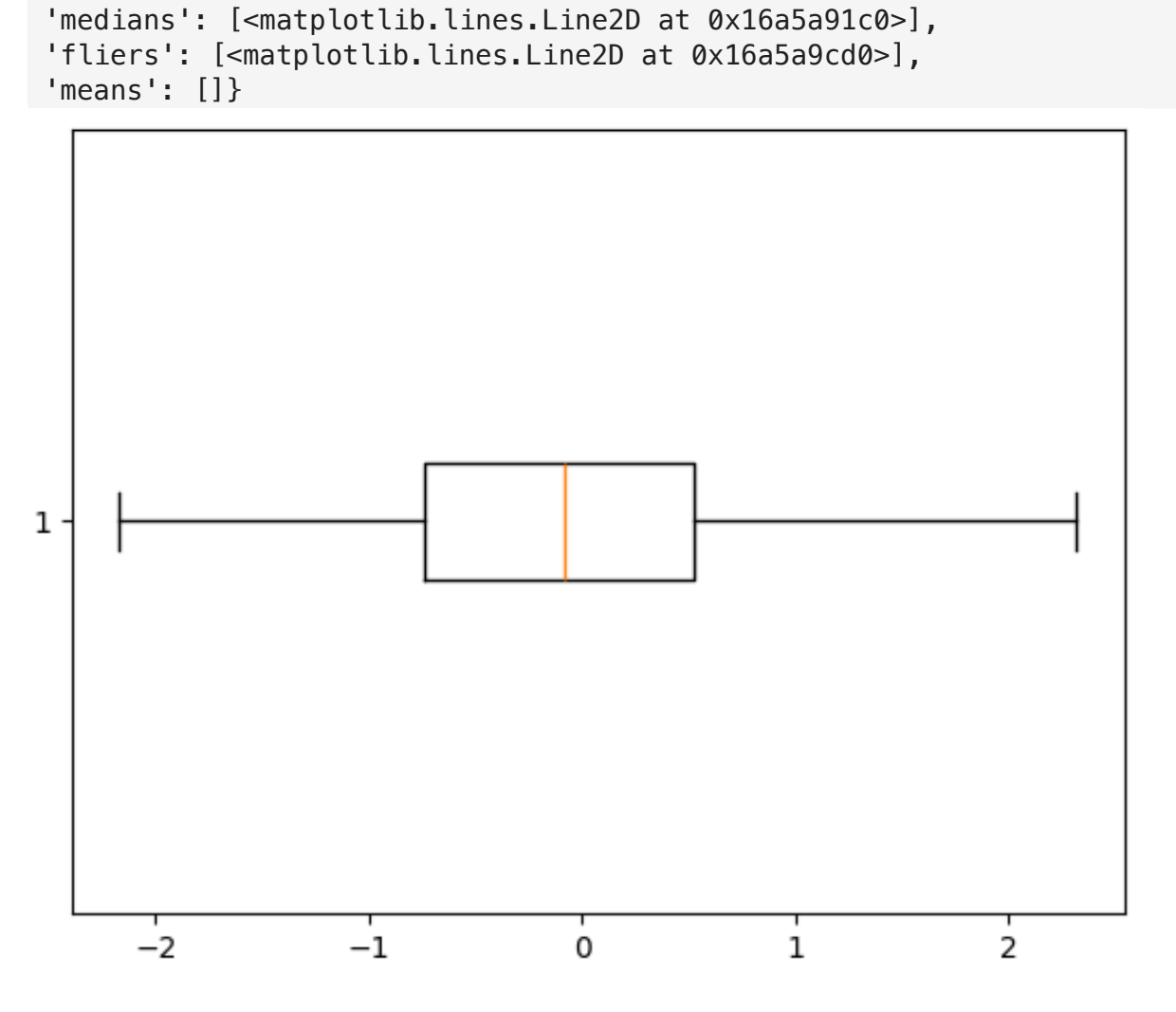


In [1…

```
std = 1
data = [np.random.normal(0, std, 100)]
plt.boxplot(data, vert = False)
```

Out[1…

```
{'whiskers': [<matplotlib.lines.Line2D at 0x16a5b63a0>,
  <matplotlib.lines.Line2D at 0x16a5b6cd0>],
 'caps': [<matplotlib.lines.Line2D at 0x16a5b6100>,
  <matplotlib.lines.Line2D at 0x16a5b6d90>],
 'boxes': [<matplotlib.lines.Line2D at 0x16a5b60a0>],
 'medians': [<matplotlib.lines.Line2D at 0x16a5a91c0>],
 'fliers': [<matplotlib.lines.Line2D at 0x16a5a9cd0>],
 'means': []}
```



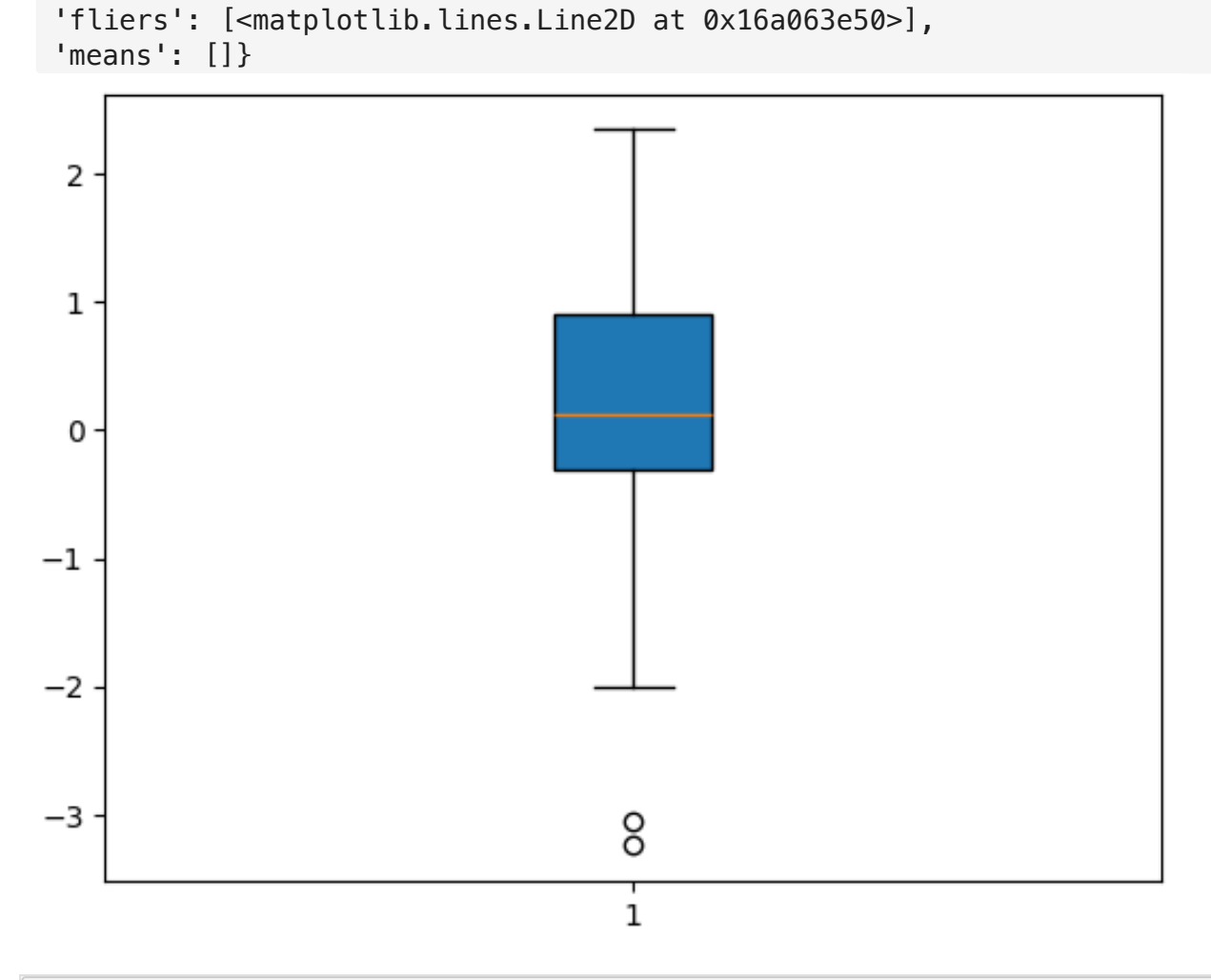
## adding colour via `patch_artist`

In [1…

```
std = 1
data = [np.random.normal(0, std, 100)]
plt.boxplot(data,vert=True,patch_artist=True)
```

Out[1…

```
{'whiskers': [<matplotlib.lines.Line2D at 0x16a0b4760>,
  <matplotlib.lines.Line2D at 0x16a0b4b50>],
 'caps': [<matplotlib.lines.Line2D at 0x16a0b4a30>,
  <matplotlib.lines.Line2D at 0x16a0b41f0>],
 'boxes': [<matplotlib.patches.PathPatch at 0x16a0b46d0>],
 'medians': [<matplotlib.lines.Line2D at 0x169f89730>],
 'fliers': [<matplotlib.lines.Line2D at 0x16a063e50>],
 'means': []}
```

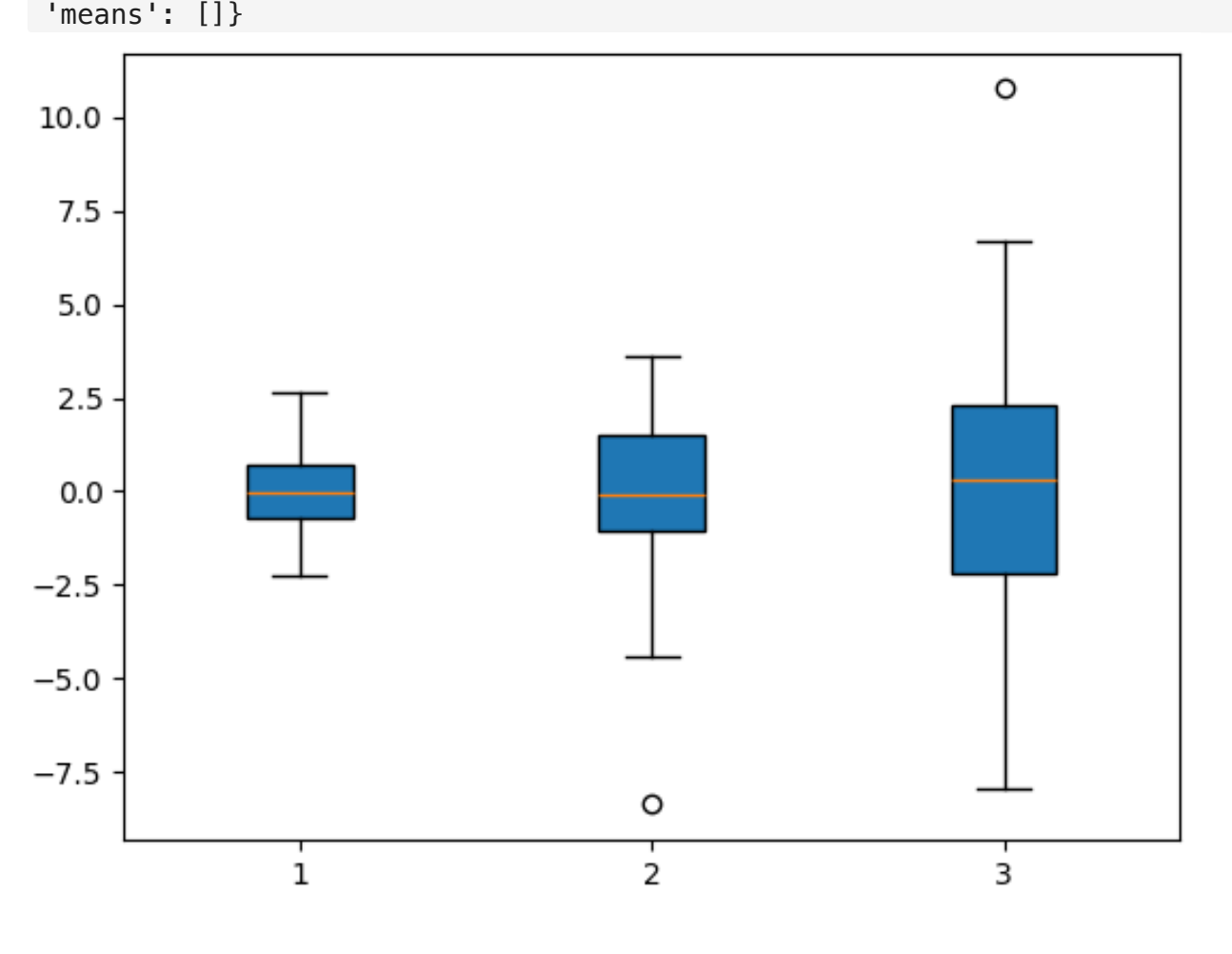


In [1…

```
std = 1
data = [np.random.normal(0, std, 100)
        for std in range(1,4)]
plt.boxplot(data,vert=True,patch_artist=True)
```

Out[1…

```
{'whiskers': [<matplotlib.lines.Line2D at 0x169f08040>,
  <matplotlib.lines.Line2D at 0x169f08460>,
  <matplotlib.lines.Line2D at 0x169f20bb0>,
  <matplotlib.lines.Line2D at 0x169f20340>,
  <matplotlib.lines.Line2D at 0x169efe8e0>,
  <matplotlib.lines.Line2D at 0x169efedc0>],
 'caps': [<matplotlib.lines.Line2D at 0x169f08f10>,
  <matplotlib.lines.Line2D at 0x169f08220>,
  <matplotlib.lines.Line2D at 0x169e85b80>,
  <matplotlib.lines.Line2D at 0x169d618b0>,
  <matplotlib.lines.Line2D at 0x169efed00>,
  <matplotlib.lines.Line2D at 0x169efe7c0>],
 'boxes': [<matplotlib.patches.PathPatch at 0x16a043280>,
  <matplotlib.patches.PathPatch at 0x169f20520>,
  <matplotlib.patches.PathPatch at 0x169d75160>],
 'medians': [<matplotlib.lines.Line2D at 0x169f08f40>,
  <matplotlib.lines.Line2D at 0x169d61760>,
  <matplotlib.lines.Line2D at 0x169efe130>],
 'fliers': [<matplotlib.lines.Line2D at 0x169f20460>,
  <matplotlib.lines.Line2D at 0x169e3faf0>,
  <matplotlib.lines.Line2D at 0x169ef64f0>],
 'means': []}
```



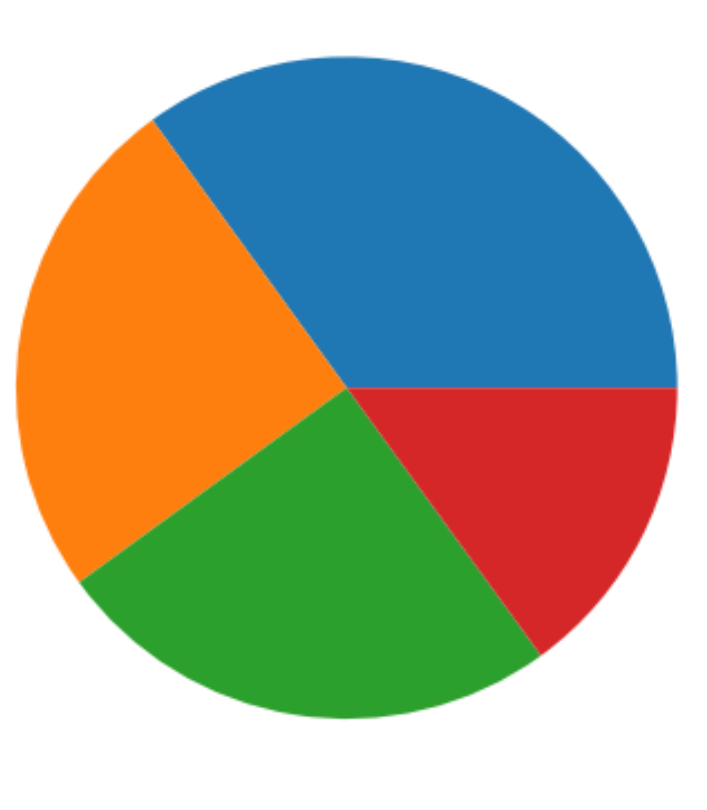
## Pie Charts

In [1…

```
data = [35, 25, 25, 15]

plt.pie(data)
```

Out[1…

```
([<matplotlib.patches.Wedge at 0x157227f70>,
  <matplotlib.patches.Wedge at 0x15716ab80>,
  <matplotlib.patches.Wedge at 0x169b438b0>,
  <matplotlib.patches.Wedge at 0x169b43fd0>],
 [Text(0.4993895680663529, 0.9801071672559597, ''),
  Text(-1.086457168210212, 0.17207795223283895, ''),
  Text(-0.172077952232839, -1.086457168210212, ''),
  Text(0.9801071672559598, -0.4993895680663526, '')])
```



In [1…

```
data
```

Out[1…

```
[35, 25, 25, 15]
```

In [1…

```
sum(data)
```

Out[1…

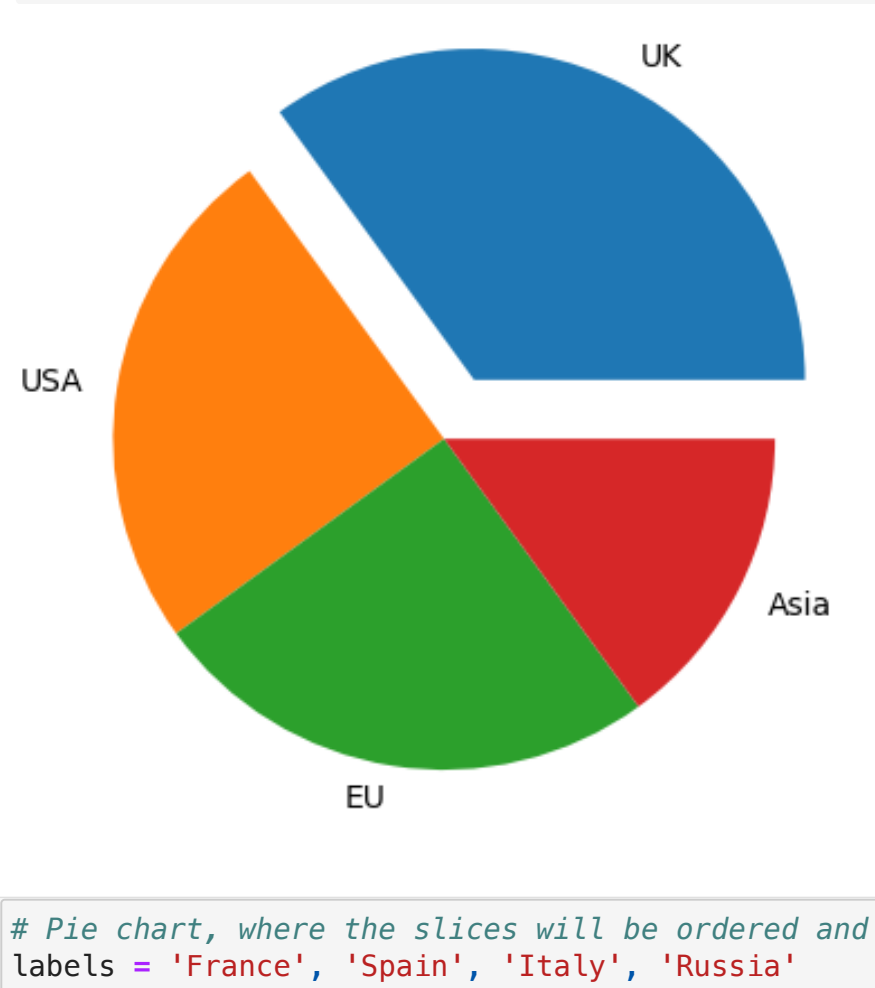
```
100
```

In [1…

```
data = [35, 25, 25, 15]
l = ["UK", "USA", "EU", "Asia"]
ex = [0.2, 0, 0, 0]
plt.pie(data, labels=l, explode=ex)
```

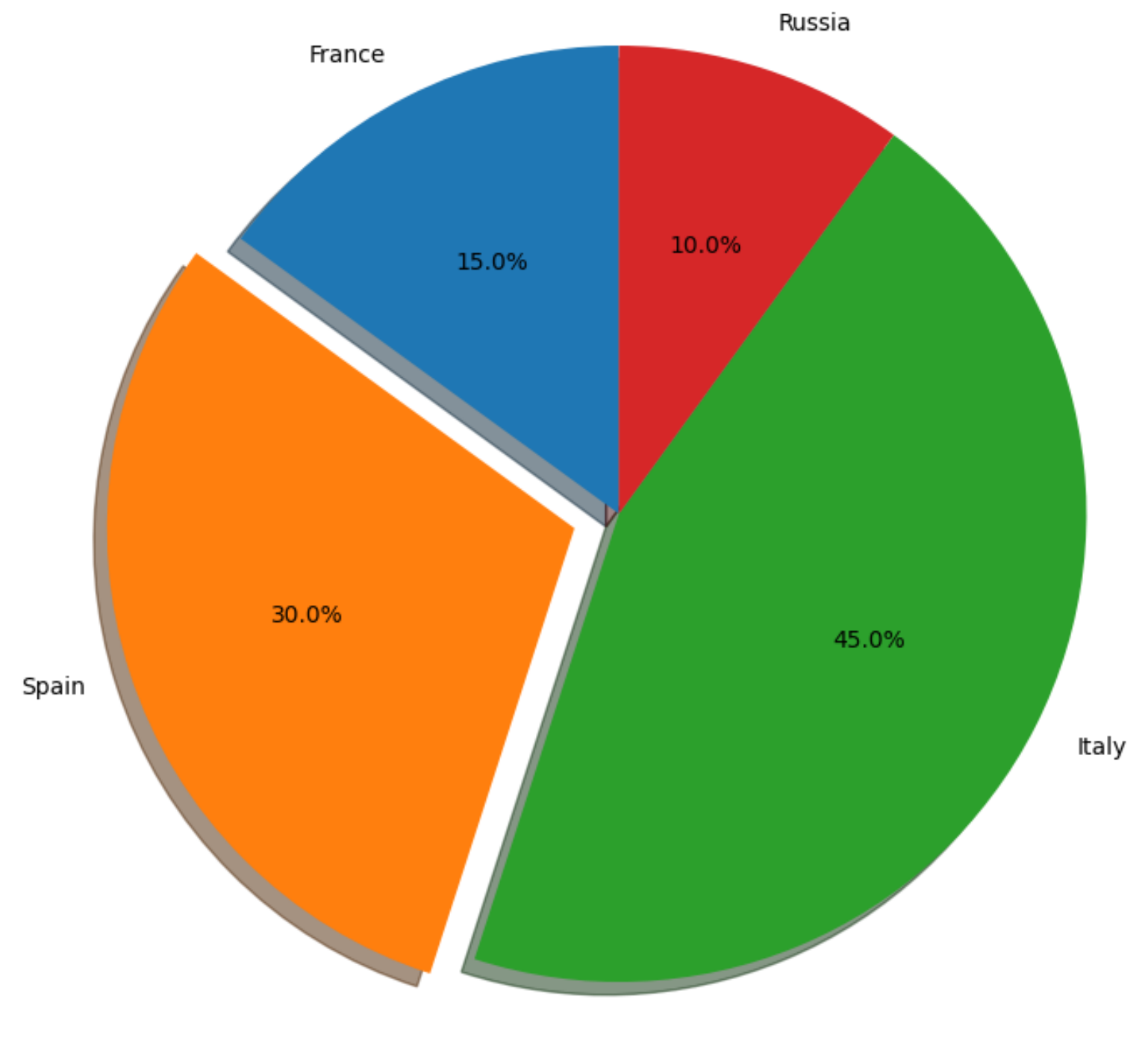
Out[1…

```
([<matplotlib.patches.Wedge at 0x169aaf4c0>,
  <matplotlib.patches.Wedge at 0x169aaf5e0>,
  <matplotlib.patches.Wedge at 0x169918c10>,
  <matplotlib.patches.Wedge at 0x15705c3d0>],
 [Text(0.5901876713511442, 1.1583084703934068, 'UK'),
  Text(-1.086457168210212, 0.17207795223283895, 'USA'),
  Text(-0.172077952232839, -1.086457168210212, 'EU'),
  Text(0.9801071672559598, -0.4993895680663526, 'Asia')])
```



In [1…

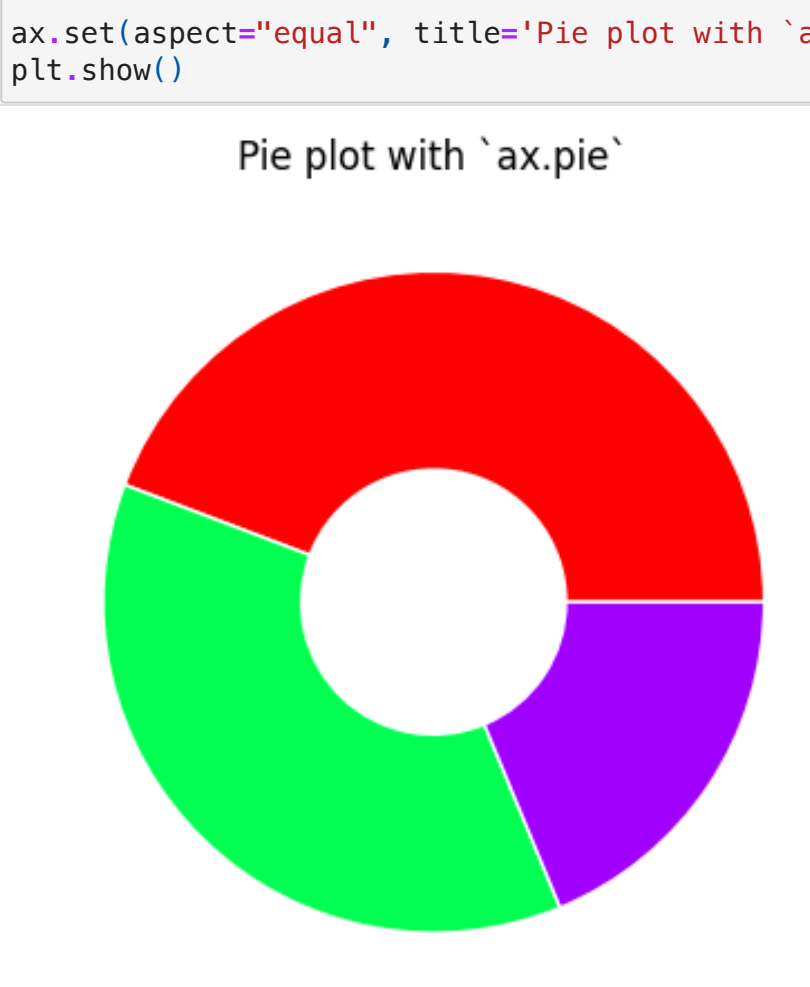
```
# Pie chart, where the slices will be ordered and plotted counter-clockwise:
labels = 'France', 'Spain', 'Italy', 'Russia'
sizes = [15, 30, 45, 10]
explode = (0, 0.1, 0, 0)  # only "explode" the 2nd slice (i.e. 'Spain')

fig1, ax1 = plt.subplots(figsize=(8, 8))
ax1.pie(
    sizes,
    explode=explode,
    labels=labels,
    autopct='%1.1f%%', # auto labels the percentages in format (you could also pass a function)
    shadow=True,   # shadow effect
    startangle=90 # custom start angle
)
ax1.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle.

plt.show()
```



### Doughnut chart

The effect of the doughnut shape is achieved by setting a width to the pie's wedges through the wedgeprops argument.

In [1…
```
fig, ax = plt.subplots()

size = 0.6
vals = np.array([92, 77, 39])

# pick a colour map
cmap = mpl.cm.hsv
outer_colors = cmap(np.arange(3)*100)

ax.pie(
    vals,
    radius=1,
    colors=outer_colors,
    wedgeprops=dict(width=size, edgecolor='w')
)

ax.set(aspect="equal", title='Pie plot with `ax.pie`')
plt.show()
```



## Timeseries (?)

In [1…
```
amzn_data = pd.read_csv("AMZN-quote-history.csv")
amzn_data
```

Out[1…

| | Date | Open | High | Low | Close | Adj Close | Volume |
|---|---|---|---|---|---|---|---|
| **0** | 2021-01-19 | 3107.000000 | 3145.000000 | 3096.000000 | 3120.760010 | 3120.760010 | 3305100 |
| **1** | 2021-01-20 | 3181.989990 | 3279.800049 | 3175.000000 | 3263.379883 | 3263.379883 | 5309800 |
| **2** | 2021-01-21 | 3293.000000 | 3348.550049 | 3289.570068 | 3306.989990 | 3306.989990 | 4936100 |
| **3** | 2021-01-22 | 3304.310059 | 3321.909912 | 3283.159912 | 3292.229980 | 3292.229980 | 2821900 |
| **4** | 2021-01-25 | 3328.500000 | 3363.889893 | 3243.149902 | 3294.000000 | 3294.000000 | 3749800 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **249** | 2022-01-12 | 3331.500000 | 3337.560059 | 3288.340088 | 3304.139893 | 3304.139893 | 2501500 |
| **250** | 2022-01-13 | 3305.010010 | 3324.429932 | 3221.820068 | 3224.280029 | 3224.280029 | 2609400 |
| **251** | 2022-01-14 | 3203.000000 | 3245.000000 | 3196.010010 | 3242.760010 | 3242.760010 | 2295800 |
| **252** | 2022-01-18 | 3182.100098 | 3194.689941 | 3153.290039 | 3178.350098 | 3178.350098 | 3364600 |
| **253** | 2022-01-19 | 3175.239990 | 3185.000000 | 3136.000000 | 3164.040039 | 3164.040039 | 1176420 |

254 rows × 7 columns

In [1…
```
amzn_data.info()
```

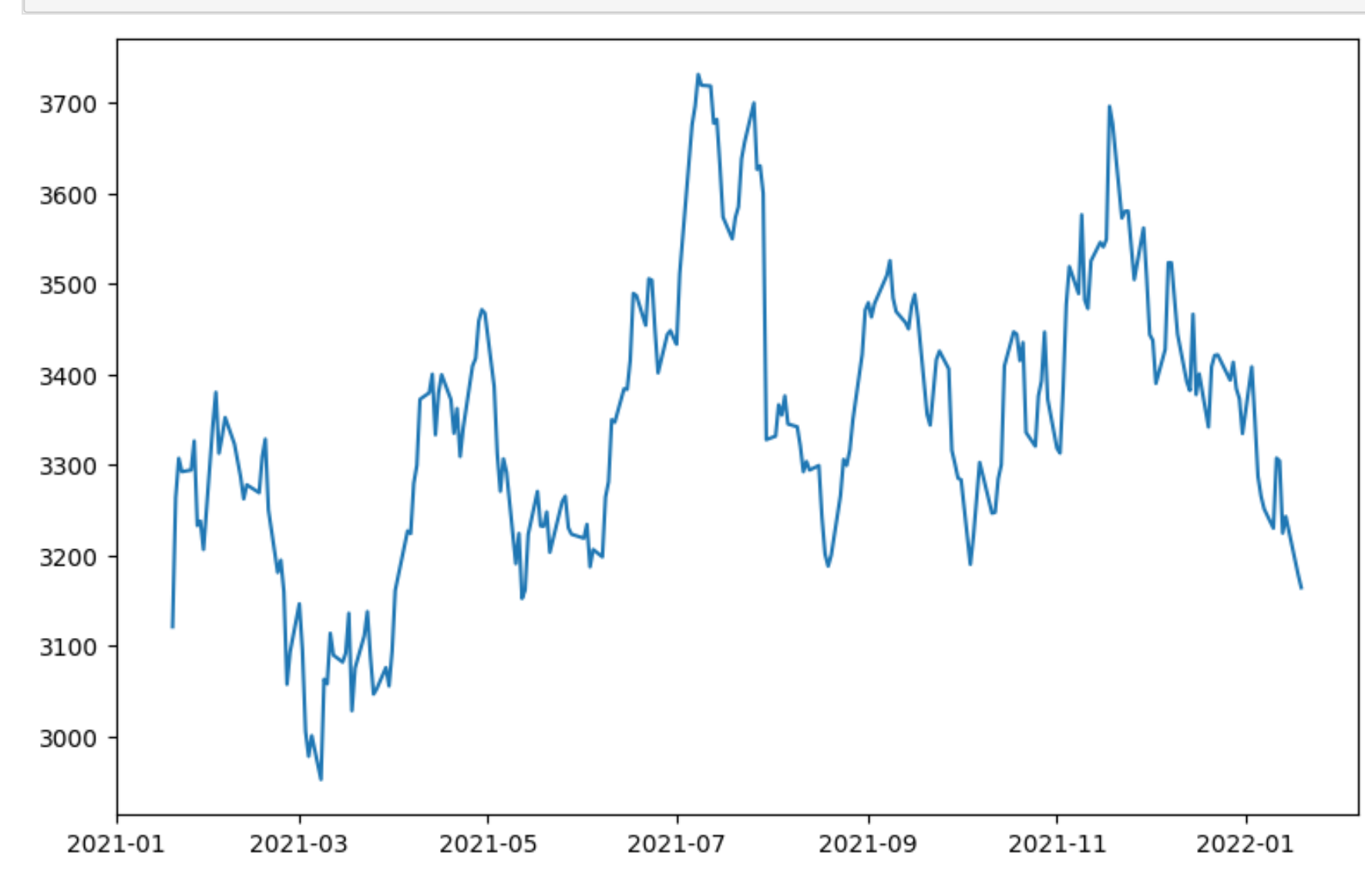
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 254 entries, 0 to 253
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Date       254 non-null    object 
 1   Open       254 non-null    float64
 2   High       254 non-null    float64
 3   Low        254 non-null    float64
 4   Close      254 non-null    float64
 5   Adj Close  254 non-null    float64
 6   Volume     254 non-null    int64  
dtypes: float64(5), int64(1), object(1)
memory usage: 14.0+ KB
```

In [1…
```
amzn_data_np = amzn_data.to_numpy()
# get the close column
amzn_cp = amzn_data_np[:, 4]
```

In [1…
```
# convert the strings containing the dates to pandas datetime object
amzn_dt = pd.to_datetime(amzn_data.Date)
amzn_dt.dt.date
```

Out[1…
```
0      2021-01-19
1      2021-01-20
2      2021-01-21
3      2021-01-22
4      2021-01-25
          ...    
249    2022-01-12
250    2022-01-13
251    2022-01-14
252    2022-01-18
253    2022-01-19
Name: Date, Length: 254, dtype: object
```

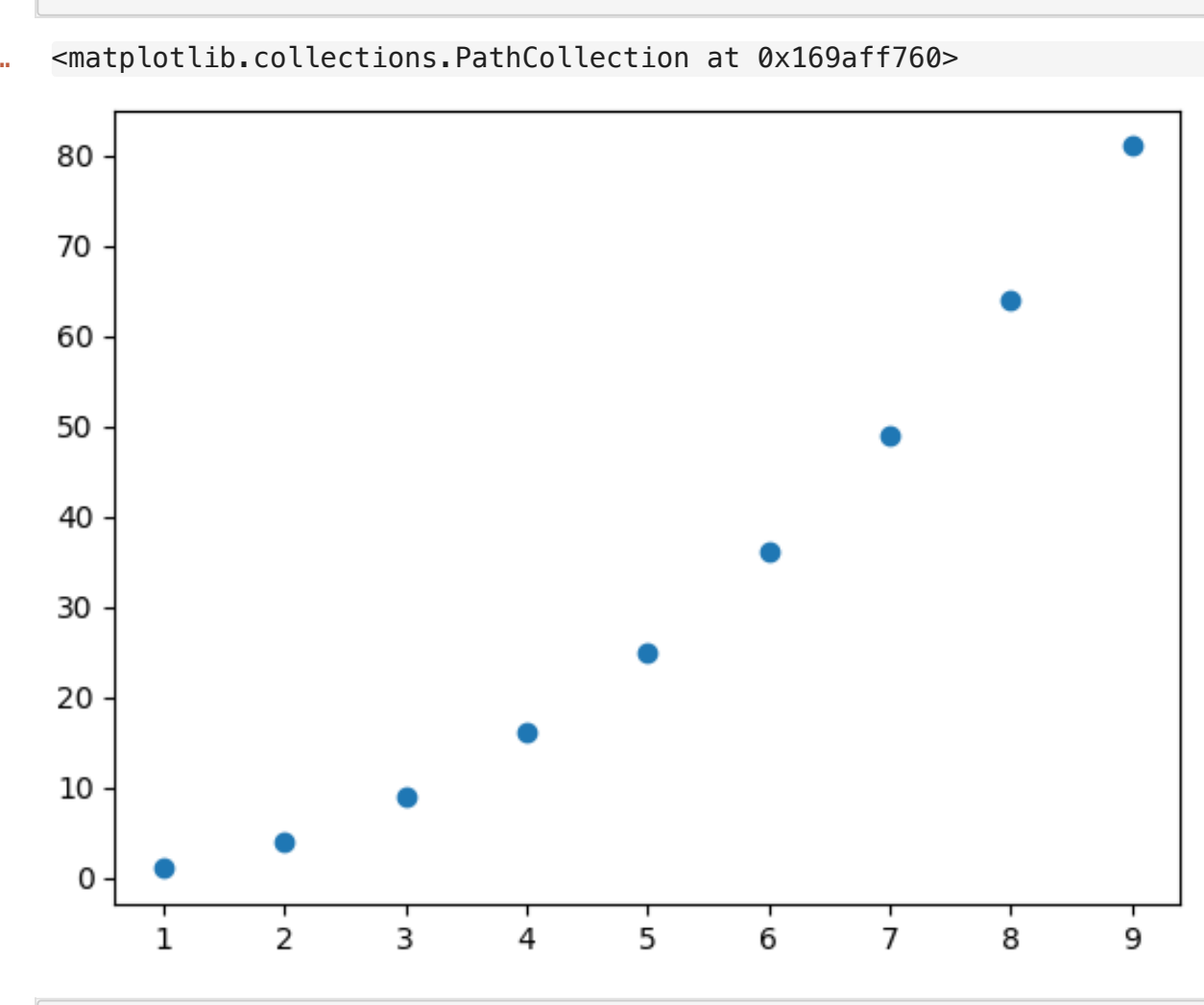
In [1…
```
fig = plt.figure(figsize=(8, 5))
ax = fig.add_axes([0.1, 0.1, 0.9, 0.9])
_ = plt.plot(amzn_dt.dt.date.to_numpy(), amzn_cp)
```



## Scatterplots

In [1…
```
plt.scatter(x, y)
```

Out[1…
```
<matplotlib.collections.PathCollection at 0x169aff760>
```



In [1…
```
# country array as of 26 Jan 2022
covid_stats = [
    # country, total_cases, total_deaths, total_tests, population
    ("USA",  69808350, 880976, 871652953, 334008890),
    ("India", 38218773, 487719, 709356830, 1401013398),
    ("Brazil", 23420861, 621927, 63776166, 214899473),
    ("UK", 15506750, 152872, 437010764, 68438346),
    ("France", 15175464, 127869, 216918555, 65497250),
    ("Russia", 10938261, 324060, 247600000, 146031401),
    ("Turkey", 10664372, 85253, 126263774, 85744257),
    ("Italy", 9219391, 142205, 159001733, 60324090),
    ("Spain", 8676916, 91437, 66213858, 46782833),
    ("Germany", 8262398, 116868, 89622218, 84198923),
    ("Argentina", 7446626, 118628, 31095885, 45837991),
    ("Iran", 6236567, 132152, 43680828, 85657310),
    ("Colombia", 5624520, 131437, 31355675, 51722642),
    ("Messico", 4495310, 302112, 13398630, 131030241),
    ("Poland", 4406553, 103378, 28939723, 37782394),
    ("Indonesia", 4277644, 144199, 68853446, 278002074),
    ("Ukraine", 3799382, 98843, 17332435, 43326674),
    ("Netherlands", 3680896, 21178, 21107399, 17193602),
    ("South Africa", 3568900, 93707, 21895771, 60466356),
    ("Philippines", 3324478, 53153, 26468749, 111838951)

]
```

In [1…
```
covid_stats_df = pd.DataFrame(
    data=covid_stats,
    columns=["country", "total_cases", "total_deaths", "total_tests", "population"]
)
covid_stats_df
```

Out[1…

| | country | total_cases | total_deaths | total_tests | population |
|---|---|---|---|---|---|
| **0** | USA | 69808350 | 880976 | 871652953 | 334008890 |
| **1** | India | 38218773 | 487719 | 709356830 | 1401013398 |
| **2** | Brazil | 23420861 | 621927 | 63776166 | 214899473 |
| **3** | UK | 15506750 | 152872 | 437010764 | 68438346 |
| **4** | France | 15175464 | 127869 | 216918555 | 65497250 |
| **5** | Russia | 10938261 | 324060 | 247600000 | 146031401 |
| **6** | Turkey | 10664372 | 85253 | 126263774 | 85744257 |
| **7** | Italy | 9219391 | 142205 | 159001733 | 60324090 |
| **8** | Spain | 8676916 | 91437 | 66213858 | 46782833 |
| **9** | Germany | 8262398 | 116868 | 89622218 | 84198923 |
| **10** | Argentina | 7446626 | 118628 | 31095885 | 45837991 |
| **11** | Iran | 6236567 | 132152 | 43680828 | 85657310 |
| **12** | Colombia | 5624520 | 131437 | 31355675 | 51722642 |
| **13** | Messico | 4495310 | 302112 | 13398630 | 131030241 |
| **14** | Poland | 4406553 | 103378 | 28939723 | 37782394 |
| **15** | Indonesia | 4277644 | 144199 | 68853446 | 278002074 |
| **16** | Ukraine | 3799382 | 98843 | 17332435 | 43326674 |
| **17** | Netherlands | 3680896 | 21178 | 21107399 | 17193602 |
| **18** | South Africa | 3568900 | 93707 | 21895771 | 60466356 |
| **19** | Philippines | 3324478 | 53153 | 26468749 | 111838951 |

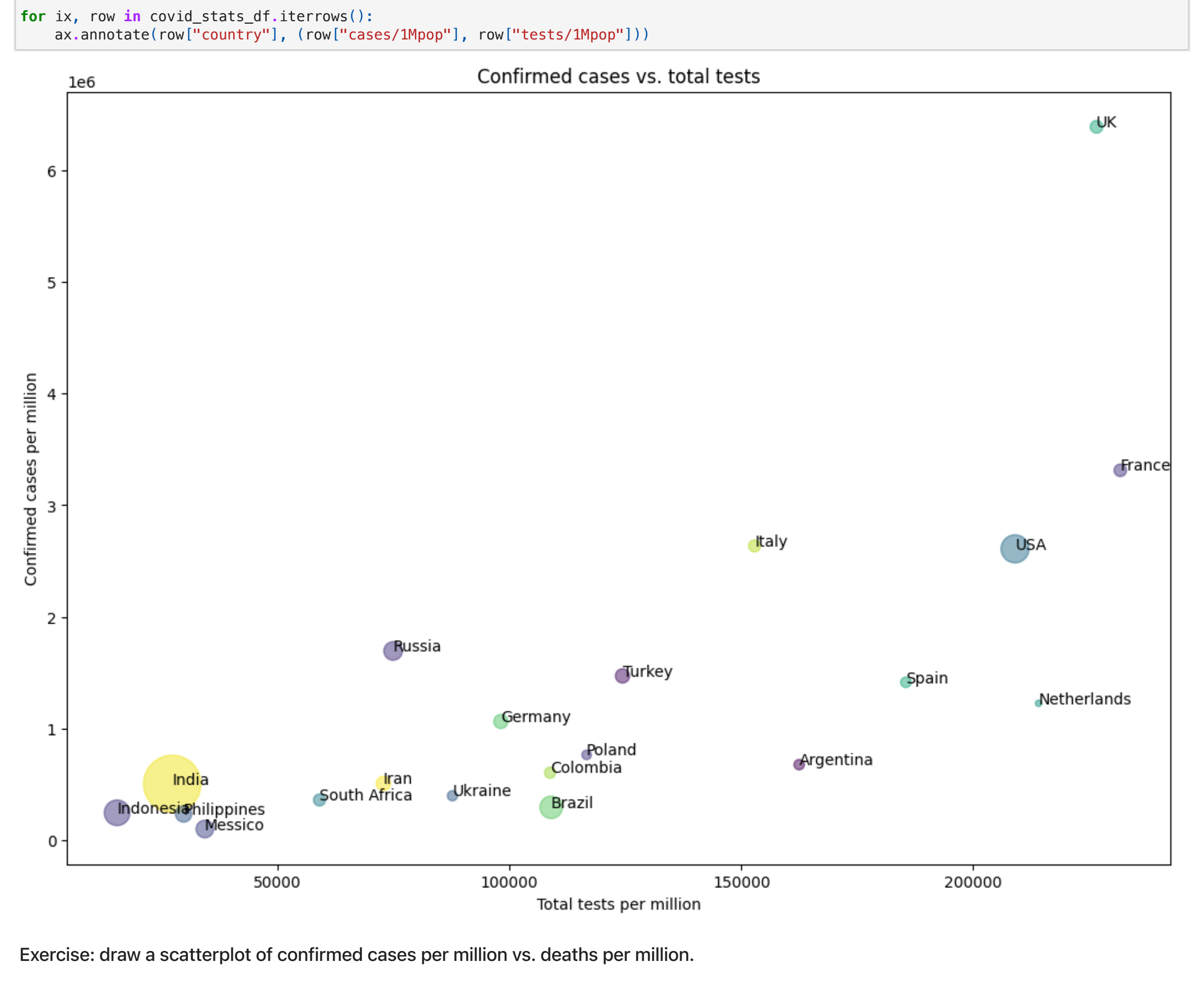
We need to normalize this data with respect to the country population. Let's do that.

In [1…
```
covid_stats_df["cases/1Mpop"] = (covid_stats_df.total_cases / covid_stats_df.population) * 1e6
covid_stats_df["tests/1Mpop"] = (covid_stats_df.total_tests / covid_stats_df.population) * 1e6
covid_stats_df["deaths/1Mpop"] = (covid_stats_df.total_deaths / covid_stats_df.population) * 1e6
covid_stats_df
```

Out[1…

| | country | total_cases | total_deaths | total_tests | population | cases/1Mpop | tests/1Mpop | deaths/1Mpop |
|---|---|---|---|---|---|---|---|---|
| **0** | USA | 69808350 | 880976 | 871652953 | 334008890 | 209001.472985 | 2.609670e+06 | 2637.582491 |
| **1** | India | 38218773 | 487719 | 709356830 | 1401013398 | 27279.377238 | 5.063169e+05 | 348.118727 |
| **2** | Brazil | 23420861 | 621927 | 63776166 | 214899473 | 108985.195138 | 2.967721e+05 | 2894.036878 |
| **3** | UK | 15506750 | 152872 | 437010764 | 68438346 | 226579.847502 | 6.385466e+06 | 2233.718506 |
| **4** | France | 15175464 | 127869 | 216918555 | 65497250 | 231696.200986 | 3.311873e+06 | 1952.280439 |
| **5** | Russia | 10938261 | 324060 | 247600000 | 146031401 | 74903.485998 | 1.695526e+06 | 2219.111765 |
| **6** | Turkey | 10664372 | 85253 | 126263774 | 85744257 | 124374.184034 | 1.472562e+06 | 994.270672 |
| **7** | Italy | 9219391 | 142205 | 159001733 | 60324090 | 152831.000020 | 2.635792e+06 | 2357.350107 |
| **8** | Spain | 8676916 | 91437 | 66213858 | 46782833 | 185472.222257 | 1.415345e+06 | 1954.498993 |
| **9** | Germany | 8262398 | 116868 | 89622218 | 84198923 | 98129.497452 | 1.064411e+06 | 1387.998751 |
| **10** | Argentina | 7446626 | 118628 | 31095885 | 45837991 | 162455.330994 | 6.783867e+05 | 2587.984277 |
| **11** | Iran | 6236567 | 132152 | 43680828 | 85657310 | 72808.345254 | 5.099486e+05 | 1542.798857 |
| **12** | Colombia | 5624520 | 131437 | 31355675 | 51722642 | 108743.865018 | 6.062272e+05 | 2541.188828 |
| **13** | Messico | 4495310 | 302112 | 13398630 | 131030241 | 34307.423734 | 1.022560e+05 | 2305.666216 |
| **14** | Poland | 4406553 | 103378 | 28939723 | 37782394 | 116629.798525 | 7.659579e+05 | 2736.142130 |
| **15** | Indonesia | 4277644 | 144199 | 68853446 | 278002074 | 15387.093839 | 2.476724e+05 | 518.697569 |
| **16** | Ukraine | 3799382 | 98843 | 17332435 | 43326674 | 87691.522317 | 4.000407e+05 | 2281.342897 |
| **17** | Netherlands | 3680896 | 21178 | 21107399 | 17193602 | 214085.216117 | 1.227631e+06 | 1231.737247 |
| **18** | South Africa | 3568900 | 93707 | 21895771 | 60466356 | 59022.905233 | 3.621149e+05 | 1549.737841 |
| **19** | Philippines | 3324478 | 53153 | 26468749 | 111838951 | 29725.582816 | 2.366684e+05 | 475.263757 |

In [1…
```
np.random.seed(42)

fig = plt.figure(figsize=(10, 7))
ax = fig.add_axes([0, 0, 1, 1])
ax.set_title("Confirmed cases vs. total tests")
ax.set_xlabel("Total tests per million")
ax.set_ylabel("Confirmed cases per million")

_ = ax.scatter(
    covid_stats_df["cases/1Mpop"],
    covid_stats_df["tests/1Mpop"],
    # "s" is the marker/dot size
    # normalise dividing by one million to display markers of reasonable size
    s=covid_stats_df["population"]/1e6,
    c=np.random.rand(len(covid_stats_df)),
    alpha=0.5 # to show overlapping dots
)

for ix, row in covid_stats_df.iterrows():
    ax.annotate(row["country"], (row["cases/1Mpop"], row["tests/1Mpop"]))
```

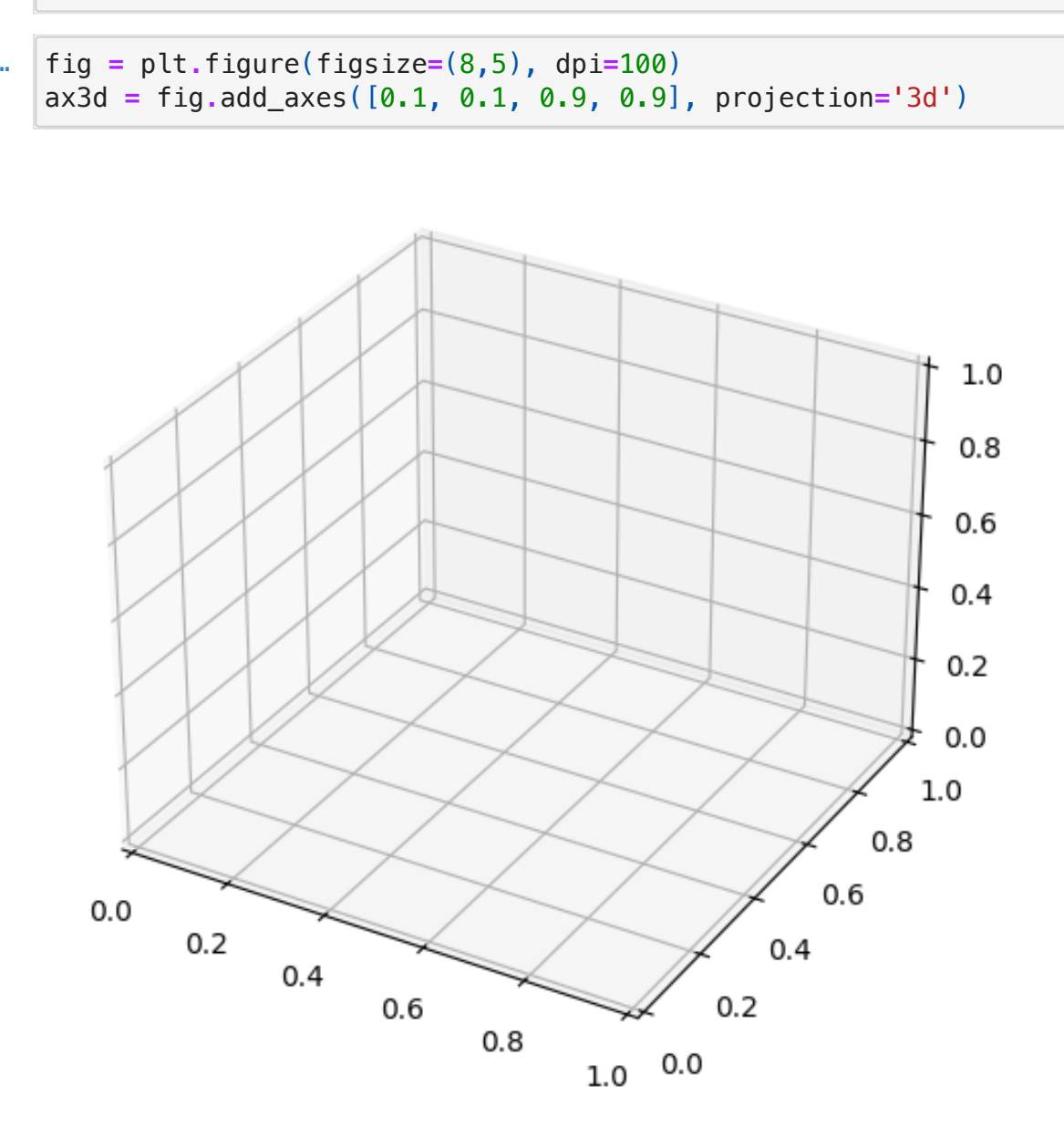


Exercise: draw a scatterplot of confirmed cases per million vs. deaths per million.

## 3D Graphs

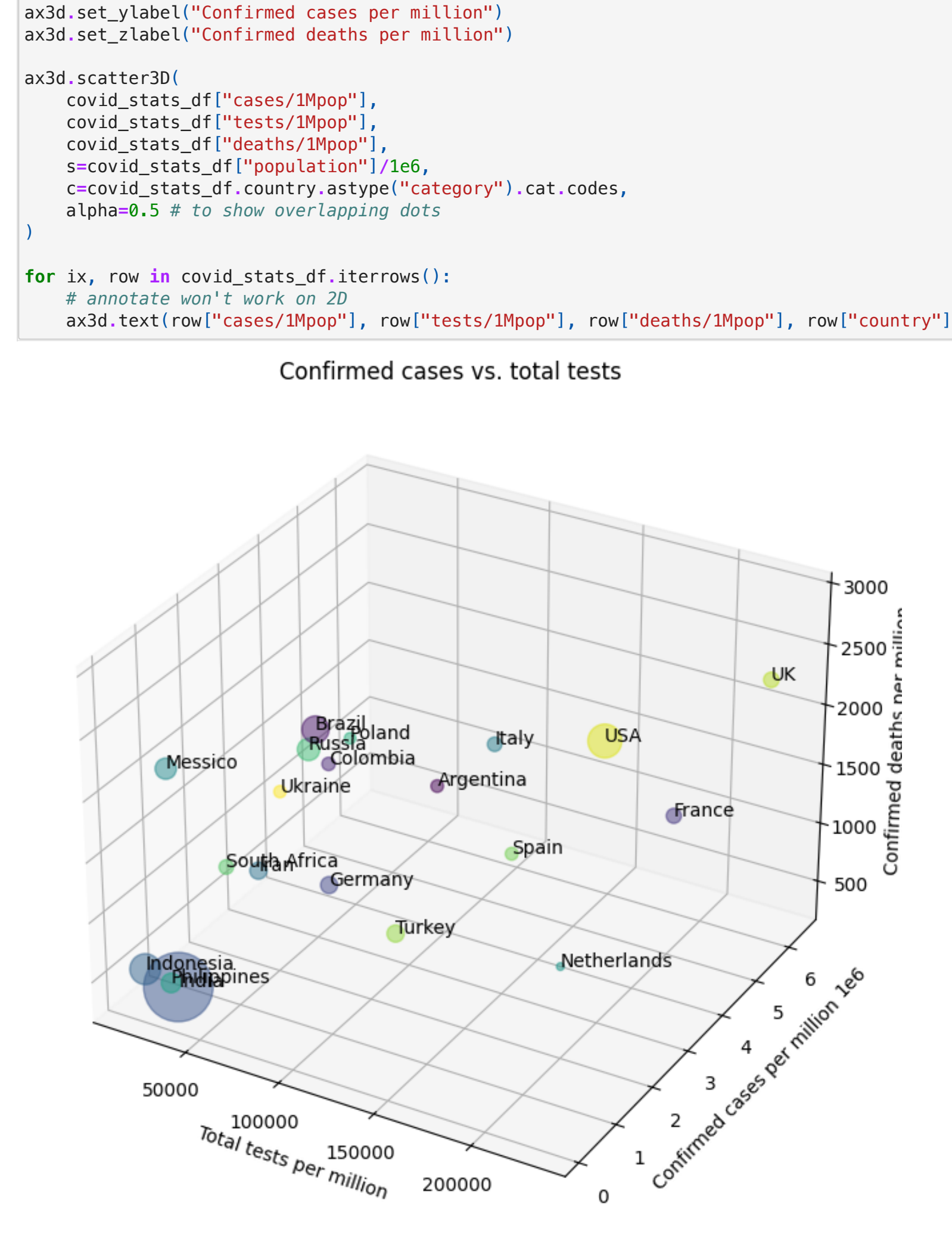
In [1…
```
from mpl_toolkits import mplot3d
```

In [1…
```
fig = plt.figure(figsize=(8,5), dpi=100)
ax3d = fig.add_axes([0.1, 0.1, 0.9, 0.9], projection='3d')
```



### 3D Scatterplot

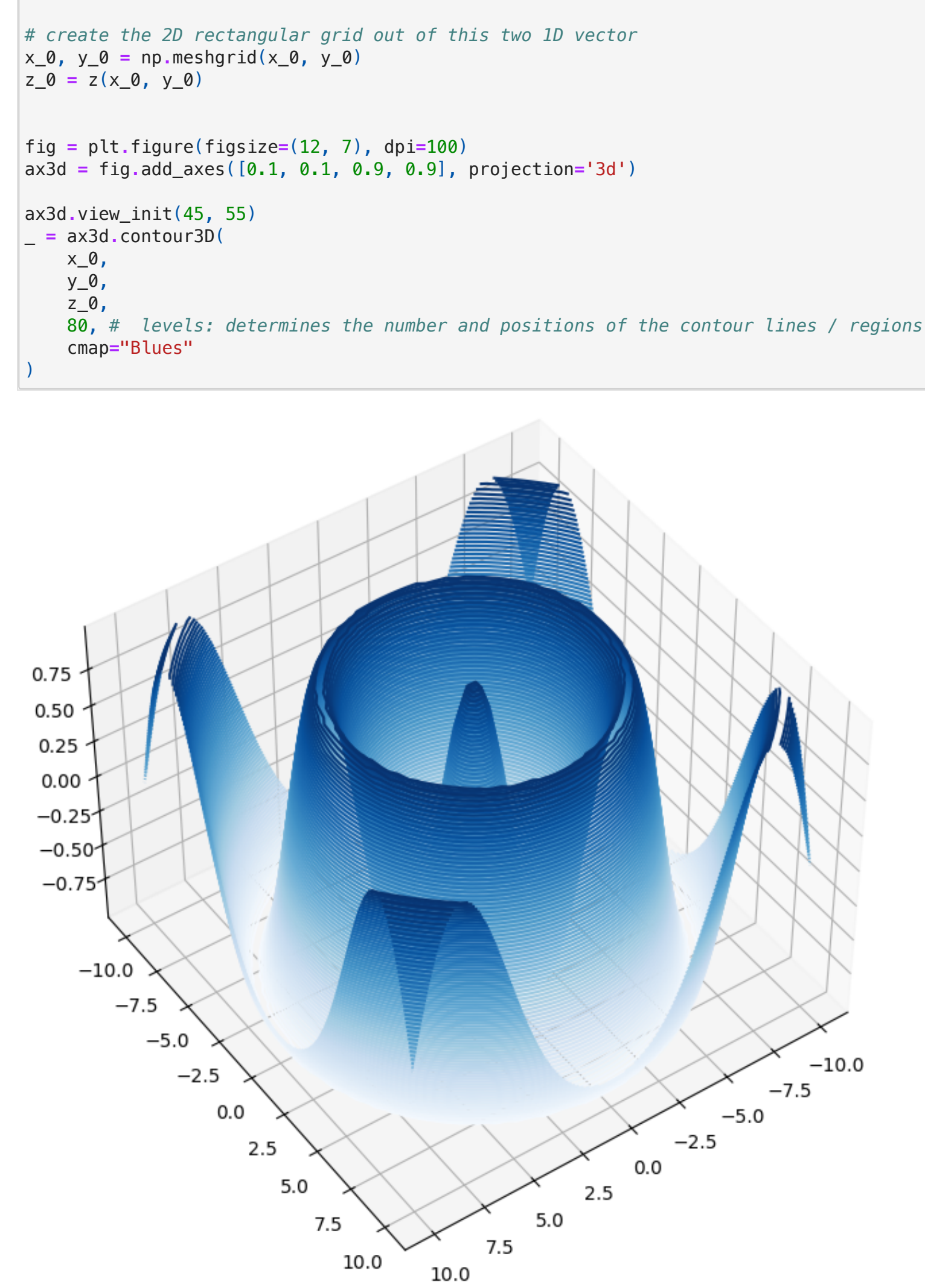
In [1…
```
fig = plt.figure(figsize=(12, 7), dpi=100)
ax3d = fig.add_axes([0.1, 0.1, 0.9, 0.9], projection='3d')

ax3d.set_title("Confirmed cases vs. total tests")
ax3d.set_xlabel("Total tests per million")
ax3d.set_ylabel("Confirmed cases per million")
ax3d.set_zlabel("Confirmed deaths per million")

ax3d.scatter3D(
    covid_stats_df["cases/1Mpop"],
    covid_stats_df["tests/1Mpop"],
    covid_stats_df["deaths/1Mpop"],
    s=covid_stats_df["population"]/1e6,
    c=covid_stats_df.country.astype("category").cat.codes,
    alpha=0.5 # to show overlapping dots
)

for ix, row in covid_stats_df.iterrows():
    # annotate won't work on 2D
    ax3d.text(row["cases/1Mpop"], row["tests/1Mpop"], row["deaths/1Mpop"], row["country"])
```



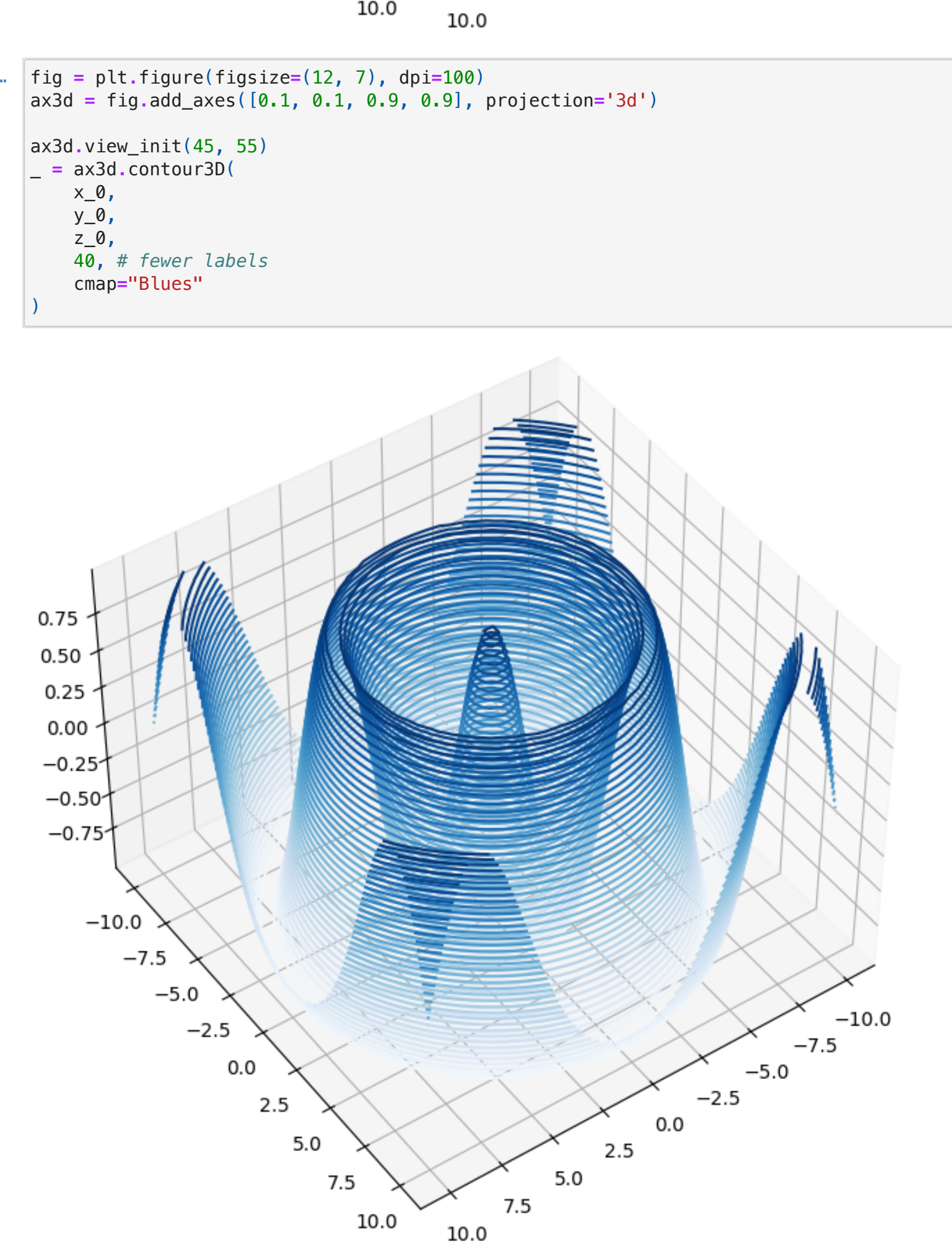
3D plots are not as easy to interpret as 2D ones. Better to project our graph to a lower dimension!

### 3D Contour plot

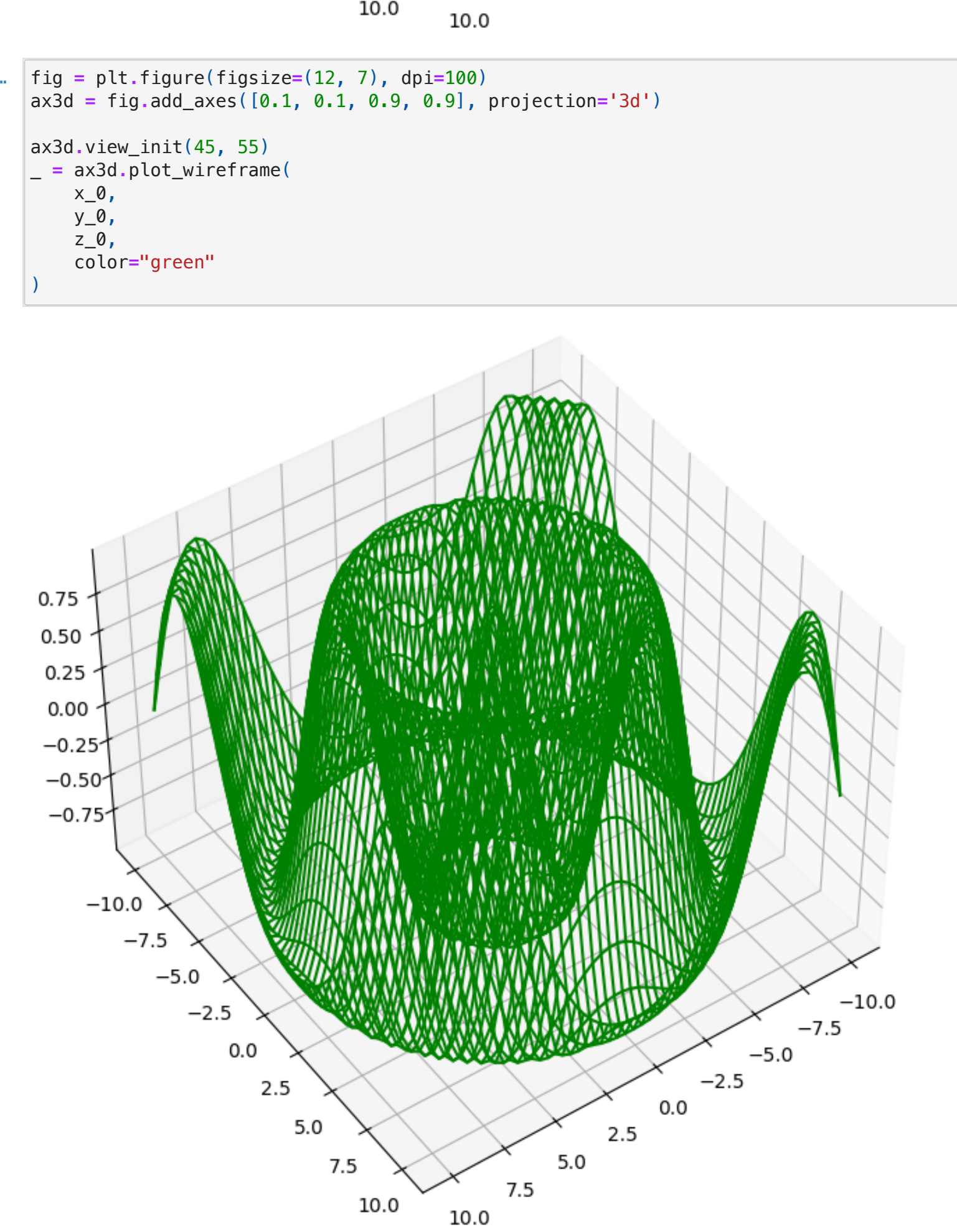
In [1…
```
def z(x, y):
    return np.cos(np.sqrt(x**2 + y**2))

x_0 = np.linspace(-10, 10, 50)
y_0 = np.linspace(-10, 10, 50)

# create the 2D rectangular grid out of this two 1D vector
x_0, y_0 = np.meshgrid(x_0, y_0)
z_0 = z(x_0, y_0)


fig = plt.figure(figsize=(12, 7), dpi=100)
ax3d = fig.add_axes([0.1, 0.1, 0.9, 0.9], projection='3d')

ax3d.view_init(45, 55)
_ = ax3d.contour3D(
    x_0,
    y_0,
    z_0,
    80, #  levels: determines the number and positions of the contour lines / regions.
    cmap="Blues"
)
```



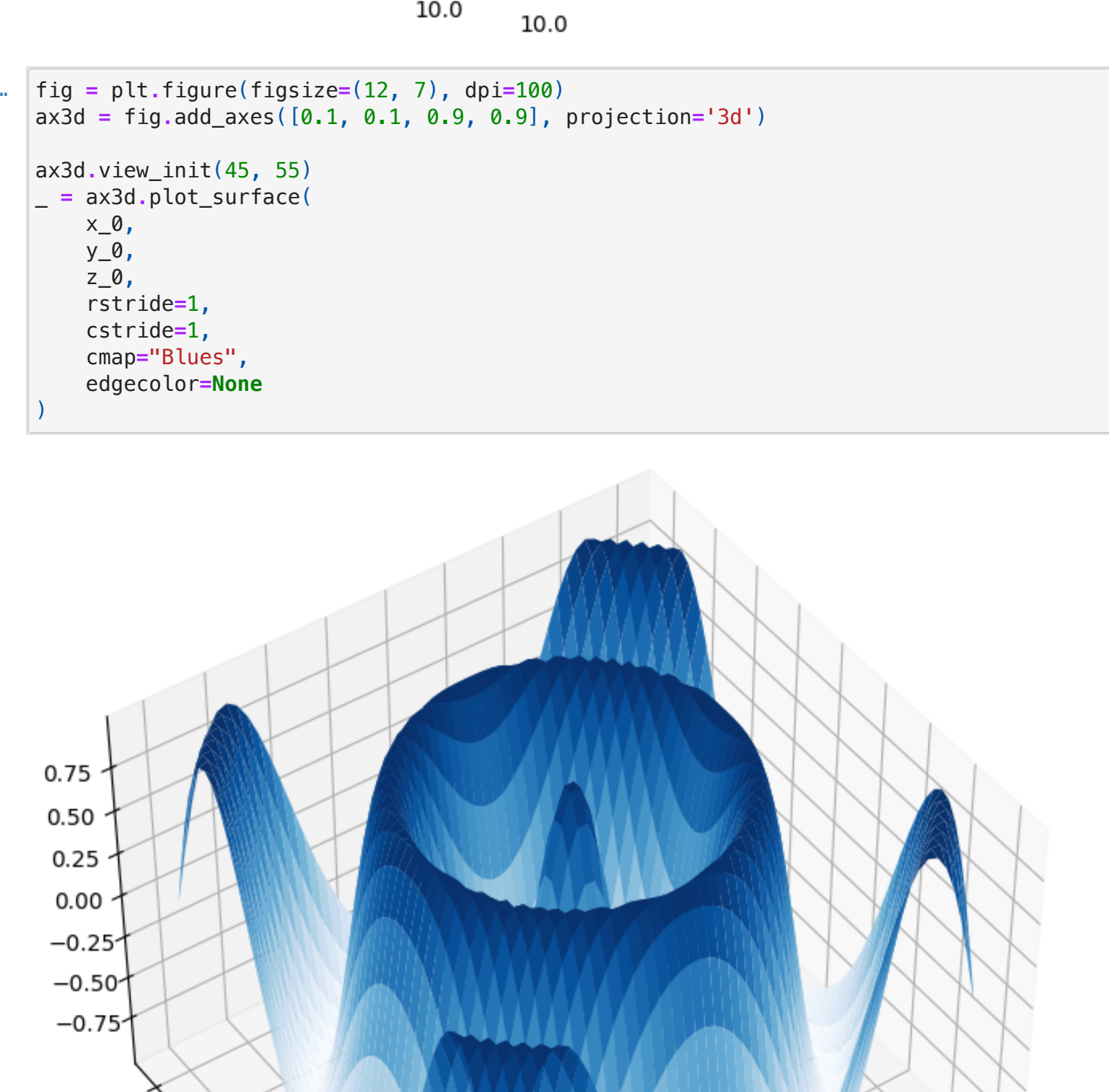
In [1…
```
fig = plt.figure(figsize=(12, 7), dpi=100)
ax3d = fig.add_axes([0.1, 0.1, 0.9, 0.9], projection='3d')

ax3d.view_init(45, 55)
_ = ax3d.contour3D(
    x_0,
    y_0,
    z_0,
    40, # fewer labels
    cmap="Blues"
)
```



In [1…
```
fig = plt.figure(figsize=(12, 7), dpi=100)
ax3d = fig.add_axes([0.1, 0.1, 0.9, 0.9], projection='3d')

ax3d.view_init(45, 55)
_ = ax3d.plot_wireframe(
    x_0,
    y_0,
    z_0,
    color="green"
)
```

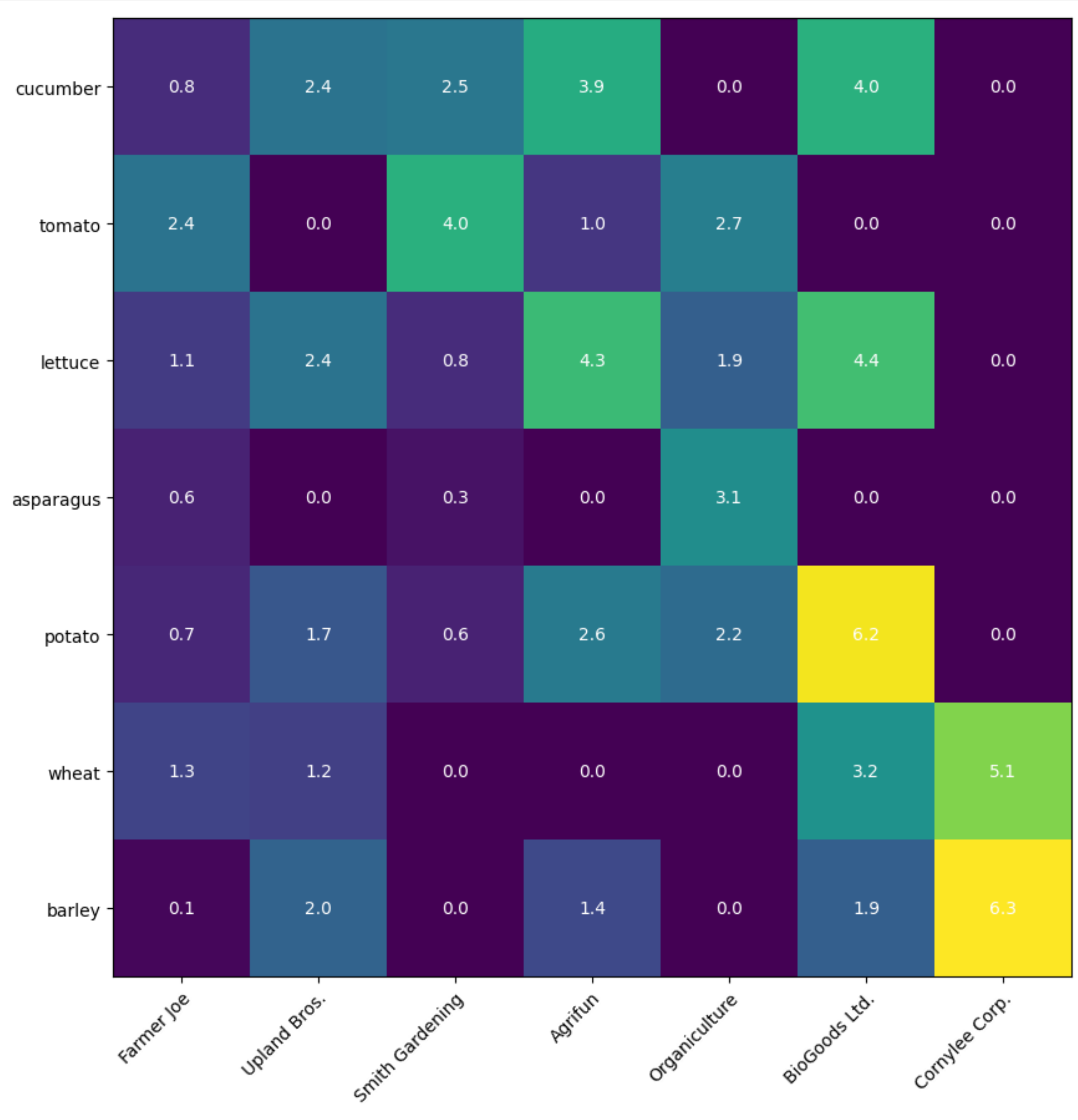


In [1…
```
fig = plt.figure(figsize=(12, 7), dpi=100)
ax3d = fig.add_axes([0.1, 0.1, 0.9, 0.9], projection='3d')

ax3d.view_init(45, 55)
_ = ax3d.plot_surface(
    x_0,
    y_0,
    z_0,
    rstride=1,
    cstride=1,
    cmap="Blues",
    edgecolor=None
)
```



## Heatmaps

In [1…
```
vegetables = ["cucumber", "tomato", "lettuce", "asparagus",
              "potato", "wheat", "barley"]
farmers = ["Farmer Joe", "Upland Bros.", "Smith Gardening",
           "Agrifun", "Organiculture", "BioGoods Ltd.", "Cornylee Corp."]

harvest = np.array([[0.8, 2.4, 2.5, 3.9, 0.0, 4.0, 0.0],
                    [2.4, 0.0, 4.0, 1.0, 2.7, 0.0, 0.0],
                    [1.1, 2.4, 0.8, 4.3, 1.9, 4.4, 0.0],
                    [0.6, 0.0, 0.3, 0.0, 3.1, 0.0, 0.0],
                    [0.7, 1.7, 0.6, 2.6, 2.2, 6.2, 0.0],
                    [1.3, 1.2, 0.0, 0.0, 0.0, 3.2, 5.1],
                    [0.1, 2.0, 0.0, 1.4, 0.0, 1.9, 6.3]])
                    [0.1, 2.0, 0.0, 1.4, 0.0, 1.9, 6.3]])

fig, ax = plt.subplots(figsize=(10, 10))
im = ax.imshow(harvest)

# Show all ticks
ax.set_xticks(np.arange(len(farmers)))
ax.set_yticks(np.arange(len(vegetables)))

# this is needed to lanel all ticks for matplotlib < 3.5
ax.set_xticklabels(farmers)
ax.set_yticklabels(vegetables)

# Rotate the tick labels and set their alignment.
# "setp" sets one or more properties on the target object(s), in our case the tick labels
# see all list of properties for TickLabels here: https://matplotlib.org/stable/api/_as_gen/mpl_toolkits.axisartist.axis_artist.TickLabe
plt.setp(
    ax.get_xticklabels(),
    rotation=45,
    ha="right",  # horizontal alignment
    rotation_mode="anchor"
)

# Loop over data dimensions and create text annotations.
for ix in range(len(vegetables)):
    for jx in range(len(farmers)):
        text = ax.text(
            jx, ix, harvest[ix, jx],
            ha="center", va="center", color="w" # horizontal & vertical alignment, colour
        )
```



This Jupyter Notebook contains exercises for you to extend your introduction to the basics with Python Matplotlib. Attempt the following exercises, which slowly build in complexity. If you get stuck, check back to the Python lecture recording on Matplotlib here or view the W3Schools page on Python Matplotlib, which includes examples, exercises and quizzes to help your understanding.

# Matplotlib Exercises (by Jose Portila)

Welcome to the exercises for reviewing matplotlib! Take your time with these, Matplotlib can be tricky to understand at first. These are relatively simple plots, but they can be hard if this is your first time with matplotlib, feel free to reference the solutions as you go along.

Also don't worry if you find the matplotlib syntax frustrating, we actually won't be using it that often throughout the course, we will switch to using seaborn and pandas built-in visualization capabilities. But, those are built-off of matplotlib, which is why it is still important to get exposure to it!

** * NOTE: ALL THE COMMANDS FOR PLOTTING A FIGURE SHOULD ALL GO IN THE SAME CELL. SEPARATING THEM OUT INTO MULTIPLE CELLS MAY CAUSE NOTHING TO SHOW UP. * **

## Exercises

Follow the instructions to recreate the plots using this data:

### Data

In [1…

```
import numpy as np
x = np.arange(0,100)
y = x*2
z = x**2
```

** Import matplotlib.pyplot as plt and set %matplotlib inline if you are using the jupyter notebook. What command do you use if you aren't using the jupyter notebook?**

In [3…

```
import matplotlib.pyplot as plt
%matplotlib inline
```

### Exercise 1

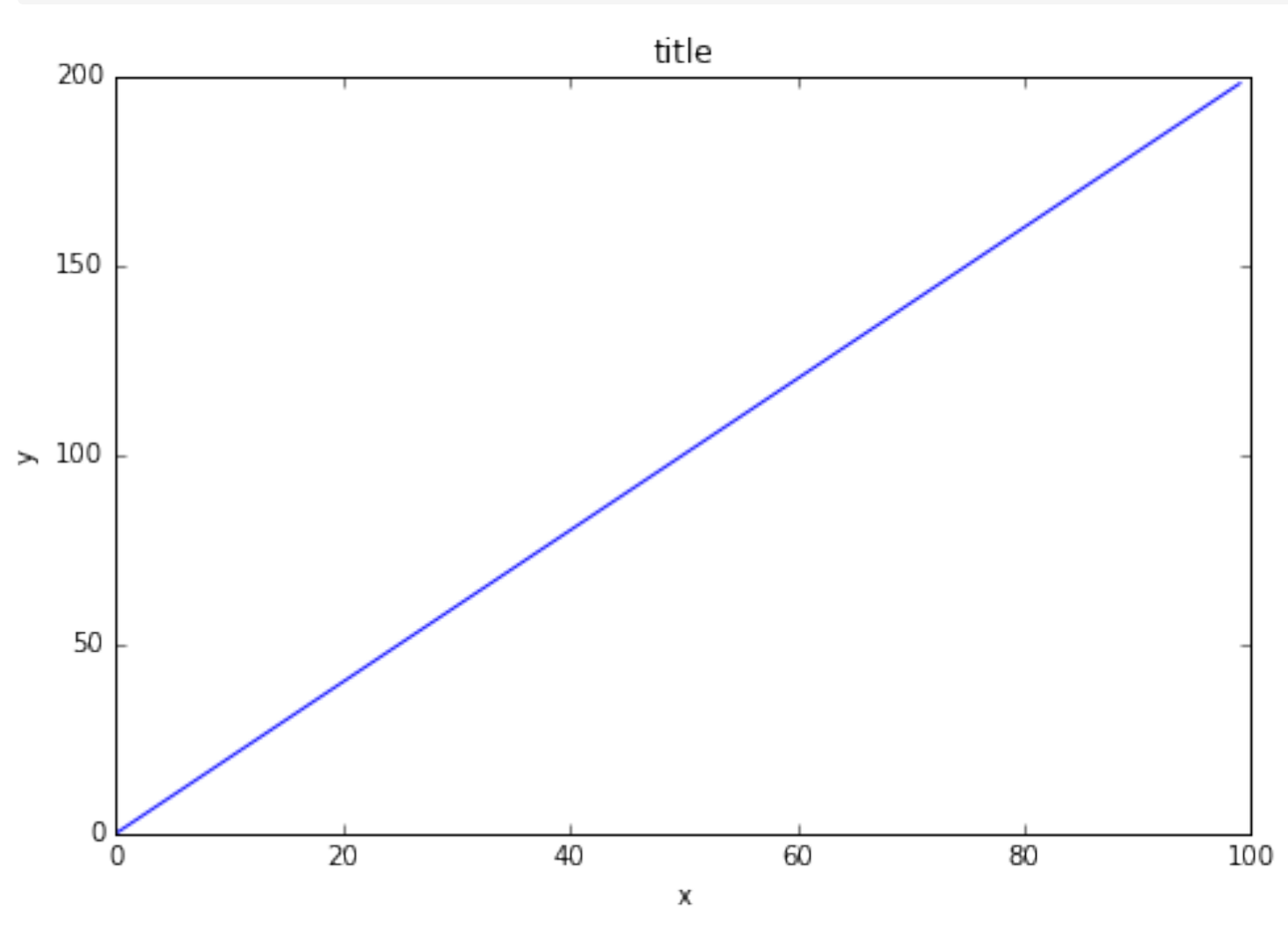
** Follow along with these steps: **

- ** Create a figure object called fig using plt.figure() **
- ** Use add_axes to add an axis to the figure canvas at [0,0,1,1]. Call this new axis ax. **
- ** Plot (x,y) on that axes and set the labels and titles to match the plot below:**

In [ …

Out[ …

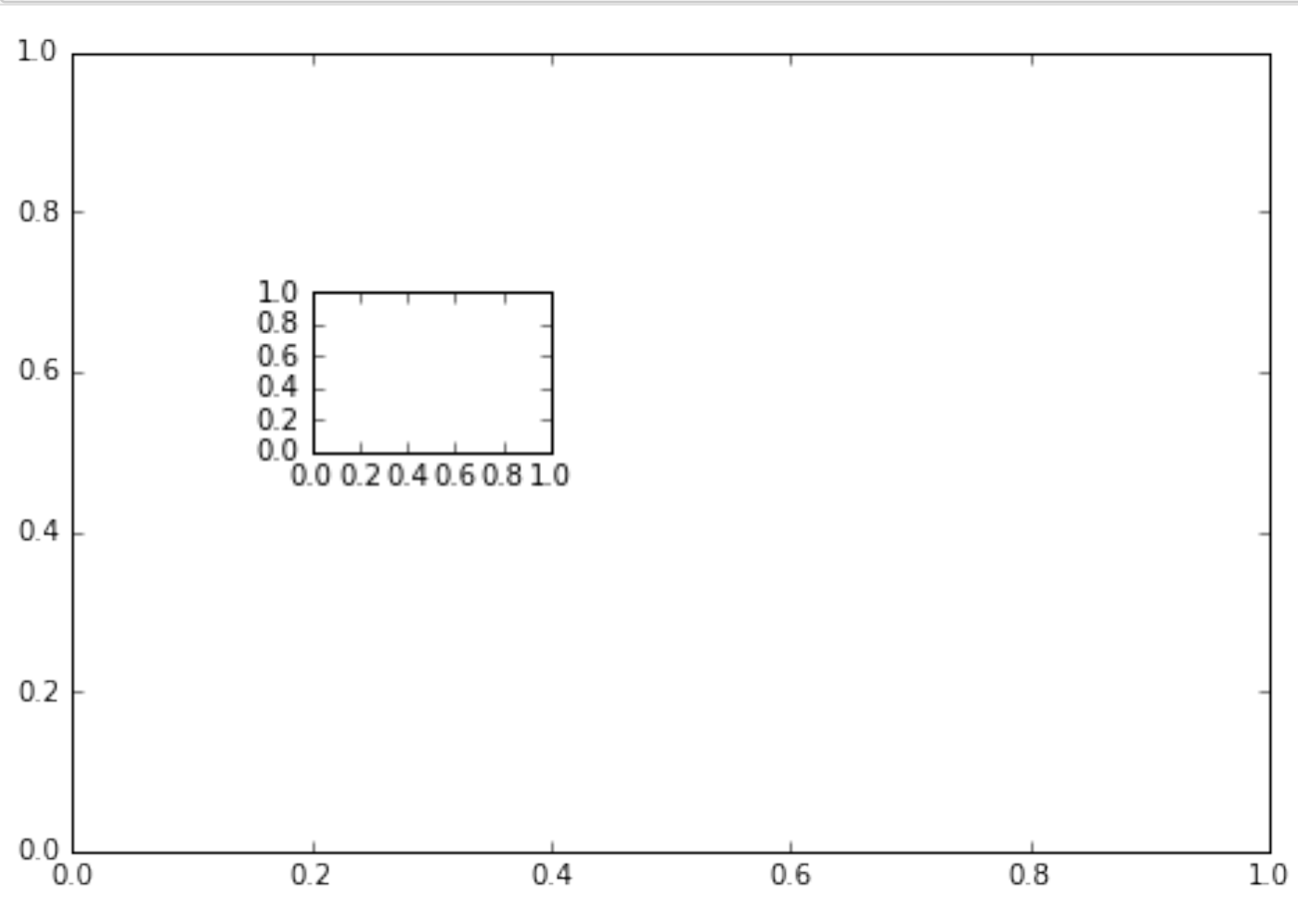
```
<matplotlib.text.Text at 0x111534c50>
```



### Exercise 2

** Create a figure object and put two axes on it, ax1 and ax2. Located at [0,0,1,1] and [0.2,0.5,.2,.2] respectively.**

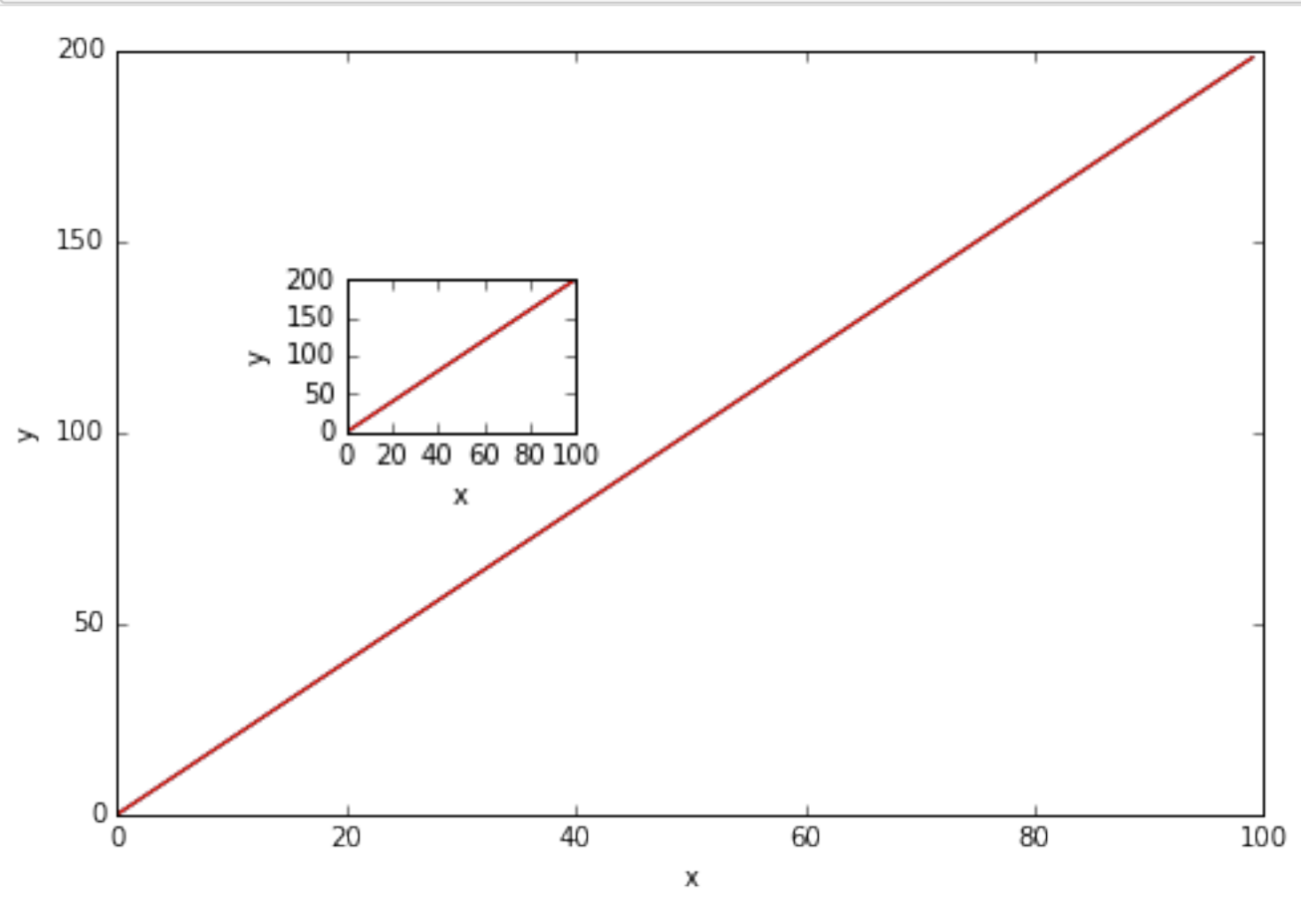
In [3…



** Now plot (x,y) on both axes. And call your figure object to show it.**

In [4…

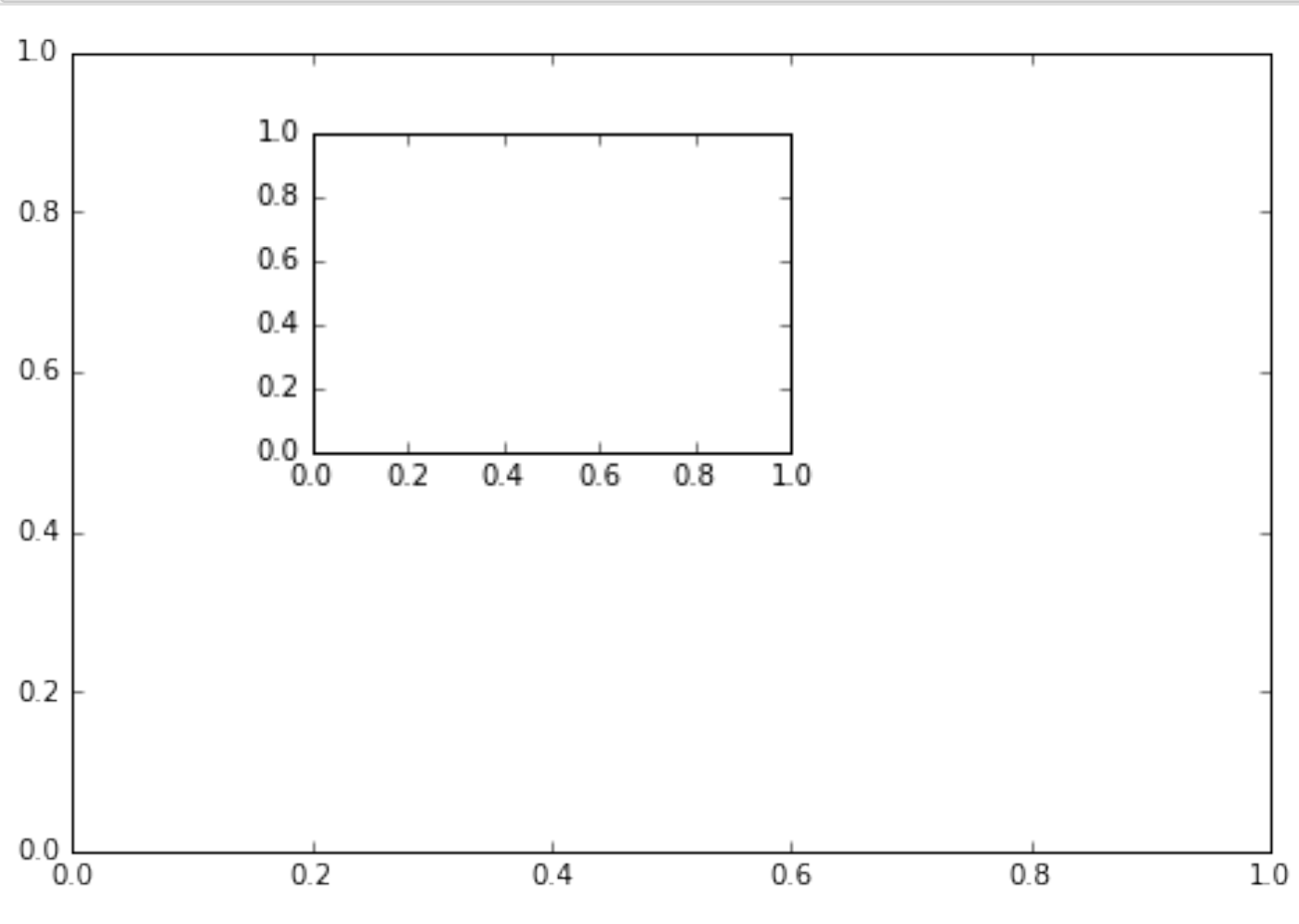
Out[4…



### Exercise 3

** Create the plot below by adding two axes to a figure object at [0,0,1,1] and [0.2,0.5,.4,.4]**

In [6…

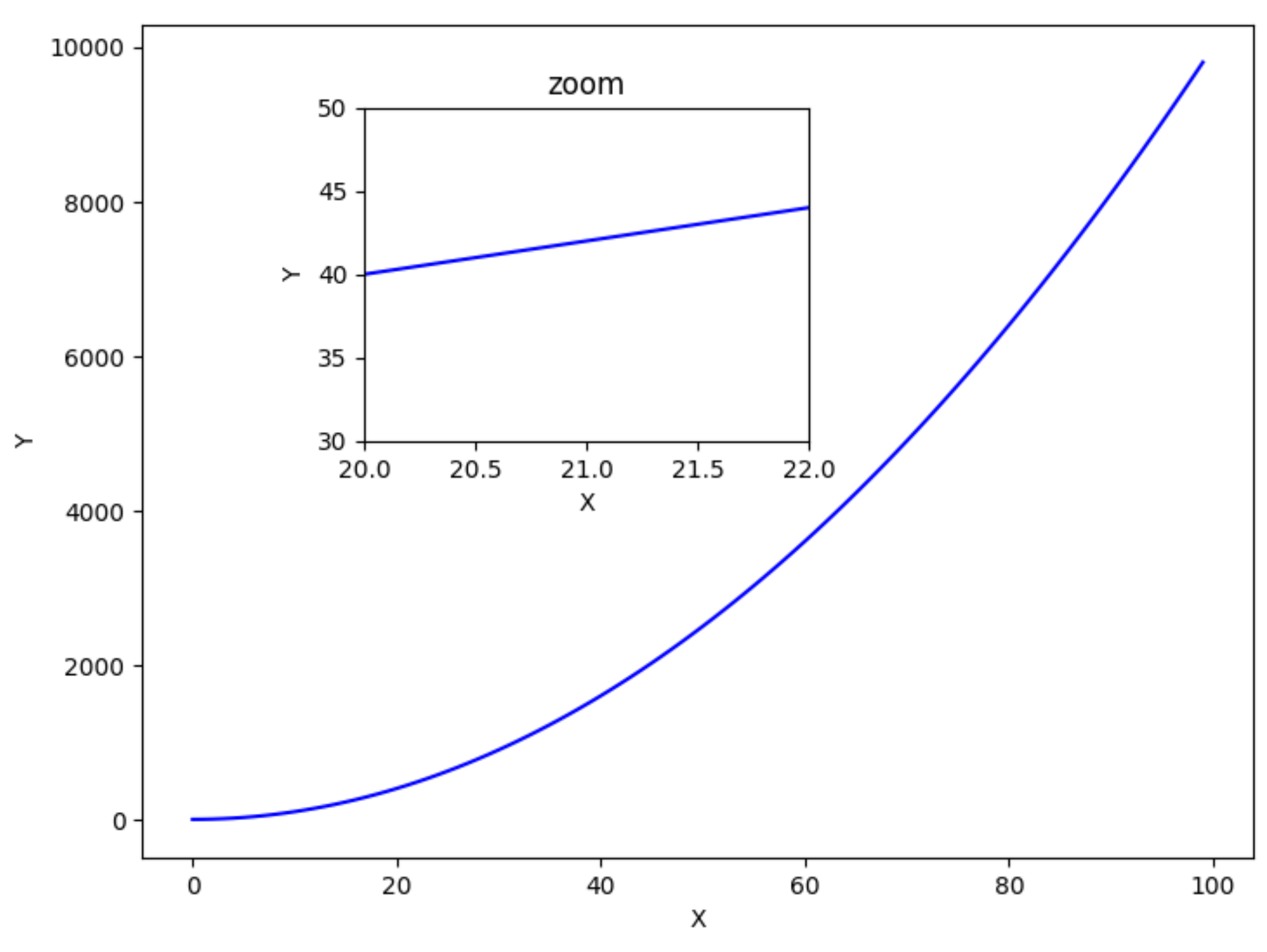


** Now use x,y, and z arrays to recreate the plot below. Notice the xlimits and y limits on the inserted plot:**

In [ …

Out[ …

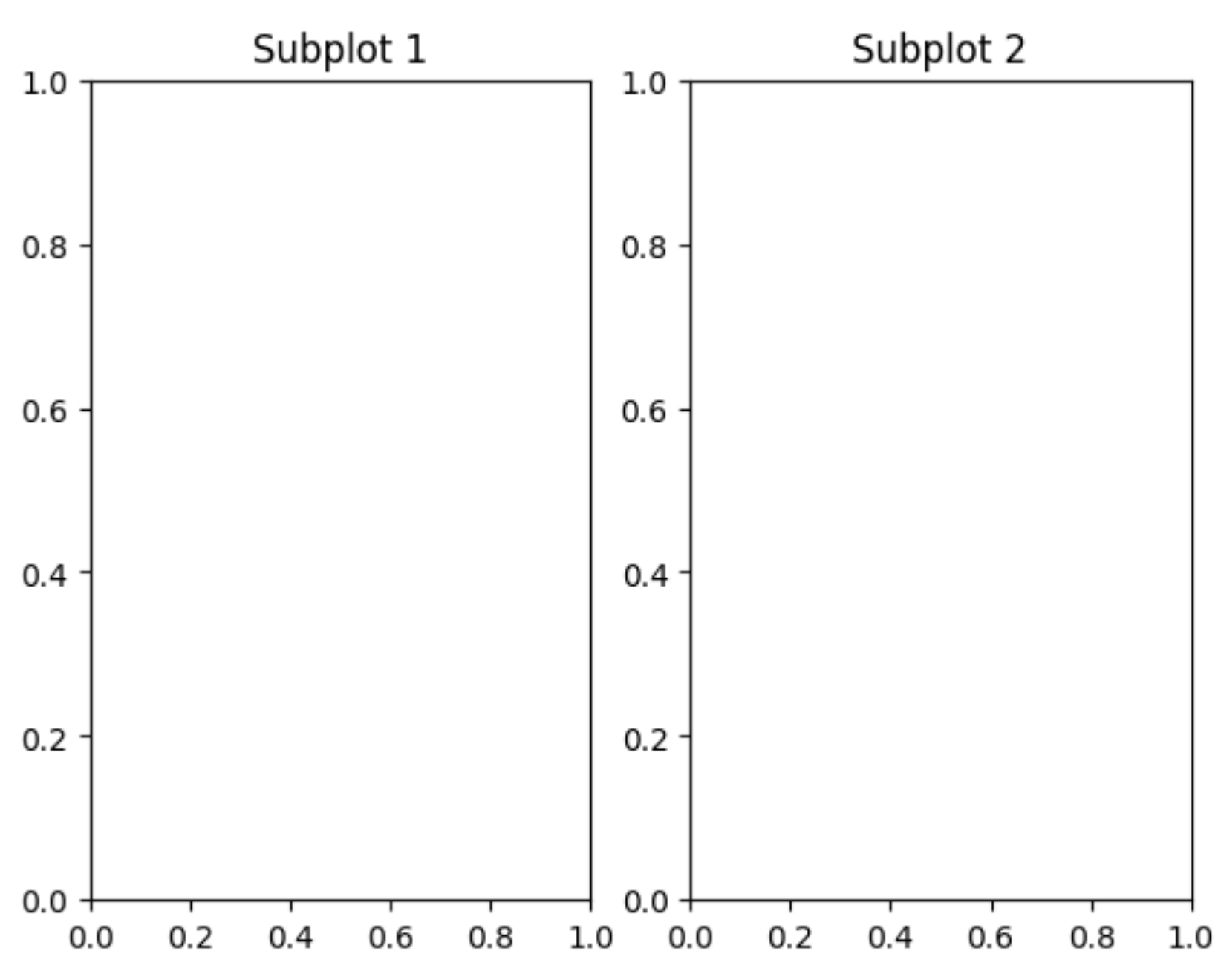
```
[<matplotlib.lines.Line2D at 0x10e0fd6d0>]
```



In [ …

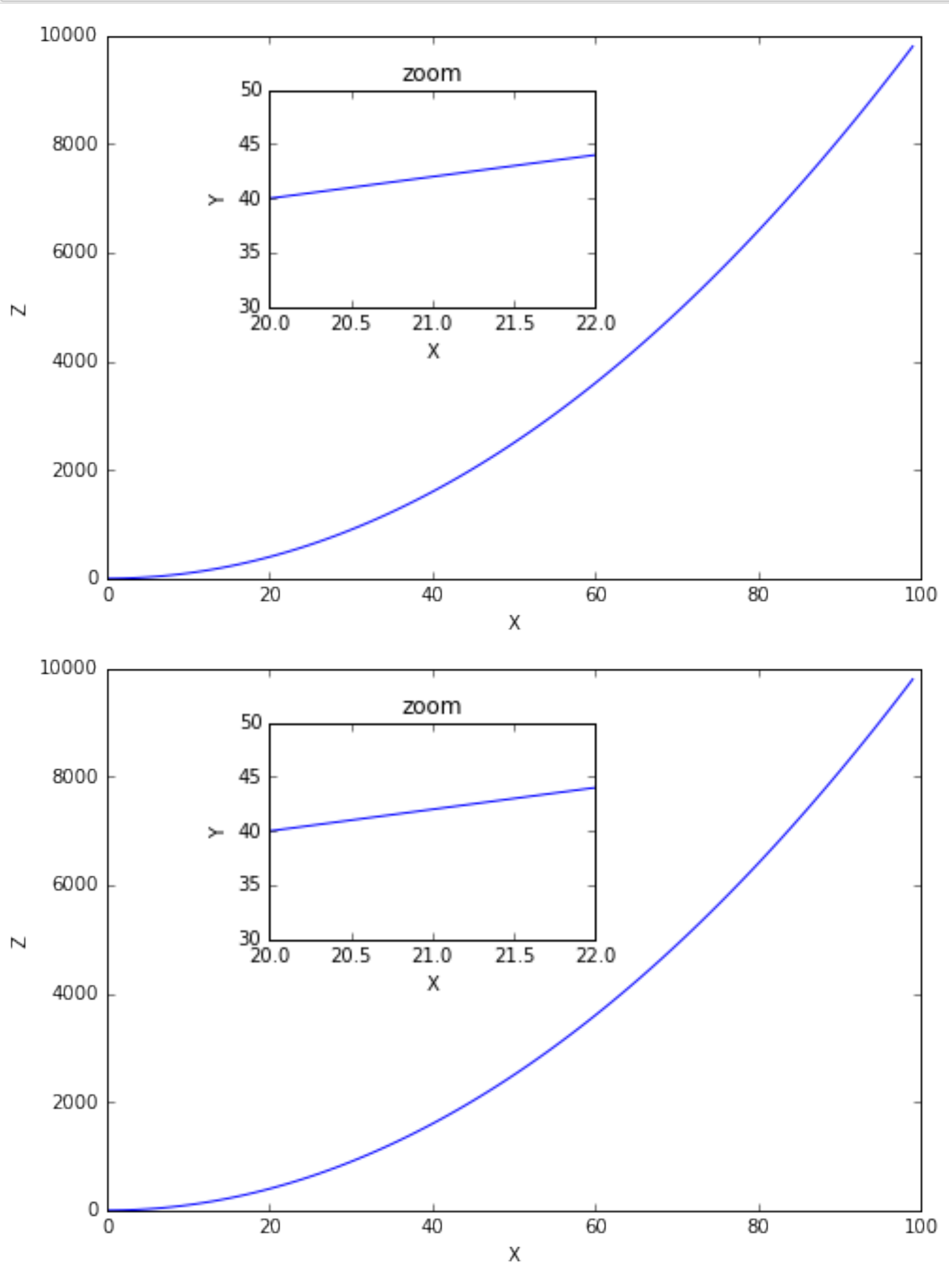
Out[ …

```
Text(0.5, 1.0, 'Subplot 2')
```



In [5…

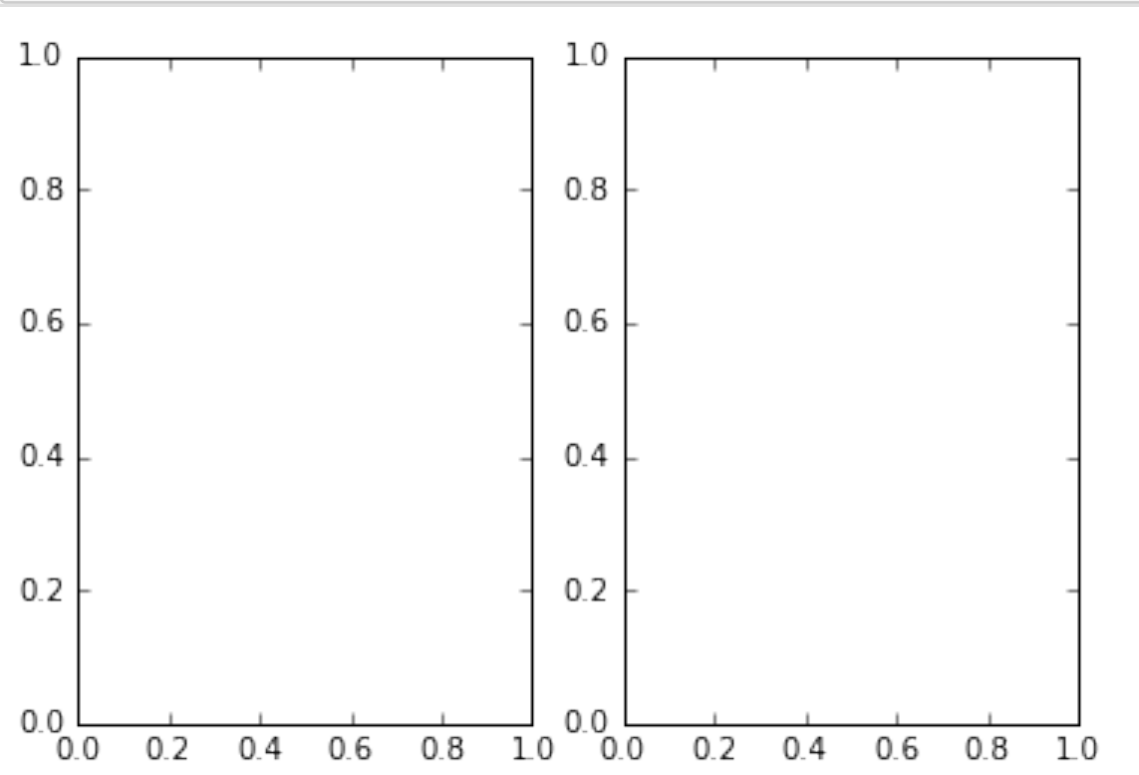
Out[5…



### Exercise 4

** Use plt.subplots(nrows=1, ncols=2) to create the plot below.**

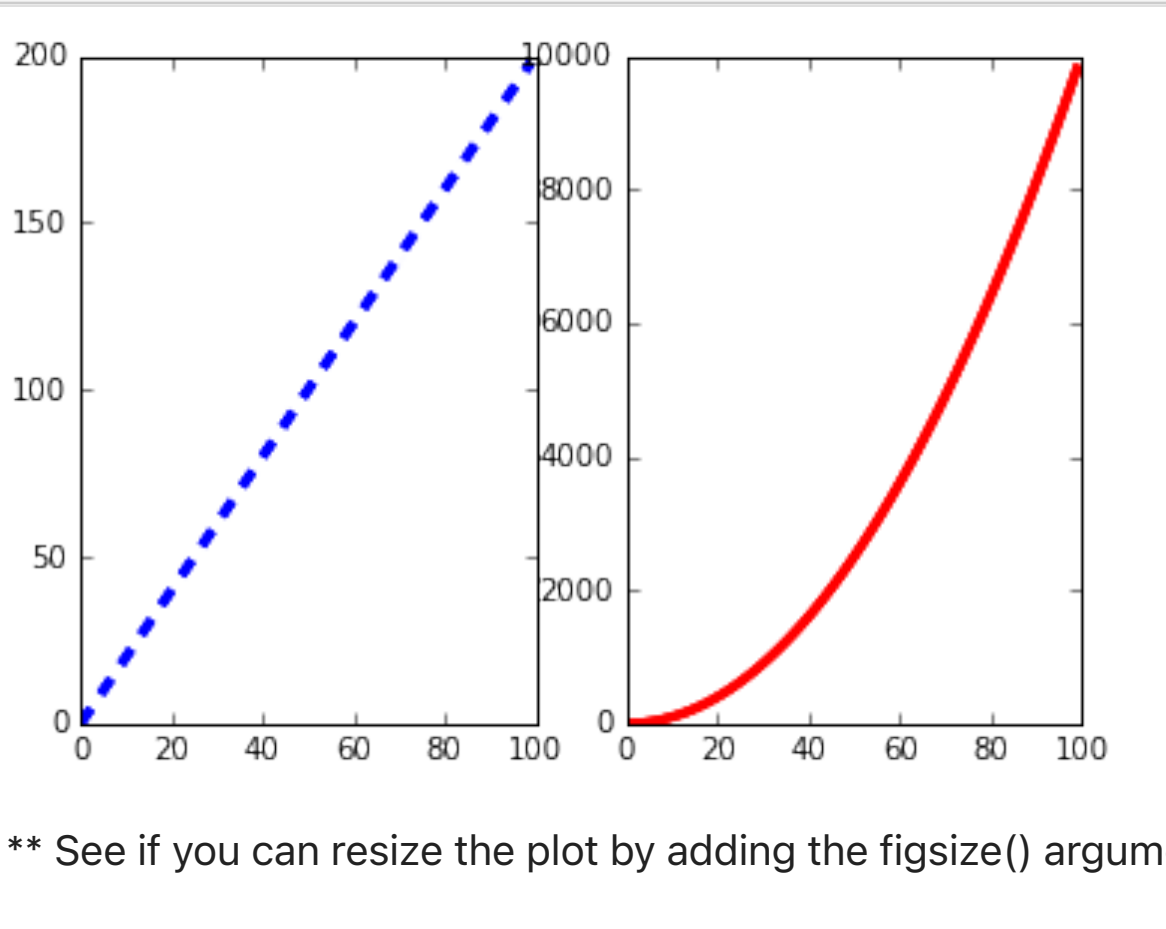
In [4…



** Now plot (x,y) and (x,z) on the axes. Play around with the linewidth and style**

In [5…

Out[5…

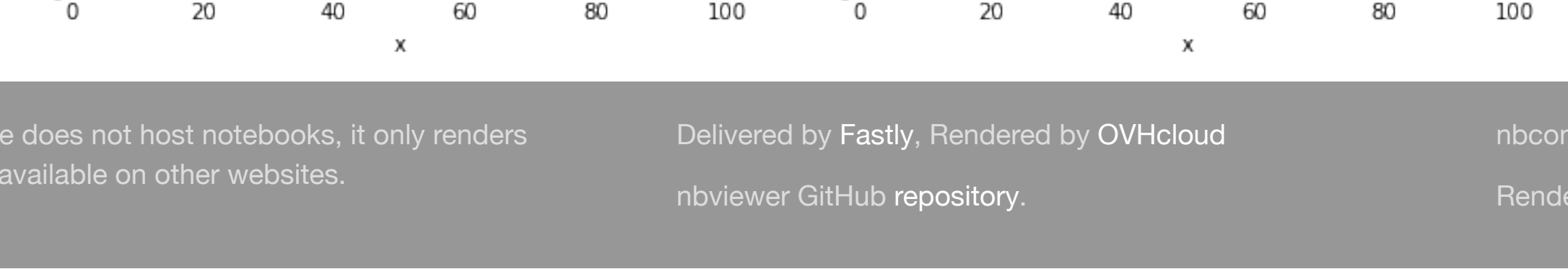


** See if you can resize the plot by adding the figsize() argument in plt.subplots() are copying and pasting your previous code.**

In [3…

Out[3…

```
<matplotlib.text.Text at 0x1141b4ba8>
```



This website does not host notebooks, it only renders notebooks available on other websites.

Delivered by Fastly, Rendered by OVHcloud

nbviewer GitHub repository.

nbconvert version: 7.16.6

Rendered a few seconds ago